# ELF

**Executable and Linkable Format** and defines the structure for binaries, libraries, and core files. The formal specification allows the operating system to interpreter its underlying machine instructions correctly. ELF files are typically the output of a compiler or linker and are a binary format. With the right tools, such file can be analyzed and better understood.

## *ELF sections*

For executable files there are four main sections: **.text**, **.data**, **.rodata**, and **.bss**. Each of these sections is loaded with different access rights, which can be seen with **readelf –S**

### .text

Contains executable code. It will be packed into a segment with read and execute access rights. It is only loaded once, as the contents will not change. This can be seen with the **objdump** utility.

### .data

Initialized data, with read/write access rights

### .rodata

Initialized data, with read access rights only (=A).

### .bss

Uninitialized data, with read/write access rights (=WA)

1.**Preprocessor** constants using **#define**

```
#define PI 22.0/7.0
#define MAX_SIZE 10
```

You can analyze the limitations and side effects of pre processor symbols.

2.**const** keyword prefix

```
const double pi=22.0/7.0
const int max_size=10;
```

As you are aware const keyword is only a modifier saying that value is not expected to be changed in program context.It's a promise only and doesn't guarantee truly constants., since const promise can be broken through various casting techniques or values can be changed outside the program scope (e.g. i/o layers).

3.C++11 introduced **constexpr** keyword to define truly constant expressions with compile time known values.

```
constexpr double pi=22.0/7.0;
constexpr int max_size=10;
```

So constexpr gives true commitment rather than a promise 😮

constexpr allows truly constant expressions (variables, functions, objects) which can be evaluated at compile time only.

constexpr improves ROMability of the code,i.e. maximum content can goes to read only memory.

constexpr variables must be initialized with compile time known values.

**Example:- constexpr** variables

```
constexpr double pi = 3.14;  //constexpr double pi ( 3.14 );
                             //constexpr double pi { 3.14 };
constexpr int maxSpeed=80;

int max=10;
constexpr int max_size=max;  //error
constexpr int max_size=10;   //ok
```

- constexpr can be prefixed to a function return type, so that the function can be evaluated at compile time if all the parameters are known at compile time.Return values of these functions can be assigned to constexpr variables.
- For functions, if any parameters are not known at compile time constexpr will be just ignored,but no error. But these functions can't be assigned to constexpr variables.
- Also normal functions (no constexpr prefix) can't be assigned to constexpr variables

**Example :-** constexpr with functions:-

```
constexpr int bitMask(int pin) {
  return 1<<pin;
}
int p1=5;
constexpr int p2=6;
int mask1=bitMask(5);               //ok
int mask2=bitMask(p1);              //ok
constexpr int mask3=bitMask(5);     //ok, evaluated at compile time
constexpr int mask3=bitMask(p1);    //error, p1 has no compile time known value
constexpr int mask5=bitMask(p2);    //ok, p2 has compile time known value
```

**Another Example:-**

```
constexpr int distanceFromOrigin(int x,int y) {
  return sqrt(x*x+y*y);
}
constexpr int cx=3;
constexpr int cy=4;
int ax=3,ay=4;

constexpr int d1=distancFromOrigin(3,4);
constexpr int d2=distancFromOrigin(ax,zy);   //error
int d3=distancFromOrigin(3,4);               //ok
constexpr int d4=distancFromOrigin(cx,cy);   //ok
```

**Read only objects:-**

constexpr can be specified with objects also to model them as read only objects, we'll discuss this in next session

**Relaxed constexpr constraints in C++14:-**

C++11 has a restriction on constexpr function that a single expression with return statement should be part of function body.However C++14 relaxes these constraints, constexpr functions may now contain conditional branches, loops.

**Ref:-** http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3597.html

The following code is not allowed C++11, due to multiple return statements,but allowed in C++14 .

```
constexpr int cmax(int x,int y) {
  if(x>y)
    return x;
  else
    return y;
}
```

all constexpr objects are const, but not all const objects are constexpr. If you want compilers to guarantee that a variable has a value that can be used in contexts requiring compile-time constants, the tool to reach for is constexpr, not const.

**Analysis:**

- Do constexpr variables are listed in **symbol table**?
- What is the **symbol state** for constexpr variables in **nm** output?
- Do constexpr variables accessible from other translation units using **external linkage**? (Justify using nm symbol state)
- constexpr variables are part of which **section** of the process? ( i.e. .data, .bss, .text, .rodata)

**Sample code:-**

```
constexpr double pi=22.0/7.0;
constexpr int maxsize=10;
constexpr double tmax=24;
```

**Try these following commands:-**

```
nm constexprdemo.o
objdump -t constexpr-demo.o
```

**Comparison:-**

- constexpr variables vs const variables vs symbolic constants
- constexpr functions vs inline functions vs macros

```cpp
#include <memory>

#define pi 3.14
constexpr int dummyConstExpr = 10;
int nonConstDummy = 20;
const int dummyTestConst = 30;
static int dummyTestStatic = 40;

int main()
{

    return 0;
}
```

**/test$ objdump -t a.out**

a.out:  file format elf64-x86-64

SYMBOL TABLE:

0000000000400200 l  d  .interp  0000000000000000  .interp

000000000040021c l  d  .note.ABI-tag  0000000000000000  .note.ABI-tag

0000000000400240 l  d  .hash  0000000000000000  .hash

0000000000400270 l  d  .dynsym  0000000000000000  .dynsym

0000000000400300 l  d  .dynstr  0000000000000000  .dynstr

00000000004003aa l  d  .gnu.version  0000000000000000  .gnu.version

00000000004003b8 l  d  .gnu.version_r  0000000000000000  .gnu.version_r

00000000004003d8 l  d  .rela.dyn  0000000000000000  .rela.dyn

00000000004003f0 l  d  .rela.plt  0000000000000000  .rela.plt

0000000000400408 l  d  .init  0000000000000000  .init

0000000000400430 l  d  .plt  0000000000000000  .plt

0000000000400450 l  d  .plt.got  0000000000000000  .plt.got

0000000000400460 l  d  .text  0000000000000000  .text

00000000004005e4 l  d  .fini  0000000000000000  .fini

00000000004005f0 l  d  .rodata  0000000000000000  .rodata

0000000000400608 l  d  .eh_frame_hdr  0000000000000000  .eh_frame_hdr

0000000000400640 l  d  .eh_frame  0000000000000000  .eh_frame

0000000000600738 l d .init_array 0000000000000000 .init_array

0000000000600740 l d .fini_array 0000000000000000 .fini_array

0000000000600748 l d .jcr 0000000000000000 .jcr

0000000000600750 l d .dynamic 0000000000000000 .dynamic

0000000000600950 l d .got 0000000000000000 .got

0000000000600958 l d .got.plt 0000000000000000 .got.plt

0000000000600978 l d .data 0000000000000000 .data

0000000000600990 l d .bss 0000000000000000 .bss

0000000000000000 l d .comment 0000000000000000 .comment

0000000000000000 l df *ABS* 0000000000000000 crtstuff.c

0000000000600748 l O .jcr 0000000000000000 __JCR_LIST__

0000000000400490 l F .text 0000000000000000 deregister_tm_clones

00000000004004d0 l F .text 0000000000000000 register_tm_clones

0000000000400510 l F .text 0000000000000000 __do_global_dtors_aux

0000000000600990 l O .bss 0000000000000001 completed.6943

0000000000600740 l O .fini_array 0000000000000000 __do_global_dtors_aux_fini_array_entry

0000000000400530 l F .text 0000000000000000 frame_dummy

0000000000600738 l O .init_array 0000000000000000 __frame_dummy_init_array_entry

0000000000000000 l df *ABS* 0000000000000000 test.cpp

00000000004005f4 l O .rodata 0000000000000001 _ZStL19piecewise_construct

00000000004005f8 l O .rodata 0000000000000004 _ZN9__gnu_cxxL21__default_lock_policyE

00000000004005fc l O .rodata 0000000000000001 _ZStL13allocator_arg

00000000004005fd l O .rodata 0000000000000001 _ZStL6ignore

**0000000000400600 l O .rodata 0000000000000004 _ZL14dummyConstExpr**

**0000000000400604 l O .rodata 0000000000000004 _ZL14dummyTestConst**

**000000000060098c l O .data 0000000000000004 _ZL15dummyTestStatic**

0000000000000000 l df *ABS* 0000000000000000 crtstuff.c

0000000000400730 l O .eh_frame 0000000000000000 __FRAME_END__

0000000000600748 l O .jcr 0000000000000000 __JCR_END__

```
0000000000000000 l    df *ABS*          0000000000000000
0000000000400608 l     .eh_frame_hdr 0000000000000000          __GNU_EH_FRAME_HDR
0000000000600958 l    O .got.plt        0000000000000000          _GLOBAL_OFFSET_TABLE_
0000000000600740 l     .init_array       0000000000000000          __init_array_end
0000000000600738 l     .init_array       0000000000000000          __init_array_start
0000000000600750 l    O .dynamic        0000000000000000          _DYNAMIC
0000000000600978  w     .data  0000000000000000          data_start
00000000004005e0 g     F .text   0000000000000002          __libc_csu_fini
0000000000400460 g     F .text   000000000000002a          _start
0000000000000000  w     *UND*          0000000000000000          __gmon_start__
0000000000000000  w     *UND*          0000000000000000          _Jv_RegisterClasses
00000000004005e4 g     F .fini    0000000000000000          _fini
0000000000000000      F *UND*          0000000000000000
__libc_start_main@@GLIBC_2.2.5
0000000000000000  w     *UND*          0000000000000000          _ITM_deregisterTMCloneTable
00000000004005f0 g     O .rodata         0000000000000004          _IO_stdin_used
0000000000000000  w     *UND*          0000000000000000          _ITM_registerTMCloneTable
0000000000600978 g      .data   0000000000000000          __data_start
0000000000600990 g     O .data 0000000000000000          .hidden __TMC_END__
0000000000600980 g     O .data 0000000000000000          .hidden __dso_handle
0000000000400570 g     F .text   0000000000000065          __libc_csu_init
0000000000600990 g      .bss    0000000000000000          __bss_start
0000000000600988 g     O .data 0000000000000004          nonConstDummy
0000000000600998 g      .bss    0000000000000000          _end
0000000000600990 g      .data   0000000000000000          _edata
0000000000400556 g     F .text   000000000000000b          main
0000000000400408 g     F .init   0000000000000000          _init


ashwini@hildzl69043:~/test$
ashwini@hildzl69043:~/test$ objdump -t a.out^C
```

ashwini@hildzl69043:~/test$ nm a.out ^C

**ashwini@hildzl69043:~/test$ nm a.out**

0000000000600990 B __bss_start

0000000000600990 b completed.6943

0000000000600978 D __data_start

0000000000600978 W data_start

0000000000400490 t deregister_tm_clones

0000000000400510 t __do_global_dtors_aux

0000000000600740 t __do_global_dtors_aux_fini_array_entry

0000000000600980 D __dso_handle

0000000000600750 d _DYNAMIC

0000000000600990 D _edata

0000000000600998 B _end

00000000004005e4 T _fini

0000000000400530 t frame_dummy

0000000000600738 t __frame_dummy_init_array_entry

0000000000400730 r __FRAME_END__

0000000000600958 d _GLOBAL_OFFSET_TABLE_

w __gmon_start__

0000000000400608 r __GNU_EH_FRAME_HDR

0000000000400408 T _init

0000000000600740 t __init_array_end

0000000000600738 t __init_array_start

00000000004005f0 R _IO_stdin_used

w _ITM_deregisterTMCloneTable

w _ITM_registerTMCloneTable

0000000000600748 d __JCR_END__

0000000000600748 d __JCR_LIST__

w _Jv_RegisterClasses

00000000004005e0 T __libc_csu_fini

0000000000400570 T __libc_csu_init

U __libc_start_main@@GLIBC_2.2.5

0000000000400556 T main

**0000000000600988 D nonConstDummy**

00000000004004d0 t register_tm_clones

0000000000400460 T _start

0000000000600990 D __TMC_END__

**0000000000400600 r _ZL14dummyConstExpr**

**0000000000400604 r _ZL14dummyTestConst**

**000000000060098c d _ZL15dummyTestStatic**

00000000004005f8 r _ZN9__gnu_cxxL21__default_lock_policyE

00000000004005fc r _ZStL13allocator_arg

00000000004005f4 r _ZStL19piecewise_construct

00000000004005fd r _ZStL6ignore

- Do constexpr variables are listed in **symbol table**?

  Ans: Yes

- What is the **symbol state** for constexpr variables in **nm** output?

  Ans: readonly: .rodata

  **0000000000400600 r _ZL14dummyConstExpr**

  **0000000000400604 r _ZL14dummyTestConst**

- Do constexpr variables accessible from other translation units using **external linkage**? (Justify using nm symbol state)

  Ans: A function declared `constexpr` is implicitly an inline function. Inline const variables at namespace scope have external linkage by default

- constexpr variables are part of which **section** of the process? ( i.e. .data, .bss, .text, .rodata)

  **Ans:** .rodata

**References:-**

https://en.cppreference.com/w/cpp/language/constexpr

https://www.learncpp.com/cpp-tutorial/2-9-symbolic-constants-and-the-const-keyword/

https://www.geeksforgeeks.org/understanding-constexper-specifier-in-c/
**Auto**

**Usage of auto keyword:-**

- Prior to C++11, auto keyword is applicable to local variables with automatic storage class and is optional (implicit), so it's seldom used.
- From C++11 auto keyword has great usage.
- It can be used for type inference of variables based on assignment or argument binding.
- Function return type can be specified with auto keyword with trailing return syntax
- Function parameters and return type can be specified with auto keyword from C++14

**Examples:-**

```
auto x=10;
auto res=compute(args);
//print typeid(res).name()
auto ptr=&val;

auto amax(int x,int y)->int {   //trailing return syntax,allowed in C++11
  return x>y?x:y;
}

auto amax(auto x,auto y) {     //allowed in C++14,but not in C++11
  return x>y?x:y;
}

auto iter=v1.begin();          //std::vector<int>::iterator iter=v1.begin();
```

We'll see the some more usage of auto keyword in further sessions

- Declaring loop variables in range based for loops
- Declaring iterators for STL containers
- Parameters of lambda expressions (from C++14)
- Declaring callbacks or callable elements (which can bind to function pointers, function objects, bind expressions, lambdas etc)

**Usage of decltype keyword:-**

decltype can be used to create a variable with type inference of available expression

**Example:-**

```
decltype(x) y;
decltype(sum(a,b)) c;

struct A {
  int x;
  int y;
};
struct A *ptr;
decltype(ptr->x) res;

auto sum(int x,int y)->decltype(x+y) {       //in C++11,not required in C++14
  return x+y;
}
```

**Explore Further:-**

Using type traits transformed types with decltype, you may analyze following example on understanding type traits.

```
std::vector<int> v1;
decltype(v1.at(0)) sum;      //decltype inferred a reference type
std::remove_reference<decltype(v1.at(0)>::type sum; //normal variable

std::list<decltype(v1.at(0)> l1; //list of references
std::list<decltype(std::remove_reference<decltype(v1.at(0)>> l2; //normal
variables
```

# <type_traits>

**type_traits**

This header defines a series of classes to obtain type information on compile-time.

**Examples**

## Classes

### Helper Classes

| | |
|---|---|
| integral_constant (C++11)<br>bool_constant (C++17) | compile-time constant of specified type with specified value<br>(class template) |
| true_type | std::integral_constant<bool, true> |
| false_type | std::integral_constant<bool, false> |

### Primary type categories

| | |
|---|---|
| is_void (C++11) | checks if a type is void<br>(class template) |
| is_null_pointer (C++14) | checks if a type is std::nullptr_t<br>(class template) |
| is_integral (C++11) | checks if a type is integral type<br>(class template) |
| is_floating_point (C++11) | checks if a type is floating-point type<br>(class template) |
| is_array (C++11) | checks if a type is an array type<br>(class template) |
| is_enum (C++11) | checks if a type is an enumeration type<br>(class template) |
| is_union (C++11) | checks if a type is an union type<br>(class template) |
| is_class (C++11) | checks if a type is a non-union class type<br>(class template) |

We can view the inferred types using auto keyword in following ways.

Let's consider a small snippet for this purpose

```
std::vector<Sample> samples;
auto ptr=samples.data();
auto x=samples.at(0);
```

**Method 1:-** using typeid operator

```
typeid(ptr).name()
typeid(x).name()
```

**Method 2:-** with the help of IDEs

In IDEs like eclipse, you may hover the mouse on auto keyword to view inferred type.

Eclipse also allows key stoke F2 to view the inferred type.

```
class Sample
{
  int i;
};

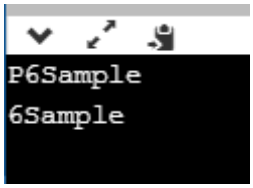
int main()
{
    Sample s;

std::vector<Sample> samples;
samples.push_back(s);
auto ptr=samples.data();
auto x=samples.at(0);

cout<<typeid(ptr).name()<<endl;
cout<<typeid(x).name()<<endl;

    return 0;
}
```

**Output:**




**Suggested Reading from Effective Modern C++, Scott Meyers**

- **Item 2 : Understanding auto type deduction**
- **Item 5 : Prefer auto to explicit type deduction**
- **Item 6: Use the explicitly typed initializer idiom when auto deduces undesired types**
- **Item 3 : Understand decltype**
- **Item 1 : Understand template type deduction**
- **Item 4 : Know how to view deduced types**

auto keyword:-

- https://en.cppreference.com/w/cpp/language/auto
- https://www.learncpp.com/cpp-tutorial/4-8-the-auto-keyword/
- https://www.geeksforgeeks.org/type-inference-in-c-auto-and-decltype/
- https://thispointer.com//c11-auto-tutorial-and-examples/

decltype:-

- https://en.cppreference.com/w/cpp/language/decltype
- https://www.geeksforgeeks.org/type-inference-in-c-auto-and-decltype/
- https://en.wikipedia.org/wiki/Decltype

## Range based for loops

Range based for loops allow convenient way of iterating through data structures(arrays,STL containers)

**Examples:-**

Traversing simple array, Loop variable can be a reference to each element in the data set

```
int arr[]={12,24,78,35,48};
int sum=0;
for(auto x:arr)    //for(int x:arr)
  sum+=x;
for(int& r:arr)    //for(auto& r:arr)
  r=r+1;
```

Traversing vector using range based for loop

```
std::vector<int> v1{12,24,78,35,48};
for(int x:v1)
  std::cout << x << "\n";
for(int& x:v1)
  x *= 0.1;
```

Loop variables can be declared with auto keyword for type inference

```
std::vector<Point> points(10);
for(auto& p:points)            //for(Point& p:points)
  p.setxy(x,y);
for(const auto& p:points)      //for(const Point& p:points)
  p.printxy();
```

- **nullptr** is a keyword added in C++11, which is of **std::nullptr_t** type
- nullptr is a better choice than symbolic constant NULL defined in stdio.h
- nullptr is implicitly compatible with any pointer type
- nullptr is not implicitly convertible to integer scalar type
- nullptr is **bool** convertible
- **type_traits** header file provide **std::is_null_pointer,std::is_pointer** to check whether the given type is **std::nullptr_t** or not

**Example:-**

```
int *ptr=nullptr;
Sample *ptr=nullptr;
ptr=new Sample;
delete ptr;
ptr=nullptr;

class MyString {
  int m_len;
  char *m_buf;
  MyString():m_len(0), m_buf(nullptr) {}
  ~MyString() {
    if(m_buf!=nullptr)     //if(nullptr)
      delete[] m_buf;
    m_buf=nullptr;
  }
};
```

Here is a code snippet on type traits usage on pointer types:-

```
int *iptr;
std::cout << std::is_null_pointer<int*>::value << "\n";                //false
std::cout << std::is_null_pointer<std::nullptr_t>::value << "\n";      //true
std::cout << std::is_null_pointer<decltype(nullptr)>::value << "\n";   //true
std::cout << std::is_null_pointer<decltype(iptr)>::value << "\n";      //false

std::cout << std::is_pointer<int*>::value << "\n";                     //true
std::cout << std::is_pointer<std::nullptr_t>::value << "\n";           //false
std::cout << std::is_pointer<decltype(nullptr)>::value << "\n";        //false
std::cout << std::is_pointer<decltype(iptr)>::value << "\n";           //true
```

## using keyword for type aliasing

- using keyword can be used to provide aliasing to existing types,similar to typedef
- aliasing can be given to templates, namespace in a convenient way

**Example code snippet:-**

```
using ivector=std::vector<int>;
using vehicle=org::kpit::ivi::examples::vehicle;
using mytype=decltype(x);
using fun_t= void(*)(int,int);

void sum(int x,int y);
using fun_t= decltype(sum);

template<typename T1, typename T2,typename T3=float>
class Sample {  };

template<typename T>
using isample=Sample<T,int,int>;           //typedef won't work here
```

**Explore Further:-**

aliasing can be given to type traits transformed types (e.g. std::remove_const, std::remove_reference etc) which is not possible with typedef.Please explore this in detail yourself,brief hints on type traits are given under additional topics.

## static_assert

- **static_assert** performs compile time check, in contrast to **assert** which does run time check
- static_assert leads to compile time error if given condition is false (as per compile time check)
- It's a better alternative to pre processor directive #error

**Examples:-**

```
static_assert( sizeof(int*) == 4, "Pointer size is not 4 bytes");
static_assert( std::is_trivial<Sample>::value, "Sample type is not a trivial
type");
```

```
template<size_t sz>
class MyBitset {
  static_assert(sz>=2, "Bitset size must be minimum 2");

}

MyBitset<3> b1;
MyBitset<1> b2;        //causes compile time error
```

```
template<typename T,size_t sz>
class MyArray {
  static_assert(sz<1, "Array size can't be less than 1");

};
MyArray<int,5> a1;    //ok
MyArray<float,-2> a2; //error
```

**Explore further:-**

- can we use normal variables or runtime evaluated expressions with static_assert?
- Do const variables,constexpr variables or constexpr expressions can be used with static_assert?

1. Can we use normal variables or runtime evaluated expressions with static_assert? : **No**

2. Do const variables,constexpr variables or constexpr expressions can be used with static_assert?

```
 8  #include <vector>
 9  #include <iostream>
10  #include <typeinfo>
11  using namespace std;
12
13  int main()
14  {
15      const int a = 1;
16      constexpr int b = 1;
17      static_assert(a==2, "a should be equal to 2");
18      static_assert(b==2, "a should be equal to 2");
19      return 0;
20  }
```

input

```
Compilation failed due to following error(s).
main.cpp: In function 'int main()':
main.cpp:18:5: error: static assertion failed: a should be equal to 2
     static_assert(a==2, "a should be equal to 2");
     ^~~~~~~~~~~~~
main.cpp:19:5: error: static assertion failed: a should be equal to 2
     static_assert(b==2, "a should be equal to 2");
```

## enum class

**Example-1:-** Let's consider following enumerations in HVAC context

```
enum FanSpeed {
  Low,
  Medium,
  High
};
enum Temperature {
  Low,                  //redeclaration error
  Medium,               //redeclaration error
  High                  //redeclaration error
};

Temperature mode=High; //compatibility error,considers High from FanSpeed
if(mode==Medium)       //considers Medium from FanSpeed
  //do something
```

In C++11 the ambiguity & re declaration errors can be resolved with strongly typed enums (Scoped enums,enum class)

```
enum class FanSpeed {
  Low     = 1 ,
  Medium  = 3,
  High    = 5
};
enum class Temperature {
  Low    = 16,
  Medium = 24,
  High   = 30
};
Temperatue tmode;
if(tmode==Temperature::Medium)
  //do something

FanSpeed fmode=Low;             //error, Low must be scoped with enum tag name
FanSpeed fmode=FanSpeed::Low;   //ok,scoped
```

**Example-2:-**

Members of scoped enums are not compatible with scalar types

```
enum DrivingModes {               //enum class DrivingModes
  Parking,
  Reverse,
  Neutral
  Driving,
};
int mode=DrivingModes::Reverse;    //allowed with normal enums
                                   //but not allowed with scoped enums

double tval=20;
if(tval < Temperature::Medium)     //not allowed with scoped enums
```

**Example-3:-**

```
enum VideoFormat {
  mp4,
  avi,
  webm
};
int webm=10;                //error with normal enums

enum class DataFormat {
  json,
  xml,
  protobuf
};
int json=10;               //allowed, as DataFormat::json is scoped
```

## String Literals

**Raw String Literals:-**

How do you store following JSON content?

```
{
  "id":"sample",
  "temperature":24,
  "humidity":72
}
```

Prior to C++11, we need to go by following clunky way

```
const char* payload= "{ \"id\":\"sample\", \n
\"temperature\":24, \n
\"humidity\":72 \n }";
```

We can do this elegantly with the help of raw string literals support added in C++11

```
const char* payload= R"(
{
  "id":"sample",
  "temperature":24,
  "humidity":72
}
");
```

Another example:-

```
const char* token="Hello\0World\nop\tuv";
const char* rtoken=R"(Hello\0World\nop\tuv)";   //escape sequences are ignored
```

```
11  using namespace std;
12
13  int main()
14  {
15     const char* payload=R"({ "id":"sample", "temperature":24, "humidity":72})";
16     std::cout << " payload:"<<payload<<std::endl;
17     const char* token="Hello\0World\nop\tuv";
18     const char* rtoken=R"(Hello\0World\nop\tuv)";   //escape sequences are ignored
19     std::cout << " token:"<<token<<std::endl;
20     std::cout << " rtoken:"<<rtoken<<std::endl;
21     return 0;
22  }
```

input

```
payload:{ "id":"sample", "temperature":24, "humidity":72}
token:Hello
rtoken:Hello\0World\nop\tuv
```

**Unicode Strings:-**

C++11 added support for unicode encodings

- new char types char16_t, char32_t to support UTF-16, UTF-32 encodings
- The literals for UTF-8, UTF-16, UTF-32 types are represented as

```
char s8[]      = "abcdxyz"          //UTF-8
char16_t s16[] = u"abcdxyz"         //UTF-16
char32_t s32[] = U"abcdxyz"         //UTF-32
char     c8    = '\u0061';
char16_t c16   = u'\u3061';
char16_t c32   = U'\u00003061';
```

- Raw string literals also can be represented in UTF-16, UTF-32 formats

```
char     rs8[]  = R"(This is a "raw UTF-8" string.)";
char16_t rs16[] = uR"(This is a "raw UTF-16" string.)";
char32_t rs32[] = UR"(This is a "raw UTF-32" string)";
```

```cpp
#include <codecvt>   // for codecvt_utf8
#include <iostream>  // cor cout
#include <locale>    // for wstring_convert
#include <string>    // for string, u16string

using namespace std;

// Compiles on LLVM 3.3 with:
// clang++ -std=c++11 -stdlib=libc++ xxx.cpp

int main() {
  u16string s = u"alpha";
  wstring_convert<codecvt_utf8<char16_t>, char16_t> cv;
  cout << cv.to_bytes(s) << endl;

  u32string s1 = U"alpha";
  wstring_convert<codecvt_utf8<char32_t>, char32_t> cv1;
  cout << cv1.to_bytes(s1) << endl;

  return 0;
}
```

input

```
alpha
alpha
```

**Comparison:-**

- char, wchar_8, char16_t, wchar32_t types

`char` is for 8-bit code units, `char16_t` is for 16-bit code units, and `char32_t` is for 32-bit code units. Any of these can be used for 'Unicode'; UTF-8 uses 8-bit code units, UTF-16 uses 16-bit code units, and UTF-32 uses 32-bit code units.

---

The guarantee made for `wchar_t` was that any character supported in a locale could be converted from `char` to `wchar_t`, and whatever representation was used for `char`, be it multiple bytes, shift codes, what have you, the `wchar_t` would be a single, distinct value. The purpose of this was that then you could manipulate `wchar_t` strings just like the simple algorithms used with ASCII.

For example, converting ascii to upper case goes like:

```
auto loc = std::locale("");

char s[] = "hello";
for (char &c : s) {
  c = toupper(c, loc);
}
```

But this won't handle converting all characters in UTF-8 to uppercase, or all characters in some other encoding like Shift-JIS. People wanted to be able to internationalize this code like so:

```
auto loc = std::locale("");

wchar_t s[] = L"hello";
for (wchar_t &c : s) {
  c = toupper(c, loc);
}
```

So every `wchar_t` is a 'character' and if it has an uppercase version then it can be directly converted. Unfortunately this doesn't really work all the time; For example there exist oddities in some languages such as the German letter ß where the uppercase version is actually the two characters SS instead of a single character.

So internationalized text handling is intrinsically harder than ASCII and cannot really be simplified in the way the designers of `wchar_t` intended. As such `wchar_t` and wide characters in general provide little value.

The only reason to use them is that they've been baked into some APIs and platforms. However, I prefer to stick to UTF-8 in my own code even when developing on such platforms, and to just convert at the API boundaries to whatever encoding is required.

### Suggested Reading from Effective Modern C++, Scott Meyers

- **Item 8 : Prefer nullptr to 0 and NULL**
- **Item 9: Prefer alias declarations to typedefs**
- **Item 10: Prefer scoped enum s to unscoped enums**

**Range based for loops:-**
https://en.cppreference.com/w/cpp/language/range-for
https://www.geeksforgeeks.org/range-based-loop-c/
https://www.learncpp.com/cpp-tutorial/6-12a-for-each-loops/
**nullptr keyword:-**
https://en.cppreference.com/w/cpp/language/nullptr
https://en.cppreference.com/w/cpp/types/nullptr_t
https://en.cppreference.com/w/cpp/types/is_null_pointer
https://www.geeksforgeeks.org/understanding-nullptr-c/
https://www.learncpp.com/cpp-tutorial/6-7a-null-pointers/
**Type Aliasing:-**
https://en.cppreference.com/w/cpp/language/type_alias
https://msdn.microsoft.com/en-us/library/dn467695.aspx
**static_assert:-**
https://en.cppreference.com/w/cpp/language/static_assert
https://www.geeksforgeeks.org/understanding-static_assert-c-11/
https://www.learncpp.com/cpp-tutorial/7-12a-assert-and-static_assert/
**Strongly Typed Enums:-**
https://en.cppreference.com/w/cpp/language/enum

https://www.learncpp.com/cpp-tutorial/4-5a-enum-classes/

**String Literals:-**

https://en.cppreference.com/w/cpp/language/string_literal

https://www.geeksforgeeks.org/raw-string-literal-c/

The following features are added in C++14.

Binary literals:-

```
int bval=0b00100101;        //0x45
uint8_t mask=0B00100000;    //0x20
auto val=0B01000110;        //0x46
```

Digit Separators:-

```
int x=1'235'659;
auto bval=0b100'1010;
auto oval=0234'724'451;
```

**References:-**

https://en.cppreference.com/w/cpp/language/integer_literal

https://isocpp.org/wiki/faq/cpp14-language#binary-literals

```
12 int main() {
13
14 int bval=0b00100101;        //0x45
15 uint8_t mask=0B00100101;
16 auto val=0B00100101;
17
18 std::cout<<"bval:"<<bval<<std::endl;
19 std::cout<<"mask:"<<mask<<std::endl;
20 std::cout<<"val:"<<val<<std::endl;
21
22 int x=1'235'659;
23 auto bval1=0b0010'0101;
24 auto oval2=0234'724'451;
25
26 std::cout<<"x:"<<x<<std::endl;
27 std::cout<<"bval1:"<<bval1<<std::endl;
28 std::cout<<"oval2:"<<oval2<<std::endl;
29
30 return 0;
31 }
32
```

```
bval:37
mask:%
val:37
x:1235659
bval1:37
oval2:41134377
```

## Inline Namespaces

- Inline namespace are applicable to nested namespaces
- Entities in nested namespace are directly accessible from enclosing namespace,if it's inline prefixed
- Inline namespace provides better support for versioning of library components

Example:-

```
namespace org {
  namespace ivi {
   inline namespace examples {
     class HVAC {  }
   }
  }
}
//Here HVAC is accessible without the need for scoping with nested namespace
examples
//i.e. org::ivi::HVAC is allowed if nested namespace examples is inline prefixed
```

Another Example:-

```
namespace org {
  namespace ivi {
    namespace examples {
      namespace v1 {
        class Cluster {}
      }
      inline namespace v2 {
        class Cluster {}
      }
    }
  }
}
//contents of v1,v2 may be provided through header files
//as illustrated in FAQs of isocpp.org

org::ivi::examples::v1::Cluster c1; //from v1
org::ivi::examples::v2::Cluster c1; //from v2
org::ivi::examples::Cluster c1;     //from v2, by default
```

**Hands-on:-**

- Check the ambiguity if a symbol is defined in both inline namespace and as well as in enclosing namespace
  - **References:-**
  - https://en.cppreference.com/w/cpp/language/namespace
  - https://www.geeksforgeeks.org/inline-namespaces-usage-using-directive-inside-namespaces/
  - https://isocpp.org/wiki/faq/cpp11-language#inline-namespace

```cpp
namespace org {
  namespace ivi {
    namespace examples {
      namespace v1 {
        constexpr int j = 1;
        class Cluster {public: const int i = 12;};
      }
      inline namespace v2 {
        constexpr int j = 2;
        class Cluster {public: const int i = 13;};
      }
    }
  }
}
int main() {
org::ivi::examples::v1::Cluster c1; //from v1
org::ivi::examples::v2::Cluster c2; //from v2
org::ivi::examples::Cluster c3;     //from v2, by default
std::cout<<"i:"<<c1.i<<" j:"<<org::ivi::examples::v1::j<<std::endl;
std::cout<<"i:"<<c2.i<<" j:"<<org::ivi::examples::v2::j<<std::endl;
std::cout<<"i:"<<c3.i<<" j:"<<org::ivi::examples::j<<std::endl;
return 0;
}
```

input

```
i:12 j:1
i:13 j:2
i:13 j:2
```

## Type Traits

Bookmark this page

Type traits defines a compile-time template-based interface to query or modify the properties of types.

Few type checks are

- std::is_null_pointer
- std::is_pod
- std::is_enum
- std::is_reference

For more listing please visit https://en.cppreference.com/w/cpp/header/type_traits

For more listing please visit https://en.cppreference.com/w/cpp/header/type_traits

**References:-**

https://en.cppreference.com/w/cpp/types

http://www.cplusplus.com/reference/type_traits/

# Default Controls

- C++11 introduced =default, =delete specifiers
- These are applicable to constructors, destructor, operator=
- =default on default constructor means, request to the compile to allow object creation with zero arguments in spite of parameterized constructors.
- =delete on default constructors, means objects objects can't be created with zero arguments (same effect if we don't write at all and parameterized constructor exists)
- =default on copy constructors has same effect as compile provided member wise copy
- =delete on copy constructors means objects can't be cloned (a better choice than private copy constructor)
- =delete on parameterized constructor means objects can't be used with such parameter combination

```
class Box {
  int l;
  int b;
  int h;
  public:
  Box()=default;          //same as Box() { }, compiler provided version
  //Box()=delete;         //restrict default constructor
  Box(int p,int q,int r):l(p),b(q),h(r) { }
  Box(const Box&)=delete;   //preventing cloning of objects
  //Box(const Box&)=default //request for compiler provided version
  ~Box()=default;
  //~Box()=delete;

  Box& operator=(const Box&)=delete;  //restricted assignment
  Box(double,double,double)=delete;

};
```

**Further Usage:-**

- Restricting parameterized constructors to prevent narrowing conversions
- Restricting overloaded assignment operators, deleted operator=
- Restricting type conversions between user defined and scalar types, deleted operator=(T), operator T()
- Restricting object creation on heap, deleted operator new

## Constructor Delegation

Bookmark this page

- With constructor delegation feature added in C++11, one constructor can invoke the other to reuse the business logic in a better way
- This feature eliminates the need for common helper function (e.g. init as private member) which is responsible for some business logic

```
class Box {
  int l;
  int b;
  int h;
 public:
  Box(int p,int q,int r):l(p),b(q),h(r) {}
  Box(int s):Box(s,s,s) {}
  Box(int p,int q):Box(p,q,10) {}
  Box(const Box& ref):Box(ref.l,ref.b,ref.h)
  //rest of the code
};
```

## Suggested Reading from Effective Modern C++, Scott Meyers

### Item 7: Prefer deleted functions to private undefined ones

**Constructors & Destructors:-**
https://thispointer.com/c11-c14-delete-keyword-and-deleted-functions-with-use-cases-examples/
https://isocpp.org/wiki/faq/cpp11-language-classes#default-delete
https://en.cppreference.com/w/cpp/language/default_constructor
https://en.cppreference.com/w/cpp/language/copy_constructor
https://en.cppreference.com/w/cpp/language/destructor
**Constructor Delegation:-**
https://isocpp.org/wiki/faq/cpp11-language-classes#delegating-ctor

## In class member initializers

Bookmark this page

- Members can be initialized within the class it self (like java)
- These values are applicable if certain member is not taken care by the constructor.
- Prior to C++11, this is applicable to **const static** members only, but now it's applicable to normal members also (except non const static variables)
- In absence of in class initlializers members will be initialized with default(garbage/zero) value if they are not taken care in any constructor

```
class Sample {
  int x=10;
  int y=20;
  int z=30;
  //int d;            //will take default if not taken care in constructor
  //static int k=0;   //still error
  const static double pi=3.14;   //allowed prior to C++11 also
  public:
  Sample(int p,int q,int r):x(p),y(q),z(r) { }
  Sample(int p,int q):x(p),y(q) { }
  Sample(int p):x(p) { }
  Sample()=default;          //Sample(){}
  void display() {
    std::cout << x << "," << y << "," << z << "\n";
  }
};
```

```
int main() {
  Sample s1(11,12,13);
  s1.display();          //11,12,13
  Sample s2(11,12);
  s2.display();          //11,12,30
  Sample s3(11);
  s3.display();          //11,20,30
  Sample s4;
  s4.display();          //10,20,30
}
```

## Uniform Initializers

Bookmark this page

- **Uniform initializations with {}**

```
class Sample {
  int x{0};
  int y{0};
  public:
  Sample(int p,int q):x{p},y{q} { }
};
class Derived:public Sample {
  int z;
  public:
  Derived(int p,int q,int r):Sample{p,q},z{r} {}
};
```

```
int volume(Box b) {

}
Box getInstance(int x,int y,int z) {
  return {x,y,z};
}

//main
Box b1{10,20,30};
Box b2={10,20,30};
Box *pb=new Box{10,20,30};
Box{10,20,30};                  //unnamed object
volume({10,20,30});
```

- Initialization with { } restricts narrowing conversions

```
//Let's consider Box constructor which takes 3 integer arguments
Box b1(10,15,8);           //ok
Box b2{10,15,8};           //ok
Box b2(2.3f,4.5f,6.8f);    //ok, implicit narrowing conversion
Box b4{2.3f,4.5f,6.8f};    //error, narrowing conversion

int x1 = 10;
int x2 ( 10 );
int x3 { 10 };

int y1 ( 2.3 );              //ok
int y2 { 2.3 };              //error
```

## Initializer List

Bookmark this page

- **std::initializer_list** allow uniform initialization of data sets underlying objects
- This is is helpful in initializing containers especially

**Examples with STL containers:-**

```
std::vector<int> v1={10,20,30,40,50};
std::list<std::string> l1{"abcd","xyz","pqr","hello"};
```

**Another example with own container:-**

```
class MyArray
{
  int *m_array;
  int m_len;
  int m_last;
  public:
  MyArray(int len):m_len(len) {
    m_arr=new int[m_len];
  }
  MyArray(const std::initializer_list<int> &list):m_len(list.size()) {
    int index=0;
    m_array=new int[m_len];
    for(auto val:list) {
      m_array[index]=val;
      index++;
    }
    m_last=index;
  }
```

```
  void fill(const std::initializer_list<int> &list) {
    //TODO: assert list.size() >= m_len
    int index=0;
    for(auto val:list)
      m_array[index++]=val;
  }
  /*void append(const std::initializer_list<int> &list) {
      //Applicable if underlying data set can be resized
  }*/
  void print()
  {
    for(int i=0;i<m_len;i++)
      std::cout << m_arr[i] << "\n";
  }
};
int main()
{
  MyArray a1{10,20,30,40,50};
  a1.print();
  a1.fill({11,12,13,14,15});
}
```

# Further Reading

Bookmark this page

## Suggested Reading from Effective Modern C++, Scott Meyers

### Item 7: Distinguish between () and {} when creating objects

**In class member initializers:-**
https://isocpp.org/wiki/faq/cpp11-language-classes#member-init
**Uniform Initializers:-**

https://isocpp.org/wiki/faq/cpp11-language#uniform-init

**Initializer List:-**

https://en.cppreference.com/w/cpp/language/list_initialization
http://www.learncpp.com/cpp-tutorial/10-7-stdinitializer_list/
https://thispointer.com/using-stdinitializer_list-in-constructors-to-initialize-member-variables/
https://thispointer.com/stdinitializer_list-tutorial-examples-c11/

## Keywords : override,final

Bookmark this page

- override,final keywords are applicable to virtual functions only
- override keyword ensures that it's a true overriding, i.e. a base class function is available with same prototype including const specifiers.
- This helps in accidental changes to parameters, const specifier in derived classes
- final keyword ensures that the virtual function is not overridden in any further derived classes.

Recall const overloading before going through examples on override,final.Observe the impact of const keyword as a suffix to function and prefix to parameters in following examples

```
class A {
  public:
    virtual void test(int);
    virtual void test(int) const;    //const overloading, allowed
    virtual void test(Sample&);
    virtual void test(const Sample&);
    virtual void test(Sample&) const;
    virtual void test(const Sample&) const;
};
```

**Example-1:-**

```
class A {
  public:
    virtual void f1(int,int);
    virtual void f2(int,int);
};
class B:public A
{
  public:
    virtual void f1(int,int) override;
    virtual void f2(int,int) override;        //ok
    virtual void f2(int,int,int) override;    //error, it's not actual overriding
};
```

**Example-2:-**

```
class A {
  public:
    virtual void f1(int,int);
    virtual void f2(int,int) const;
    virtual void f3(const Sample&);
};
class B:public A
{
  public:
    virtual void f1(int,int) override;
    virtual void f2(int,int) override;        //error,const mismatch
virtual
    void f3(Sample&) override;                //error, const mismatch
};
```

**Example-3:-**

```
class A {
  public:
    virtual void f1(int,int) final;
    virtual void f2(int,int) const final;
    virtual void f3(const Sample&) final;
};
class B:public A
{
  public:
    virtual void f1(int,int);          //error, A::f1 is final
    virtual void f2(int,int);          //ok,as there is no non const
A::f2
                                       //but avoid this
    virtual void f2(int,int) const;    //error
    virtual void f3(const Sample&);    //error
    virtual void f3(Sample&);          //ok, but avoid this
};
```

## Inheriting Constructors

Bookmark this page

- Constructors can be inherited from base class to derived class with the help of using statement
- This is different from constructor delegation from derived class to base class
- This is more meaningful, if derived class doesn't require any specific initialization.
- Normal member functions can be inherited through using statement in C++98 also, this is helpful if such function is hidden by another overloaded function in base class.

Example:-

```
class A {
  int x;
  public:
  A(int p):x(p) {}
  void f1(int);
};
```

```
class B:public A
{
  int y;
  public:
  using A::A;                    //C++11 onwards
  B(int p,int q):A(p),y(q) {
  }
  using A::f1;                   //allowed in C++98 also,adds f1(int) in scope of
B
  void f1(int,int);
};
class C:public B {
  int z;
public:
  using B::B;
  C(int p,int q,int r):B(p,q),z(r) {
  }
};
class D:public A {
public:
  using A::A;     //more meaningful
};
```

```
int main() {
  B b1(10,20);    //actual constructor
  B b2(15);       //works with inherited constructor

  C c1(10);
  C c2(12,24);
}
```

**class Sample {**

**int x;**

**int y;**

```
public:
 Sample() {std::cout<<"Sample()"<<std::endl;};
 Sample(int p,int q):x(p),y(q) {std::cout<<"Sample(int p,int q)"<<std::endl; }
 Sample(const Sample& rs) {std::cout<<"Sample(const Sample& rs)"<<std::endl; }
 Sample(Sample& rs) {std::cout<<"Sample(Sample& rs)"<<std::endl; }
 Sample(Sample&& rr) {std::cout<<"Sample(Sample&& rr)"<<std::endl; }
 Sample& operator=(const Sample&) {std::cout<<" operator=(const
Sample&)"<<std::endl; }
 Sample& operator=(Sample&&) {std::cout<<"operator=(Sample&&)"<<std::endl; }
 void printxy() const {
   std::cout << x << "," << y << "\n";
 }
 protected:
 void Foo()
 {
    std::cout<<"Foo()"<<std::endl;
 }
 virtual void FooVirtual()
 {
    std::cout<<"FooVirtual()"<<std::endl;
 }
};

class Derived: public Sample
{
  public:
  using Sample::Foo; //allowed in C++98
  using Sample::FooVirtual; //allowed in C++98

 Derived() {std::cout<<"Derived()"<<std::endl;};
```

**//Derived(int p,int q): Sample(p, q) {std::cout<<"Derived(int p,int q)"<<std::endl; }**

**using Sample::Sample; //C++ 11, as derived class does not need specific initilization but base class does**

**Derived(const Derived& rs) {std::cout<<"Derived(const Derived& rs)"<<std::endl; }**

**Derived(Derived& rs) {std::cout<<"Derived(Derived& rs)"<<std::endl; }**

**Derived(Derived&& rr) {std::cout<<"Derived(Derived&& rr)"<<std::endl; }**

**};**

```
28  class Derived: public Sample
29  {
30     public:
31     using Sample::Foo; //allowed in C++98
32     using Sample::FooVirtual; //allowed in C++98
33
34    Derived() {std::cout<<"Derived()"<<std::endl;};
35
36    //Derived(int p,int q): Sample(p, q) {std::cout<<"Derived(int p,int q)"<<std::endl; }
37    using Sample::Sample; //C++ 11, as derived class does not need specific initilization but base class does
38
39    Derived(const Derived& rs) {std::cout<<"Derived(const Derived& rs)"<<std::endl; }
40    Derived(Derived& rs) {std::cout<<"Derived(Derived& rs)"<<std::endl; }
41    Derived(Derived&& rr) {std::cout<<"Derived(Derived&& rr)"<<std::endl; }
42
43  };
44                char32_t                    keyword
45
46  int main() {
47  Derived s1(1, 2);
48  s1.Foo();
49  s1.FooVirtual();
50    return 0;
51  }
```

input

```
Sample(int p,int q)
Foo()
FooVirtual()
```

**Suggested Reading from Effective Modern C++, Scott Meyers**
**Item 12: Declare overriding functions override**
**Tutorial - override, final keywords:-**
https://www.learncpp.com/cpp-tutorial/12-2a-the-override-and-final-specifiers-and-covariant-return-types/
https://www.geeksforgeeks.org/override-keyword-c/
https://www.geeksforgeeks.org/c-final-specifier/

**Override, final keywords:-**
https://en.cppreference.com/w/cpp/language/override
https://en.cppreference.com/w/cpp/language/final
https://isocpp.org/wiki/faq/cpp11-language-classes#override
https://isocpp.org/wiki/faq/cpp11-language-classes#final
**Inheriting Constructors:-**
https://isocpp.org/wiki/faq/cpp11-language-classes#inherited-ctors

# Trivial, standard-layout, and POD types

The term *layout* refers to how the members of an object of class, struct or union type are arranged in memory. In some cases, the layout is well-defined by the language specification. But when a class or struct contains certain C++ language features such as virtual base classes, virtual functions, members with different access control, then the compiler is free to choose a layout. That layout may vary depending on what optimizations are being performed and in many cases object might not even occupy a contiguous area of memory. For example, if a class has virtual functions, all the instances of that class might share a single virtual function table. Such types are of course very useful, but they also have limitations. Because the layout is undefined they cannot be passed to programs written in other languages, such as C, and because they might be non-contiguous they cannot be reliably copied with fast low-level functions such as `memcopy` or serialized over a network.

To enable compilers as well as C++ programs and metaprograms to reason about the suitability of any given type for operations that depend on a particular memory layout, C++14 introduced three categories of simple classes and structs: *trivial*, *standard-layout*, and *POD* or Plain Old Data. The Standard Library has the function templates `is_trivial<T>`, `is_standard_layout<T>` and `is_pod<T>` that determine whether a given type belongs to a given category.

## Trivial types

When a class or struct in C++ has compiler-provided or explicitly defaulted special member functions, then it is a trivial type. It occupies a contiguous memory area. It can have members with different access specifiers. In C++, the compiler is free to choose how to order members in this situation. Therefore, you can memcopy such objects but you cannot reliably consume them from a C program. A trivial type T can be copied into an array of char or unsigned char, and safely copied back into a T variable. Note that because of alignment requirements, there might be padding bytes between type members.

Trivial types have a trivial default constructor, trivial copy constructor, trivial copy assignment operator and trivial destructor. In each case, *trivial* means the constructor/operator/destructor is not user-provided and belongs to a class that has

---

- no virtual functions or virtual base classes,
- no base classes with a corresponding non-trivial constructor/operator/destructor
- no data members of class type with a corresponding non-trivial constructor/operator/destructor

The following examples show trivial types. In Trivial2, the presence of the `Trivial2(int a, int b)` constructor requires that you provide a default constructor. For the type to qualify as trivial, you must explicitly default that constructor.

C++

```
struct Trivial
{
    int i;
private:
    int j;
};

struct Trivial2
{
    int i;
    Trivial2(int a, int b) : i(a), j(b) {}
    Trivial2() = default;
private:
    int j;  // Different access control
};
```

## Standard layout types

When a class or struct does not contain certain C++ language features such as virtual functions which are not found in the C language, and all members have the same access control, it is a standard-layout type. It is memcopy-able and the layout is sufficiently defined that it can be consumed by C programs. Standard-layout types can have user-defined special member functions. In addition, standard layout types have these characteristics:

- no virtual functions or virtual base classes
- all non-static data members have the same access control
- all non-static members of class type are standard-layout
- any base classes are standard-layout
- has no base classes of the same type as the first non-static data member.
- meets one of these conditions:
    - no non-static data member in the most-derived class and no more than one base class with non-static data members, or
    - has no base classes with non-static data members

C++

```
struct SL
{
   // All members have same access:
   int i;
    int j;
    SL(int a, int b) : i(a), j(b) {} // User-defined constructor OK
};
```

The last two requirements can perhaps be better illustrated with code. In the next example, even though Base is standard-layout, `Derived` is not standard layout because both it (the most derived class) and `Base` have non-static data members:

C++

```
struct Base
{
    int i;
    int j;
};

// std::is_standard_layout<<Derived> == false!
struct Derived : public Base
{
    int x;
    int y;
};
```

In this example `Derived` is standard-layout because `Base` has no non-static data members:

C++

```
struct Base
{
    void Foo() {}
};

// std::is_standard_layout<<Derived> == true
struct Derived : public Base
{
    int x;
    int y;
};
```

Derived would also be standard-layout if `Base` had the data members and `Derived` had only member functions.

## POD types

When a class or struct is both trivial and standard-layout, it is a POD (Plain Old Data) type. The memory layout of POD types is therefore contiguous and each member has a higher address than the member that was declared before it, so that byte for byte copies and binary I/O can be performed on these types. Scalar types such as int are also POD types. POD types that are classes can have only POD types as non-static data members.

## Example

The following example shows the distinctions between trivial, standard-layout, and POD types:

C++

```
#include <type_traits>
#include <iostream>

using namespace std;

struct B
{
protected:
    virtual void Foo() {}
};
```

```cpp
// Neither trivial nor standard-layout
struct A : B
{
    int a;
    int b;
    void Foo() override {} // Virtual function
};

// Trivial but not standard-layout
struct C
{
    int a;
private:
    int b;  // Different access control
};

// Standard-layout but not trivial
struct D
{
    int a;
    int b;
    D() {} //User-defined constructor
};

struct POD
{
    int a;
    int b;
};
```

```cpp
int main()
{
 cout << boolalpha;
    cout << "A is trivial is " << is_trivial<A>() << endl; // false
    cout << "A is standard-layout is " << is_standard_layout<A>() << endl;  // false

    cout << "C is trivial is " << is_trivial<C>() << endl; // true
    cout << "C is standard-layout is " << is_standard_layout<C>() << endl;  // false

    cout << "D is trivial is " << is_trivial<D>() << endl;  // false
    cout << "D is standard-layout is " << is_standard_layout<D>() << endl; // true

    cout << "POD is trivial is " << is_trivial<POD>() << endl; // true
    cout << "POD is standard-layout is " << is_standard_layout<POD>() << endl; // true

return 0;
}
```

input

```
A is trivial is false
A is standard-layout is false
C is trivial is true
C is standard-layout is false
D is trivial is false
D is standard-layout is true
POD is trivial is true
POD is standard-layout is true
```

# Trivial Types

Bookmark this page

A class is said to be Trivially Copyable by meeting following requirements

- Copy constructor is trivial or deleted
- Assignment operator overloading is trivial or deleted
- Trivial non deleted destructor

A class is said to be trivial if it's trivially copyable and meeting additional requirement

- Trivial non deleted default constructor

From the above requirement it implies following facts about Trivial classes

- No virtual functions or virtual base class
- All data members are trivial without the need for any external representations or custom initialization (for e.g. no dynamic memory, file handling, network connections, database connections etc. are involved)
- All base classes(if any) must be trivial

- Objects of trivial class can be managed with compiler provided copy constructor, destructor, operator= as they can be copied by compiler implicit rules and no destructor code is required.
- You can observe that constructor body can be blank for trivial class, as all members can be initialized through initializer list trivially.

**std::is_trivial,std::is_trivially_copyable** from type traits are used to check these types.

Example of a trivial class:-

```
class Simple
{
  int x;
  int y;
  public:
  Sample(int p,int q):x(p),y(q) {
  }
  void printxy() {
    std::cout << x << "," << y << "\n";
  }
};
```

```
//Check:-
std::cout << std::is_trivial<Simple>::value << "\n";
```

Example of a non trivial class:-

Here copy constructor is not deleted

```
class Stack
{
  int *arr;
  int top;
  const int maxlen;
  public:
  Stack(int len):maxlen(len),top(-1) {
     arr=new int[maxlen];
  }
  ~Stack() {
     delete[] arr;
  }
  //remaining code
};

//Check:-
std::cout << std::is_trivial<Stack>::value << "\n";
```

# Standard Layout types

Bookmark this page

A class is known to be standard layout type by meeting following requirements

- All non static data members have same access control
- No virtual functions and no virtual base class
- No non static member of reference type
- All data members and base class (if any) are also of standard layout type.
- Not getting inherited from more than one base class

Type traits library provides std::is_standard_layout check for standard layout types.

**Ref:-**

https://en.cppreference.com/w/cpp/named_req/StandardLayoutType

## POD Types

Bookmark this page

A class is known to be a Plain Old Data (POD) type by meeting following requirements

- It is a trivial type
- It is a standard layout type
- All non static data members are also of POD type

Type traits provides **std::is_pod** check for POD type (going to be deprecated in C++20)

**Ref:-**

https://en.cppreference.com/w/cpp/named_req/PODType

Click to add Bookmark this page

- std::is_trivial

  https://en.cppreference.com/w/cpp/types/is_trivial
  https://en.cppreference.com/w/cpp/named_req/TrivialType

- std::is_trivially_copyable

  https://en.cppreference.com/w/cpp/types/is_trivially_copyable
  https://en.cppreference.com/w/cpp/named_req/TriviallyCopyable

- std::is_standard_layout

  https://en.cppreference.com/w/cpp/types/is_standard_layout
  https://en.cppreference.com/w/cpp/named_req/StandardLayoutType

- std::is_pod

  https://en.cppreference.com/w/cpp/types/is_pod
  https://en.cppreference.com/w/cpp/named_req/PODType

**Concept Checking:-**

- What is the impact if following if following members are deleted a) default constructor b) copy constructor c) destructor d) copy assignment operator

Prior to C++11, I saw code like this:

```
class Car {
public:
  Car() {}
private:
  Car(const Car&);
  Car& operator=(const Car&);
};
```

For C++11 (and later), I see code like this:

```
class Car {
public:
  Car() {}
private:
  Car(const Car&) = delete;
  Car& operator=(const Car&) = delete;
};
```

They are similar in most respects, but differ in some others.

Consider the following *external* code trying to copy objects of the class:

```
int main() {
   Car c;
   Car other{c};
}
```

Both of these versions will cause the above code to fail. At this point, though, the developer would look at the interface to see why. With the `delete` version, it's obvious that `Car` was not meant to be copied. With the `private` version, (at least without a comment), it raises a doubt whether the copy constructor was perhaps placed in the `private` section by accident.

Now consider *member* code trying to copy objects of the class:

```
void Car::foo() {
   Car c;
   Car other{c};
}
```

The `delete` version fails like before. The `private` version is a link error. This raises an even greater doubt - it's not uncommon to forget to define a method that has been declared. Perhaps this is what's happened here? The `delete` version doesn't have this problem.

- Do deleted members should be kept under public or private section of the class?

```
using namespace std;

class Sample {
    Sample (int a, int b) = delete;
    public:
    Sample (int a) = delete;
    Sample(){}
};

int main()
{
Sample s1;
return 0;
}
```

```
..Program finished with exit code 0
ress ENTER to exit console.
```

- For implementing creational design patterns (factory/singleton) do you prefer private members or deleted members?

  Private

- How override keyword prevents accidental deviation from overriding like const mismatch?

  True

- Do trivial classes will have custom definition for copy constructor, destructor, copy assignment operator?

  ## No

  Check the applicability of =default, =delete on: Parameterized constructors

```
6  class Sample {
7  public:
8      Sample (int a, int b) = default;
9      Sample (int a) = default;
10     Sample() = default;
11 };
12
13 int main()
14 {
15 //Sample s1;
16 return 0;
17 }
18
```

input

Compilation failed due to following error(s).

```
main.cpp:8:29: error: 'Sample::Sample(int, int)' cannot be defaulted
     Sample (int a, int b) = default;
                             ^~~~~~~
main.cpp:9:22: error: 'Sample::Sample(int)' cannot be defaulted
     Sample (int a) = default;
                      ^~~~~~~
```

## Read only objects

- Read only/ROMable objects with constexpr
- Non trivial constructors must be prefixed with constexpr to model read only objects(literals)
- constexpr can't invoke non const member functions (need not have constexpr prefix, const suffix is must)
- result of non constexpr functions can't be assigned to constexpr variables.

```
class Sample {
  int x;
  int y;
 public:
  constexpr Sample(int p,int q):x(p),y(q) { }
  constexpr int getx() const {
    return x;
  }
  //TODO:gety
  constexpr int getsum() const {
    return x+y;
  }
  void printxy() const {
    //print x,y
```

```
  int getdiff() const {
    return x-y;
  }
  int getproduct() {
    return x-y;
  }
};
int main()
{
  constexpr Sample s1(10,20);        //read-only object(ROMable)
  Sample s2(11,22);                  //normal object, constexpr ignored
  std::cout << s1.getsum() << "\n";
  s1.printxy();
  s2.printxy();
  constexpr int s=s1.getsum();       //ok
  int d1=s1.getdiff();               //ok
  constexpr int d2=s1.getdiff();     //error
  s1.getproduct();                   //error
}
```

# Lambda Expressions & Function Wrappers

- Lambdas or Lambda expressions are unnamed functions introduced by C++11
- Lambdas allow most convenient way of passing business logic to other functions (callbacks)

**Simple lambda example:-**

```
[](int x,int y) {
  std::cout << x + y << "\n";
}
[](int x,int y) -> int {            //Trailing return syntax
  return x + y;
}
```

**Naming lambdas:-**

```
auto sum = [](int x,int y)->int {
  return x + y;
};
std::function<int(int,int)> fsum = [](int x,int y)->int {
  return x+y;
};
```

**Passing lambdas as function arguments:-**

```
void test(int x, int y, auto cbfun)
{
  int z;
  z=cbfun(x,y);
}
test(10, 20, [](int x, int y) {
  return x*x + y*y;
});
```

```
auto sumsqr= [](int x, int y) {
  return x*x + y*y;
};
test(10,20,sumsqr);
```

**Passing lambdas to STL algorithms:-**

```
std::array<int,n> arr;
for_each(arr.begin(), arr.end(), [](int &r) {
  r=rand()%100;
});
for_each(arr.begin(), arr.end(), [](int x) {
  sum+=x;
});

std::vector<Book> books;
std::sort(books.begin(), books.end(), [](Book& r1,Book& r2)->bool {
                          return r1.getPrice() < r2.getPrice()
});

int k=std::find_if(books.begin(), books.end(), [](Book& r1)->bool {
                          return r1.getPrice() > 300;
});
```

**Tutorial:-**

- C++11 Lambda Function : Example & Tutorial
- C++11 Lambda : Capturing local variables
- C++11 Lambda : Capturing Member Variables
  - **Additional Examples:-**
  - http://www.java2s.com/example/cpp/stl/lambda-index.html

Lambdas can access variables/expressions from enclosing scope

**Examples:-**

```
std::vector<int> tarr{18,25, 16, 26, 22};
constexpr int tmax=30;
std::for_each(tarr.begin(), tarr.end(), [=] (int tval) {
  if(tval> tmax)
    std::cout << "temperature is high\n";
});

std::vector<Book> books;
constexpr int minPrice=300;
int k=std::find_if(books.begin(), books.end(), [minPrice] (Book& r1) {
   return r1.getPrice() > minPrice;
});
```

**Move capture:-**

```
[tmax=30] (int x) {
  if(x > tmax)
    //do some
}
[rr= std::move(s1)] (int x) {
  //captured move instance of s1
}
```

### Capturing variables/objects by value, reference

- [&] capture all variables from enclosing scope by reference
- [=] capture all variables from enclosing scope by value
- [&val] capture single value by reference
- [val] capture single value by value
- [=, &val] capture all by value, specific variable by refernce
- [&x,y] capture x by reference, y by value
- [this] capture current object(*this) by reference
- [] no capture

**Further Reading:-**

**Item 31: Avoid default capture modes.,** Effective Modern C++, Scott Meyers

**Item 32: Use init capture to move objects into closures,** Effective Modern C++, Scott Meyers

## Generic Lambdas

Bookmark this page

From C++14, lambda parameters can be of generic type with auto prefix

**Example:-**

```
auto f1= [](auto x, auto y) {
  return x + y;
};

std::sort(books.begin(),books.end(), [](auto& r1,auto& r2) {
  return r1.getPrice() > r2.getPrice();
});
```

## std::function

Bookmark this page

- **std::function** creates wrapper object for callable elements like normal functions, function objects and lambdas

**Examples:-**

**Wrapping normal functions as std::function objects:-**

```
#include<functional>

int sum(int,int);
std::function<int(int,int)> fsum=sum;

void sumsqr(int,int);
std::function<void(int,int)> fsumsqr=sumsqr;
```

**std::function objects as callbacks:-**

```
void test(int x,int y,std::function<int(int,int)> psum ) {
  int z;
  z=psum(x,y);
  //print z
}
```

**Wrapping lambdas as std::function objects:-**

```
std::function<int(int,int)> f1 = [] (int x,int y) {
  return x+y;
}

std::function<double(double)> fsquare = [] (double x)->double {
  return x*x;
}
```

**Note:-** auto keyword is mostly replaceable wherever std::function is used as parameter type.

```
#include<functional>
#include <iostream>
class Adder {
  public:
  /*int operator()(int x,int y){
    std::cout<<"() operator called"<<std::endl;
    return x + y;
  }*/
};
int main() {
  Adder a1;
  std::function<int(int,int)> sum=a1;
  /*int x=10,y=20,z;
  z=sum(x,y);
  std::function<int(int,int)> add=Adder();
  int a=10,b=20,c;
  c=add(a,b); */
  return 0;
```

input

Compilation failed due to following error(s).

```
main.cpp: In function 'int main()':
main.cpp:12:35: error: conversion from 'Adder' to non-scalar type 'std::function' requested
   std::function<int(int,int)> sum=a1;
                                   ^~
```

```
#include<functional>
#include <iostream>
class Adder {
  public:
  int operator()(int x,int y){
    std::cout<<"() operator called"<<std::endl;
    return x + y;
  }
};
int main() {
  Adder a1;
  std::function<int(int,int)> sum=a1;
  int x=10,y=20,z;
  z=sum(x,y);
  std::function<int(int,int)> add=Adder();
  int a=10,b=20,c;
  c=add(a,b);
  return 0;
```

```
() operator called
() operator called
```

**Array of std::function instances is a good choice for dispatch tables:-**

```
std::function<void(int)> service_table[n];
services[0]=action1;
services[1]=action2;
services[2]=action3;
services[3]=action4;
services[i)(args);
```

**References:-**
https://en.cppreference.com/w/cpp/utility/functional
http://www.cplusplus.com/reference/functional/function/

# std::bind

std::bind minimizes no.of arguments required for any callable element(normal function, lambda, std::function object)by fixing certain arguments with known values and treating other arguments as place holders.

Simple examples:-

```
bool is_greater(int x,int y) {
  return x>y;
}
bool is_lesser(int x,int y) {
  return x<y;
}
auto is_greater_than_100 = std::bind(is_greater, std::placeholders::_1,100);
//(or)
auto is_greater_than_100 = std::bind(is_greater,100,std::placeholders::_1);
```

```
bool compare(int,int);
auto compare60=std::bind(compare,60,std::placeholders::_1);
auto compare40=std::bind(compare,std::placeholders::_1,40);
//std::function<bool(int)> compare40;
```

```
auto factors_of_5 = std::bind(std::multiplies<int>(), 5 , std::placeholders::_1);
for(i=1;i<=10;i++)
  std::cout << factors_of_5(i);
```

- Using bind objects with STL algorithms

**Example:-**

```
std::vector<int> v1{10,20,30,40,50};
std::find_if(v1.begin(), v2.begin(), std::bind(std::greater<int>,
                                     std::placeholders::_1, 25);
```

**Comparison - lambdas vs std::bind:-**

```
std::vector<int> v1{10,20,30,40,50};
std::find_if(v1.begin(), v2.begin(), std::bind(std::greater<int>,
                                     std::placeholders::_1, 25));
//(or)
std::find_if(v1.begin(), v2.begin(), [](auto x) {
   std::greater<int>(x,25);
});
```

**Suggested Reading from Effective Modern C++, Scott Meyers:-**
**Item 34: Prefer lambdas to std::bind**
**Tutorial:-**
https://www.geeksforgeeks.org/bind-function-placeholders-c/
https://thispointer.com//stdbind-tutorial-and-usage-details/

**References:-**
https://en.cppreference.com/w/cpp/utility/functional/bind
http://www.cplusplus.com/reference/functional/bind/

**Concept Checking:-**

- Capturing current object when lambda expressions are used inside the class member functions.
  - capture `this`:

```
auto lambda = [this](){};
```

- When lambdas capture current object, whether they can access private members of enclosing class

```cpp
#include <iostream>

class A
{
public:
    A() : x(5){}
    void f() const
    {
        ([this]()
        {
            std::cout << x << std::endl;
        })();
    }
private:
    int x;
};

int main()
{
    A a;
    a.f();
}
```

```
5
```

- What are the advantages of std::bind

- Declaring callbacks a) function pointer syntax b) std::function objects c) auto keyword usage

## Mutable Lambdas

Bookmark this page

Variables captured by value can't be modified inside lamda expressions

```
auto tmax=20;
auto k=0;
auto test=[tmax,k] (int x) {
  if(x>tmax) {
    k++;                        //error
    tmax--;                     //error
  }
}
```

But mutable lambdas allows modifications to these variables but protects the original value outside the lambda expression.

```
auto tmax=20;
auto k=0;
auto test=[tmax,k] (int x) mutable{
  if(x>tmax) {
    k++;                        //ok
    tmax--;                     //ok
  }
}
test(20);
test(18);
test(30);
std::cout << tmax << "\n";     //any change in the value?
std::cout << k << "\n";        //any change in the value?
```

```cpp
int main()
{
   auto tmax=20;
   auto k=0;
   auto test=[tmax,k] (int x) mutable{
   if(x>tmax) {
    k++;                    //ok
    tmax--;                 //ok
    std::cout <<"inside test method: " << tmax << "\n";    //value changed
    std::cout <<"inside test method: " << k << "\n";       //value changed
   }};
  test(20);
  test(18);
  test(30);
  std::cout << "outside test method: " <<tmax << "\n";    //no change in value
  std::cout << "outside test method: " <<k << "\n";       //no change in value
return 0;
}
```

input

```
inside test method: 19
inside test method: 1
outside test method: 20
outside test method: 0
```

**Analysis:-**

- Compare capture by reference vs mutable lambdas

## Lambdas and bind for Qt slots

Bookmark this page

**Example:-**

**connect API without lambdas**

```
connect(timer, SIGNAL(timeout()), this, SLOT(onTimedout());

void MyController::updateDisplay() {
  this->lcdDisplay->display(QString::from(k));
}
```

Qt5 introduced new connect syntax using function pointers (or any callable elements) which need not be defined as slot in class. (Ref:- https://wiki.qt.io/New_Signal_Slot_Syntax)

```
connect(timer, &QTimer::timeout,this, &MyController::updateDisplay);
```

This allows lambdas as callback as shown below

```
connect(timer, &QTimer::timeout, [this]() {
  this->lcdDisplay->display(now.toString("hh:mm:ss"));
});
```

Using std::bind

```
connect(timer,&QTimer::timeout,std::bind(&MyDialog::updateDisplay,this));
```

---

**Example 2:-**

```
connect(slider, SIGNAL(valueChanged()), progressBar, SLOT(setValue(int));
```

Using new syntax:-

```
connect(slider, &QSLider::valueChanged, progressBar, &QProgressBar::setValue);
```

Using Lambdas

```
connect(slider,&QSlider::valueChanged,[this](int val) {
   progressBar->setValue(val);
});
(or)
connect(slider,&QSlider::valueChanged,[&](int val) {
   progressBar->setValue(val);
});
```

Using std::bind

```
connect(slider,&QSlider::valueChanged,
    std::bind(&QProgressBar::setValue,progressBar,std::placeholders::_1));
```

# Object Model & Move Semantics

- C++ introduced move semantics in the form of r-value references, move constructor, move operator=
- Move semantics ensure that underlying resources are moved from one object to another object.
- This is especially useful when passing temporary objects to containers or returning temporary objects.
- Please note that the term **move** refers to underlying resources, not to the object's own memory

Let's consider a scenario at the outset with MyString example:-

```
MyString s1("abcdxyz");
MyString s2(s1);         //underlying char buffer is duplicated from s1 to s2
MyString s3(std::move(s1));  //underlying char buffer is moved from s1 to s3
                             //i.e. detach from s1 and attach to s3
                             //s1 buffer eventually become null
MyString s4;
s4=std::move(s2);            //detach char buffer from s2 and attach to s4
                             //s2 buffer eventually becomes null
```

Prior to the discussion on the implementation for above operations, let's understand the syntax & semantics of r-value references.

```
#include <memory>
#include <iostream>
class MyString {
public:
  MyString() = default;
  MyString(char* str): MyString(){}
  MyString(MyString& string) = default;
  MyString(MyString&& string) = default;
  MyString& operator=(const MyString& string) = default;
};

int main() {

MyString s1("abcdxyz");
MyString s2(s1);         //underlying char buffer is duplicated from s1 to s2
MyString s3(std::move(s1));  //underlying char buffer is moved from s1 to s3
                             //i.e. detach from s1 and attach to s3
                             //s1 buffer eventually become null
MyString s4;
s4=std::move(s2);            //detach char buffer from s2 and attach to s4
                             //s2 buffer eventually becomes null
return 0;
}
```

input

```
...Program finished with exit code 0
Press ENTER to exit console.
```

# Rvalue references and std::move

Bookmark this page

- Expressions wrapped in std::move  are compatible with r-value references
- Unnamed/temporary objects are by default compatible with r-value references
- Move operations are differentiated from copy operations based on r-value references
- Please note that r-value reference doesn't mean object is immutable, they just match move operations.

**Examples:-**

r-value references on user defined objects

```
Sample s1{10,20};
Sample &r1=s1;
Sample &r2 = {10,20};                  //error
Sample &&r3 = {10,20};                 //ok
Sample &&r4 = s1;                      //error
Sample &&r5=std::move(s1);
```

r-value references with unnamed objects

```
Sample getInstance(int x,int y) {
  return Sample(x,y);
}
Sample   s1  = getInstance(10,20);      //ok
Sample&  r1  = getInstance(11,21);      //error
Sample&& rr1 = getInstance(12,22);      //ok

Sample s4    = Sample(10,20);          //ok
Sample &r2   = Sample(11,21);          //error
Sample &&rr2 = Sample(12,22);          //ok
```

r-value references with STL containers

```
std::vector<Sample> samples;
samples.push_back(Sample(10,20));    //Matches push_back(Sample&&)
Sample s1(30,40);
samples.push_back(std::move(s1));    //Matches push_back(Sample&&)
samples.push_back(s1);               //Matches push_back(Sample&)
```

r-value references on primitive types (No practical use, just for syntax)

```
int x=10;
int &r1 = x;                          //ok
int &r2 = 10;                         //error
int &&r3 = 10;                        //ok
int &&r4 = x+5;                       //ok
int &&r5 = x;                         //error

int max(int a,int b) { return a > b ? a : b; }
int& rmax(int& r1,int& r2) { return r1 > r2 ? r1 : r2; }

int z    = max(x,y);                   //ok
int &r1  = rmax(x,y);                  //ok
int &r2  = max(x,y);                   //error
int &&rr = max(x,y);                   //ok
```

Unnamed objects can be binded to r-value references:-

```
Sample &ref=Sample(10,20);//error
const Sample &cref=Sample(10,20);//ok
Sample&& rr=Sample(10,20);//ok
```

```
#include <memory>
#include <iostream>
#include <string.h>
#include <vector>

class MyString {
public:
  MyString();
  MyString(char* str);
  MyString(MyString& string);
  MyString(MyString&& string);
  MyString& operator=(const MyString&);
  MyString& operator=(MyString&& rr);
  friend std::ostream & operator << (std::ostream &out, const MyString &c);
  friend std::istream & operator >> (std::istream &in,  MyString &c);

protected:
  char* m_buf;
```

```
  int m_len;
};

std::ostream & operator << (std::ostream &out, const MyString &c)
{
    out << c.m_buf<<std::endl;
    return out;
}

std::istream & operator >> (std::istream &in,  MyString &c)
{
    //in >> c.m_buf;
    //in >> c.m_len;
    return in;
}

MyString::MyString()
{
  std::cout<<"MyString() called"<<std::endl;
  m_len = 1;
  m_buf = new char[m_len];
}

MyString::MyString(char* string)
{
  std::cout<<"MyString(char* string) called"<<std::endl;
  m_len = strlen(string);
  m_buf = new char[m_len + 1];
  memcpy(m_buf, string, m_len);
  m_buf[m_len] = '\0';
}

MyString::MyString(MyString& string)
{
```

```
  std::cout<<"MyString(MyString& string) called"<<std::endl;
  m_len=string.m_len;
  m_buf = new char[m_len];
  memcpy(m_buf, string.m_buf, m_len);
}

MyString::MyString(MyString&& rr) {
  std::cout<<"MyString(MyString&& rr) called"<<std::endl;
  m_len=rr.m_len;
  m_buf=rr.m_buf;
  rr.m_len=0;
  rr.m_buf=nullptr;
}

MyString& MyString::operator=(const MyString& string)
{
  std::cout<<"operator=(const MyString& string) called"<<std::endl;
  m_len=string.m_len;
  m_buf = new char[m_len];
  memcpy(m_buf, string.m_buf, m_len);
  return *this;
}

MyString& MyString::operator=(MyString&& rr)
{
  std::cout<<"operator=(MyString&& rr) called"<<std::endl;
  m_len=rr.m_len;
  m_buf=rr.m_buf;
  rr.m_len=0;
  rr.m_buf=nullptr;
  return *this;
}

int main() {
```

```
std::vector<MyString> sarr;
MyString s1("abcdxyz");
std::cout<<"s1:"<<s1;

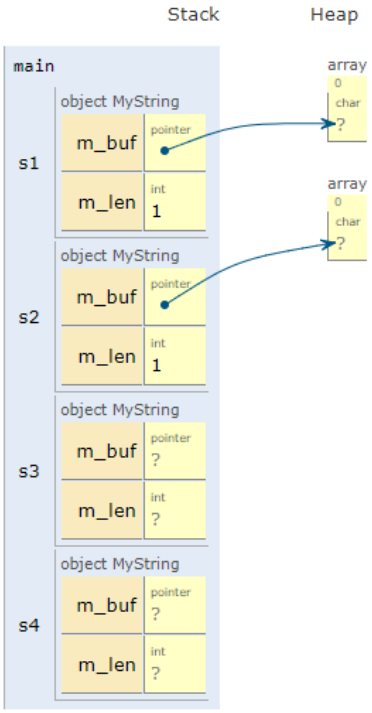
sarr.push_back(std::move(s1));
std::cout<<"sarr[0]:"<<sarr[0]<<std::endl;

sarr.push_back(MyString("hello"));
std::cout<<"sarr[1]:"<<sarr[1]<<std::endl;

std::cout<<"end of program"<<std::endl;
return 0;
}
int main() {
MyString s1("abcdxyz");
MyString s2(s1);          //underlying char buffer is duplicated from s1 to s2
MyString s3(std::move(s1));  //underlying char buffer is moved from s1 to s3
                             //i.e. detach from s1 and attach to s3
                             //s1 buffer eventually become null
MyString s4;
s4=std::move(s2);            //detach char buffer from s2 and attach to s4
                             //s2 buffer eventually becomes null
std::cout<<"end of program"<<std::endl;
return 0;
}
```

C++ (gcc 4.8, C++11)
EXPERIMENTAL! known bugs/limitations

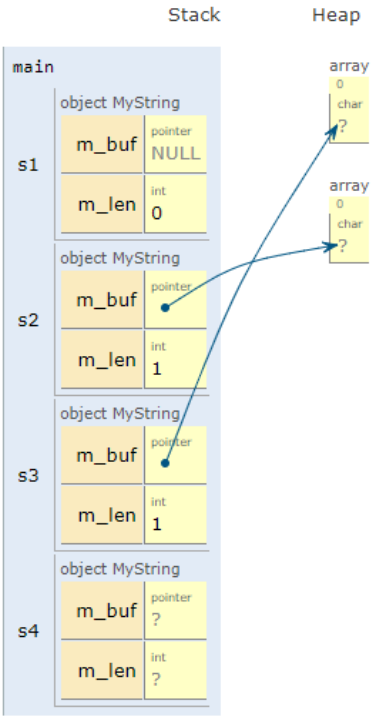
```
45
46 MyString& MyString::operator=(MyString&& rr)
47 {
48   m_len=rr.m_len;
49   m_buf=rr.m_buf;
50   rr.m_len=0;
51   rr.m_buf=nullptr;
52   return *this;
53 }
54
55 int main() {
56
57 MyString s1("abcdxyz");
58 MyString s2(s1);         //underlying char buffer is duplic
59 MyString s3(std::move(s1));  //underlying char buffer is mo
60                              //i.e. detach from s1 and atta
61                              //s1 buffer eventually become
62 MyString s4;
63 s4=std::move(s2);            //detach char buffer from s2 a
64                              //s2 buffer eventually becomes
65 return 0;
66 }
```




C++ (gcc 4.8, C++11)
EXPERIMENTAL! known bugs/limitations

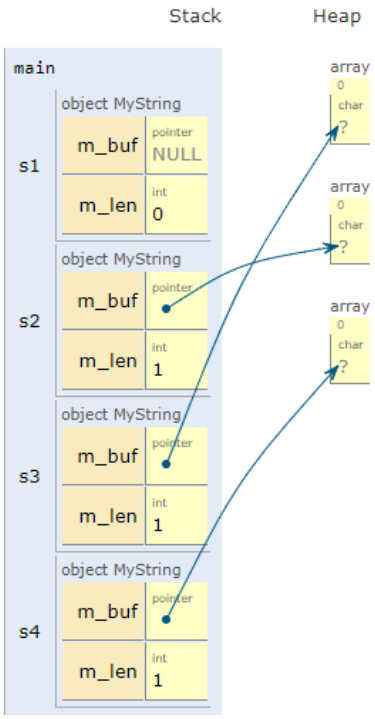
```
45
46 MyString& MyString::operator=(MyString&& rr)
47 {
48   m_len=rr.m_len;
49   m_buf=rr.m_buf;
50   rr.m_len=0;
51   rr.m_buf=nullptr;
52   return *this;
53 }
54
55 int main() {
56
57 MyString s1("abcdxyz");
58 MyString s2(s1);         //underlying char buffer is duplic
59 MyString s3(std::move(s1));  //underlying char buffer is mo
60                              //i.e. detach from s1 and atta
61                              //s1 buffer eventually become
62 MyString s4;
63 s4=std::move(s2);            //detach char buffer from s2 a
64                              //s2 buffer eventually becomes
65 return 0;
66 }
```

Edit this code

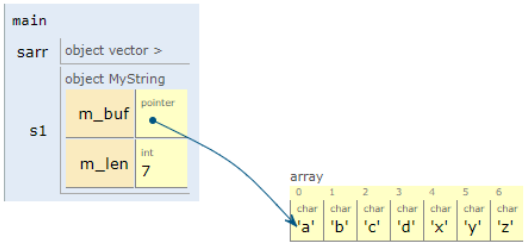


C++ (gcc 4.8, C++11)
EXPERIMENTAL! known bugs/limitations

```
45
46  MyString& MyString::operator=(MyString&& rr)
47  {
48    m_len=rr.m_len;
49    m_buf=rr.m_buf;
50    rr.m_len=0;
51    rr.m_buf=nullptr;
52    return *this;
53  }
54
55  int main() {
56
57  MyString s1("abcdxyz");
58  MyString s2(s1);         //underlying char buffer is duplic
59  MyString s3(std::move(s1));  //underlying char buffer is mo
60                               //i.e. detach from s1 and atta
61                               //s1 buffer eventually become
62  MyString s4;
63  s4=std::move(s2);            //detach char buffer from s2 a
64                               //s2 buffer eventually becomes
65  return 0;
66  }
```




```
49    m_buf = new char[m_len];
50    memcpy(m_buf, string.m_buf, m_len);
51    return *this;
52  }
53
54  MyString& MyString::operator=(MyString&& rr)
55  {
56    m_len=rr.m_len;
57    m_buf=rr.m_buf;
58    rr.m_len=0;
59    rr.m_buf=nullptr;
60    return *this;
61  }
62
63  int main() {
64  std::vector<MyString> sarr(5);
65  MyString s1("abcdxyz");
66  sarr.push_back(std::move(s1));
67  sarr.push_back(MyString("hello"));
68
69  std::cout<<"end of program"<<std::endl;
70  return 0;
71  }
```



```
86  int main() {
87  std::vector<MyString> sarr;
88  MyString s1("abcdxyz");
89  std::cout<<"s1:"<<s1;
90
91  sarr.push_back(std::move(s1));
92  std::cout<<"sarr[0]:"<<sarr[0]<<std::endl;
93
94  sarr.push_back(MyString("hello"));
95  std::cout<<"sarr[1]:"<<sarr[1]<<std::endl;
96
97  std::cout<<"end of program"<<std::endl;
98  return 0;
99  }
```

```
MyString(char* string) called
s1:abcdxyz
MyString(MyString&& rr) called
sarr[0]:abcdxyz

MyString(char* string) called
MyString(MyString&& rr) called
MyString(MyString&& rr) called
sarr[1]:hello

end of program
```

## Move operations with containers

Bookmark this page

Let's consider a  container holding MyString objects, without move semantics objects need to be copied (duplication of underlying resources for non trivial class) and the original object is no more required once it's added to the container element.Move semantics allows better optimization by moving resources to the container element.

For e.g. std::vector provides overloaded push_back operations which takes rvalue references of instantiated type,this prevents additional copy of resources in source object

```
std::vector<MyString> sarr(5);
MyString s1("abcdxyz");
sarr.push_back(std::move(s1));
sarr.push_back(MyString("hello"));
```

Here s1 need not maintain the buffer any more once a copy of it(which is moved) is pushed on to vector, similarly unnamed object MyString("hello") matches with r-value reference thus move semantics are applied.

## Deleted move operations

Bookmark this page

What if move constructor, move operator= are disabled for a class?

```
#include<iostream>
#include<vector>
class Sample {
  int x;
  int y;
public:
  Sample()=default;
  Sample(int p,int q):x(p),y(q) { }
  Sample(const Sample& rs)=default;
  Sample(Sample&& rr)=delete;
  Sample& operator=(const Sample&)=default;
  Sample& operator=(Sample&&)=delete;
  void printxy() const {
    std::cout << x << "," << y << "\n";
  }
```

```
int main() {
  Sample s1(10,20);
  Sample s2(s1);
  //Sample s3(std::move(s1));
  //Sample s4(Sample(11,12));
  Sample s5,s6;
  //s5=std::move(s1);
  //s5=Sample(15,16);
  std::vector<Sample> samples;
  //samples.push_back(Sample(11,12));
  samples.push_back(s1);
  //samples.push_back(std::move(s1));
  return 0;
}
```

main.cpp

```cpp
#include<iostream>
#include<vector>
class Sample {
  int x;
  int y;
public:
  Sample() {std::cout<<"Sample()"<<std::endl;};
  Sample(int p,int q):x(p),y(q) {std::cout<<"Sample(int p,int q)"<<std::endl; }
  Sample(const Sample& rs) {std::cout<<"Sample(const Sample& rs)"<<std::endl; }
  Sample(Sample& rs) {std::cout<<"Sample(Sample& rs)"<<std::endl; }
  Sample(Sample&& rr) {std::cout<<"Sample(Sample&& rr)"<<std::endl; }
  Sample& operator=(const Sample&) {std::cout<<" operator=(const Sample&)"<<std::endl; }
  Sample& operator=(Sample&&) {std::cout<<"operator=(Sample&&)"<<std::endl; }
  void printxy() const {
    std::cout << x << "," << y << "\n";
  }
};
int main() {
  Sample s1(10,20); //Sample(int p,int q)
  Sample s2(s1); //Copy constructor
  Sample s3(std::move(s1)); //move constructor
  Sample s4(Sample(11,12)); //Sample(int p,int q)
  Sample s5,s6; //Sample() twice
  s5=std::move(s1);//move = operator
  s5=Sample(15,16);//Sample(int p,int q) and move = operator
  std::vector<Sample> samples;
  samples.push_back(Sample(11,12));//Sample(int p,int q) and Move constructor
  samples.push_back(s1); //copy constructor
  samples.push_back(std::move(s1));//move constructor
  return 0;
}
```

input

```
Sample(Sample& rs)
Sample(Sample&& rr)
Sample(int p,int q)
Sample()
Sample()
operator=(Sample&&)
Sample(int p,int q)
operator=(Sample&&)
Sample(int p,int q)
Sample(Sample&& rr)
Sample(const Sample& rs)
Sample(const Sample& rs)
Sample(Sample&& rr)
Sample(const Sample& rs)
Sample(const Sample& rs)
```

# Deleted move operations

Bookmark this page

What if move constructor, move operator= are disabled for a class?

```
#include<iostream>
#include<vector>
class Sample {
  int x;
  int y;
public:
  Sample()=default;
  Sample(int p,int q):x(p),y(q) { }
  Sample(const Sample& rs)=default;
  Sample(Sample&& rr)=delete;
  Sample& operator=(const Sample&)=default;
  Sample& operator=(Sample&&)=delete;
```



```
public:
  Sample() {std::cout<<"Sample()"<<std::endl;};
  Sample(int p,int q):x(p),y(q) {std::cout<<"Sample(int p,int q)"<<std::endl; }
  Sample(const Sample& rs) {std::cout<<"Sample(const Sample& rs)"<<std::endl; }
  Sample(Sample& rs) {std::cout<<"Sample(Sample& rs)"<<std::endl; }
  Sample(Sample&& rr) = delete;//{std::cout<<"Sample(Sample&& rr)"<<std::endl; }
  Sample& operator=(const Sample&) {std::cout<<" operator=(const Sample&)"<<std::endl; }
  Sample& operator=(Sample&&) = delete;//{std::cout<<"operator=(Sample&&)"<<std::endl; }
  void printxy() const {
    std::cout << x << "," << y << "\n";
  }
```

input stderr

Compilation failed due to following error(s).

```
main.cpp: In function 'int main()':
main.cpp:21:26: error: use of deleted function 'Sample::Sample(Sample&&)'
   Sample s3(std::move(s1)); //move constructor
                         ^
main.cpp:11:3: note: declared here
   Sample(Sample&& rr) = delete;//{std::cout<<"Sample(Sample&& rr)"<<std::endl; }
   ^~~~~~
main.cpp:24:18: error: use of deleted function 'Sample& Sample::operator=(Sample&&)'
   s5=std::move(s1);//move = operator
                  ^
main.cpp:13:11: note: declared here
   Sample& operator=(Sample&&) = delete;//{std::cout<<"operator=(Sample&&)"<<std::endl; }
           ^~~~~~~~
```

```
int main() {
  Sample s1(10,20);
  Sample s2(s1);
  //Sample s3(std::move(s1));
  //Sample s4(Sample(11,12));
  Sample s5,s6;
  //s5=std::move(s1);
  //s5=Sample(15,16);
  std::vector<Sample> samples;
  //samples.push_back(Sample(11,12));
  samples.push_back(s1);
  //samples.push_back(std::move(s1));
  return 0;
}
```

Analysis:-

From above code, you can identify the significance of move operations even for trivial classes,i.e. even though underlying resources can't be moved, to support unnamed objects move operations are required in this example.

Click to add Bookmark this page

**Suggested Reading from Effective Modern C++, Scott Meyers:-**

**Item 29: Assume that move operations are not present, not cheap, and not used**

**Item 17: Understand special member function generation**

**Tutorials - R-Value References:-**

https://thispointer.com/difference-between-lvalue-and-rvalue-in-c/

https://thispointer.com/is-rvalue-immutable-in-c/

https://thispointer.com/what-is-a-rvalue-reference/

**Tutorial - Move operations:-**

https://thispointer.com/c11-move-contsructor-rvalue-references/

**RVO vs std::move:-**

RVO vs std::move

https://shaharmike.com/cpp/rvo/

**Concept Checking:-**

- Do compiler provided move constructor, move operator= are applicable for trivial classes?

- Compatibility between unnamed objects, l-value & r-value references

you have an rvalue when the object **has no name**. Typically **rvalues** are temporary objects that exist on the stack as the result of a function call or other operation

Returning a value from a function will turn that value into an **rvalue**. Once you call return on an object, the name of the object does not exist anymore (it goes out of scope), so it becomes an rvalue

Similarly, calling a function will give back an **rvalue**. The return value of a function has no name.

```
MyType MakeMyType() {
 MyType type;
 return type;
}


int main(int argc, char **argv) {
 // MakeMyType returns an rvalue MyType.
 MyType type = MakeMyType();
 ProcessMyType(MakeMyType());
}
```

- Why const prefix is not used with r-value reference in move operations
-
- How to ensure that only move operations are applied on objects of a particular class,i.e. objects can't be clone but can be moved.

- Identify the additional checks in destructor when move operations are enabled, e.g. releasing heap memory,closing files

-
- Impact of deleted move operations on the instances of a class
-

- Whether Return Value Optimization(RVO) is still required when move operations are enabled?

- rvalues of builtin data type is Immutable
    1. We can not modify the rvalue of builtin data type i.e.

    (x+7) = 7; // Compile error - Can not Modify rvalue

    2. Here getData() is a rvalue of buildin data type int and if we try to modify it then it will result in compile error i.e int getData();

    getData() = 9; // Compile Error - Can not modify rvalue

- rvalue of User Defined data type is **not** Immutable

rvalue of User Defined Data type can be modified. But it can be modified in same expression using its own member functions only. Let's see an example,
Create a User Defined Data Type i.e. a class of Person i.e

```
class Person {
        int mAge;
public:
        Person() {
                mAge = 10;
        }
        void incrementAge()
        {
                mAge = mAge + 1;
        }
};
```

It has an incrementAge() member function that modifies it's state i.e. increments mAge data member. Now create a function that will return Person's object as a rvalue i.e.

```
Person getPerson()
{
        return Person();
}
```

Now getPerson() is a rvalue and we can not take its address i.e

```
Person * personPtr = &getPerson(); // COMPILE ERROR
```

But we can modify this rvalue because it is of User Defined Data type (of Person class) i.e.

```
getPerson().incrementAge();
```

Here we have modified a rvalue of Person Class Type with its member function in the same expression.

This proves that we can modify the rvalue of User Defined Data type with its own member function but in the same expression because rvalue persist in single expression only.

the defaulr behaviour of move constructor is same as a copy constructor, is it correct? if it's true

No. Its wrong. Its true only for primitives. Its similar to that of copy constructor.

The default generated copy constructor calls the **copy** constructor of all its members in the declared order
But The default generated move constructor calls the **move** constructor of all its members in the declared order

With **raw pointers**, you have to define move constructors by yourself, manually clearing the old "moved-from" pointer:

```
class Foo
{
public:

   // Move constructor
   Foo(Foo&& other)
      : m_ptr(other.m_ptr) // copy pointer value
   {
      // Clear out old "moved-from" pointer, to avoid dangling references
      other.m_ptr = nullptr;
   }

private:
   int* m_ptr;
```

```
};
```

Instead, if you use a **smart pointer** like `std::unique_ptr`, move constructor is properly defined, and you can just call `std::move`:

```
class Foo
{
public:

   // Move constructor
   Foo(Foo&& other)
      : m_ptr(std::move(other.m_ptr)) // move from other,
                                      // old pointer automatically cleared
   {
   }

private:
   std::unique_ptr<int> m_ptr;
};
```

With *automatically generated* move constructors, you don't have to define a custom move constructor explicitly, if *member-wise* move is OK for you.

2. Move constructor is used with rvalues.

Move constructors are similar to copy constructors. They construct an object out of another object of the same type. The signature of the method changes in order for it to accept rvalues only.

Here's an example of a class with a move constructor. It will cause the compiler to not generate an implicit copy constructor, making this class move-only.

```
class MyType {
public:
 MyType() {}
 MyType(MyType&& other) { fprintf(stderr, "move ctor\n"); }
 // Copy constructor is implicitly deleted.
 // MyType(const MyType& other) = delete;
};


MyType MakeMyType(int a) {
 // This is here to circumvent some compiler optimizations,
 // to ensure that we will actually call a move constructor.
 if (a % 2) {
   MyType type;
   return type;
 }
 MyType type;
 return type;
}


int main(int argc, char **argv) {
 // MakeMyType returns an rvalue MyType.
 // Both lines below call our move constructor.
```

```
 MyType type = MakeMyType(2);
 ProcessMyType(MakeMyType(2));
}
```
http://pastebin.com/raw.php?i=tjxdwQuL

The argument of the move constructor is a `MyType&&` which is a reference (like a `MyType&`) except that it will only bind to an rvalue. That means the following won't compile, since it tries to construct a new `MyType` without an rvalue.

```
class MyType {
public:
 MyType() {}
 MyType(MyType&& other) { fprintf(stderr, "move ctor\n"); }
 // Copy constructor is implicitly deleted.
 // MyType(const MyType& other) = delete;
};

int main(int argc, char** argv) {
 MyType type;
 // |type| is an lvalue, so this will not compile because move
 // constructor's argument will only bind to rvalues.
 MyType other_type = type;
}
```

3. Move constructor does a shallow copy.

A move constructor (like a copy constructor) works by copying the members of another object into itself. The differences of a move constructor are the following:

- The move constructor **can be destructive**. Since the argument is a non-const reference, it may modify the object passed to it. This is okay since the argument is an **rvalue**, so the caller does not hold onto a name for the original object, and won't make use of it (note that its destructor will happen though).

- The move constructor only needs to **shallow copy**. For example if the object holds a pointer, it doesn't need to deref the pointer and copy its contents. The move constructor can just copy the pointer's address. **This is where the performance benefits of move constructors happen.**

```
class MyType {
public:
 MyType() {
   pointer_ = new int;
   *pointer_ = 1;
   memset(array_, 0, sizeof(array_));
   vector_.push_back(3.14);
 }

 MyType(MyType&& other) {
```

```
    fprintf(stderr, "move ctor\n");

    // Steal the memory, null out |other|.
    pointer_ = other.pointer_;
    other.pointer_ = nullptr;

    // Copy the contents of the array.
    memcpy(array_, other.array_, sizeof(array_));

    // Swap with our (empty) vector.
    vector_.swap(other.vector_);
  }

  ~MyType() {
    delete pointer_;
  }

private:
  int* pointer_;
  char array_[42];
  std::vector<float> vector_;
};


int main(int argc, char **argv) {
  // MakeMyType returns an rvalue MyType.
  // Both lines below call our move constructor.
  MyType type = MakeMyType(2);
  ProcessMyType(MakeMyType(2));
}
```

4. std::move() will cast a variable to an rvalue.

If you want to construct a new object from a variable, you can still use the move constructor to do so. **std::move()** will cast your variable to an rvalue, allowing it to bind to a move constructor.

```
int main(int argc, char **argv) {
MyType type;
  // MyType other_type = type; won't work, since there is no copy
ctor.
  // std::move casts |type| to an rvalue, so it uses the move
  // constructor.
  MyType other_type = std::move(type);
}
```

This also means that you can pass an rvalue to a function that uses pass-by-value.

```
int main(int argc, char **argv) {
  MyType type;

  // The return value of MakeMyType is an rvalue, so the move
constructor
  // is used.
  ProcessMyType(MakeMyType(2));

  // Error: |type| is an lvalue, and is not copyable.
  ProcessMyType(type);
```

```
 // But std::move() turns lvalues into rvalues, so the move
constructor
 // is used again.
 ProcessMyType(std::move(type));
}
```

5. You must not use a variable after calling std::move() on it.

Since you have casted your variable to an rvalue, functions that receive rvalues may act destructively on your variable, so using the variable's contents afterward may result in undefined behaviour. The only valid things you may do **after calling std::move()** on a variable are:

- Destroy it
- Assign to it (ie. replace its contents)

This applies to member variables too, don't leave your class in an undefined state when leaving a method.

6. std::move() does not move, it casts.

You may think of **std::move<T>()** as instead **rvalue_cast<T>()**. It does not perform any action of its own on the value passed to it, only changing what methods you can pass it to.

So the following does nothing:

```
MyType t;
std::move(t);  // Casts it but it's unused.
```

Just the same as:

```
char t;
reinterpret_cast<unsigned char>(t);  // Cast it but it's unused.
```

In fact, the actual implementation of std::move() is done with static_cast. It is simply a cast from MyType to MyType&& (a reference to an rvalue of MyType):

```
return static_cast<typename std::remove_reference<T>::type&&>(t);
```

7. Copy constructors can also bind to rvalues (with lower priority).

Never read std::move() on an unfamiliar type and assume anything about what is going to happen!

If you see code that calls **std::move()** it looks like it should be calling a move constructor, but you could be wrong.

When a move constructor on the type being cast to an rvalue does not exist, the compiler will fall back to binding the rvalue as a const-reference. This means the **copy-constructor would be used instead**.

```
class MyType {
public:
 MyType() {
   pointer_ = new int;
   *pointer_ = 1;
```

```
    memset(array_, 0, sizeof(array_));
    vector_.push_back(3.14);
  }

  MyType(const MyType& other) {
    fprintf(stderr, "copy ctor\n");

    // Copy memory.
    pointer_ = new int;
    *pointer_ = *other.pointer_;

    // Copy the contents of the array.
    memcpy(array_, other.array_, sizeof(array_));

    // More copies.
    vector_ = other.vector_;
  }

  ~MyType() {
    delete pointer_;
  }

private:
  int* pointer_;
  char array_[42];
  std::vector<float> vector_;
};


int main(int argc, char** argv) {
  {
    MyType type;
    // MyType other_type = type; works the same way here.
    // std::move casts |type| to an rvalue, but since there is no move
    // constructor, the copy constructor is used.
    MyType other_type = std::move(type);
  }

  {
    MyType type;
    // MakeMyType returns an rvalue, but since there is no move
    // constructor, the copy constructor is used.
    ProcessMyType(MakeMyType(2));
    // ProcessMyType(type); works the same way here.
    // std::move casts |type| to an rvalue, but since there is no move
    // constructor, the copy constructor is used.
    ProcessMyType(std::move(type));
  }
}
```

So do not assume anything from reading std::move(). You need to understand the class being moved to know what will happen.

Beware of const! Const variables can not be moved from, since move constructors take a non-const (rvalue) reference. However the compiler will **silently fall-back to the copy constructor**.

```
int main(int argc, char** argv) {
 const MyType type = MakeMyType(2);
 // std::move casts to an rvalue, but since this is const, it becomes
an
 // rvalue to const, which uses the copy constructor, not the move
 // constructor.
 ProcessMyType(std::move(type));
}
```

This type of error can only be caught by code inspection (and performance testing). **Always look at the type** of the variable on which you call move().

This won't bite you with function return rvalues, but requires caution when using std::move().

## 9. Move constructors should be accompanied by move-assignment operators.

Move-assignment operators work and look the same as a Move constructor. Most things we've discussed about move constructors (behaviour and performance) will apply in all the same ways to move-assignment.

In following example, the class defines its move constructor in terms of operator=, using a move assignment to construct itself.

```
class MyType {
public:
 MyType() {
   pointer_ = new int;
   *pointer_ = 1;
   memset(array_, 0, sizeof(array_));
   vector_.push_back(3.14);
 }

 MyType(MyType&& other) : pointer_(nullptr) {
   // Note that although the type of |other| is an rvalue reference,
   // |other| itself is an lvalue, since it is a named object. In
order
   // to ensure that the move assignment is used, we have to
explicitly
   // specify std::move(other).
   *this = std::move(other);
 }

 ~MyType() {
   delete pointer_;
 }


 MyType& operator=(MyType&& other) {
   // Steal the memory, null out |other|.
   delete pointer_;
   pointer_ = other.pointer_;
   other.pointer_ = nullptr;

   // Copy the contents of the array.
   memcpy(array_, other.array_, sizeof(array_));


   // Swap with our vector. Leaves the old contents in |other|.
```

```
    // We could destroy the old content instead, but this is equally
    // valid.
    vector_.swap(other.vector_);

    return *this;
  }

private:
  int* pointer_;
  char array_[42];
  std::vector<float> vector_;
};
```

10. Don't go cray cray with move semantics.

Move-constructing is always as cheap or cheaper than copy-constructing. So you *could* use it in every situation possible. But it may be adding very little value (or making code misleading).

Almost always, just use a copy constructor. For example gfx::Point is 2 integers, a move constructor would do nothing different than a copy constructor. There's no reason to write a move constructor for this class, or to use std::move() when using Points.

```
class Point {
  int x_;
  int y_;
  ...
  Point(const Point& p) { x_ = p.x_; y_ = p.y_; }
  Point(Point&& p) { x_ = p.x_; y_ = p.y_; }
};
```

Move constructors are especially valuable for types that hold a pointer to something large, since moving a pointer is much cheaper than copying everything it points to. Other scenarios benefit less or not at all.

11. Don't return references. Don't use std::move() on rvalues.

Functions return an rvalue without you doing anything, they already do this (see point #1). So you don't need to do anything to make this happen. You should not see functions that return a MyType&&. Simply return MyType.

```
MyType&& MakeMyType() {  // This is WRONG. The return type should not
be
                         // a reference.
  MyType type;
  return std::move(type);  // This is usually WRONG. Returning will
                           // already make it an rvalue.
}
```

Just as returning a MyType& would be wrong (returning a reference to a temporary object), returning a MyType&& is wrong for the same reasons.

Putting std::move() on the return call actually makes things less efficient, it interferes with the compiler doing RVO (Return Value Optimization). You can read a previous chromium-dev thread about this in the context of scoped_ptr: link.

However, if the types of the variable and the return type do not match exactly, then you will have to use std::move() in order to convert without creating an extra temporary.

```
std::unique_ptr<MyType> MakeMyType()
{
 std::unique_ptr<ChildClassOfMyType> ptr;
 // This call to std::move() is needed to convert from a pointer-to-
the
 // child class to a pointer-to-the parent.
 return std::move(ptr);
}
```

Similarly do not use move() on the return value from calling a function. You already have an rvalue, so casting it there adds no value, only noise.

```
// This is WRONG. It is casting an rvalue to an rvalue, which is not
// needed.
ProcessMyType(std::move(MakeMyType()));
```

**12. std::move() is the same thing as our familiar Pass() function.**

For some time, scoped_ptr::Pass() has been implemented as being the same thing as std::move().

Places that previously called a.Pass() can be changed to call std::move(a) instead, and they will be changed in the near future. It may help you to consider this, if you are already familiar with Pass().

Using std::unique_ptr instead of scoped_ptr will require the use of std::move() instead, as Pass() does not exist there (and is not needed). So code will be change from:

```
scoped_ptr<MyType> t = ...;
CallFunction(t.Pass());
```

To instead be:

```
std::unique_ptr<MyType> t = ...;
CallFunction(std::move(t));
```

# Perfect forwarding

The google style guide allows the following: **Use rvalue references only to define move constructors and move assignment operators, or for perfect forwarding.** We'll talk about the latter here.

So far, we've seen that we can write an rvalue reference in a move constructor, or a move-assignment operator:

```
MyType(MyType&& type);  // Move constructor
operator=(MyType&& type);  // Move-assignment operator
```

In both of these cases, the type `MyType` is fixed, so we know that we have a reference to something that is definitely an rvalue. However when writing a templated method, we may want to support an unknown type instead, at which point we don't know if it is an rvalue.

## Logger example

Bookmark this page

```
class MyLogger {
  int fd;
  public:
  MyLogger():fd(-1) { }
  MyLogger(const char* path) {
    fd=open(path, O_WRONLY|O_CREAT,0666);
  }
  MyLogger(const MyLogger&)=delete;
  MyLogger(MyLogger&& rr):fd(rr.fd) {
    rr.fd=-1;
  }
  ~MyLogger() {
    if(fd!=-1)
       close(fd);
  }
  MyLogger& operator=(MyLogger&& rr) {
    if(fd!=-1) close(fd);
```

```
    fd=rr.fd;
    rr.fd=-1;
  }
  void print(std::string msg) {
    int ret;
    ret=write(fd,msg.c_str(),msg.length());
    if(ret<0)
      perror("write");
  }

};

int main() {
  MyLogger logger1("output.log");
  logger.print("hello");

  MyLogger logger2(logger1); //error
  MyLogger logger2(std::move(logger1));
  //logger1 is detached from file and only logger2 can be used now

  MyLogger logger3;
  logger3=std::move(logger2);
  //file control goes from logger2 to logger3 now
```

From C++11 an expression can be of one of the following types

- lvalue
- rvalue
- prvalue (pure r-value)
- xvalue (movable expressions
- glvalue (generalized lvalue)

**Reference:-**

https://en.cppreference.com/w/cpp/language/value_category

**Further Reading:-**

**Item 24: Distinguish universal references from rvalue references.,** Effective Modern C++, Scott Meyers

**Item 26: Avoid overloading on universal references.,** Effective Modern C++, Scott Meyers

**Item 27: Familiarize yourself with alternatives to overloading on universal references.**, Effective Modern C++, Scott Meyers

## Forwarding References

Bookmark this page

- When r-value reference is passed to another function it's considered as l-value reference
- std::forward ensures that r-value references remain unchanged in chain of function call

Here is an example

```
void print(const Sample& r) {
  std::cout << "print with l-value ref\n";
}
void print(Sample&& r) {
  std::cout << "print with r-value ref\n";
}

template<typename T>
void test(T&& rr) {
  print(rr);                          //r-value ref is passed as  l-value ref
  print(std::forward<Sample>(rr));    //forwards r-value ref as r-value ref
                                      //l-value ref as l-value, perfect
forwarding
  print(std::move(rr));               //always forwarded as r-value
```

```
#include<iostream>

#include<vector>

class Sample {

  int x;

  int y;

public:

  Sample() {std::cout<<"Sample()"<<std::endl;};

  Sample(int p,int q):x(p),y(q) {std::cout<<"Sample(int p,int q)"<<std::endl; }

  Sample(const Sample& rs) {std::cout<<"Sample(const Sample& rs)"<<std::endl; }

  Sample(Sample& rs) {std::cout<<"Sample(Sample& rs)"<<std::endl; }

  Sample(Sample&& rr) {std::cout<<"Sample(Sample&& rr)"<<std::endl; }

  Sample& operator=(const Sample&) {std::cout<<" operator=(const Sample&)"<<std::endl; }

  Sample& operator=(Sample&&) {std::cout<<"operator=(Sample&&)"<<std::endl; }

  void printxy() const {

    std::cout << x << "," << y << "\n";
```

```
  }
};

void print(const Sample& r) {
  std::cout << "print with l-value ref\n";
}
void print(Sample&& r) {
  std::cout << "print with r-value ref\n";
}

template<typename T>
void test(T&& rr) {
  print(rr);                          //r-value ref is passed as  l-value ref
  print(std::forward<Sample>(rr));    //forwards r-value ref as r-value ref
                                      //l-value ref as l-value, perfect forwarding
  print(std::move(rr));               //always forwarded as r-value
}

int main() {
Sample s1;
test(std::move(s1));
//test(s1);
//print(std::forward<Sample>(s1));

  return 0;
}
```

```
Sample()
print with l-value ref
print with r-value ref
print with r-value ref
```

```
34  int main() {
35  Sample s1;
36  test(std::move(s1));
37  test(s1);
38  //print(std::forward<Sample>(s1));
39
40    return 0;
```

```
Sample()
print with l-value ref
print with r-value ref
print with r-value ref
print with l-value ref
print with r-value ref
print with r-value ref
```

```
34  int main() {
35  Sample s1;
36  test(std::move(s1));
37  test(s1);
38  print(std::forward<Sample>(s1)); //r-value reference
39
40    return 0;
41  }
```

```
Sample()
print with l-value ref
print with r-value ref
print with r-value ref
print with l-value ref
print with r-value ref
print with r-value ref
print with r-value ref
```

**References:-**


https://en.cppreference.com/w/cpp/utility/forward
http://www.cplusplus.com/reference/utility/forward/
https://isocpp.org/blog/2012/11/universal-references-in-c11-scott-meyers


**Suggested Reading:-**

**Item 25: Use std::move on rvalue references, std::forward on universal references.,** Effective Modern C++, Scott Meyers
**Item 30: Familiarize yourself with perfect forwarding failure cases,**Effective Modern C++, Scott Meyers
**Item 33: Use decltype on auto&& parameters to std::forward them.,**Effective Modern C++, Scott Meyers

# Operator Overloading, Templates & Exception Handling > Explicit Type Conversions > Explicit and Restricted Type Conversions

User defined to scalar conversions can be controlled through explicit, =delete as follows

- if operator T() is explicit, user defined to scalar conversion allowed only with the syntax T(object)
- if operator T() is deleted, no conversion from user defined to scalar is allowed

```
//Let's consider MyTime class, user define to scalar conversions
//may be controlled in of the following ways
operator int() {                    //1
  return hh*3600+mm*60+ss;
}
explicit operator int() {           //2
  return hh*3600+mm*60+ss;
}
operator int()=delete;              //3


//Usage:-
val=t1;                 //error in case of 2, allowed in case of 1
val=int(t1);            //allowed in case of 1 & 2
val=(int)t1;            //allowed in case of 1 & 2
val=t1.operator int(); //also works in case of 1 & 2,but not required
val=int{b1};            //error in case of 2
//None of the above are allowed in case of 3, i.e. =delete
```

Object creation with single parameter, Scalar to user defined conversions are controlled as follows

- If single argument constructor is explicit, it can't be used for scalar to user defined conversion and usage of = is not allowed during object creation, usage of parenthesis is must for object creation.
- If single argument constructor is deleted, object creation with single argument & scalar to user defined conversions are not allowed
- If overloaded assignment operator is deleted, scalar to user defined conversion is not allowed.

```
MyTime(int);          //1
explicit MyTime(int); //2
MyTime(int)=delete;   //3


MyTime t1(3720);      //error in case of 3, allowed in case of 1 or 2
MyTime t2=4000;       //error in any case of 2 or 3, allowed with 1
//in case of 3 no object creation allowed with single argument

MyTime& operator=(int);          //4
MyTime& operator=(int)=delete;   //5


t1=4500;              //error in any case of 5, allowed with 4
```

```
#include<iostream>

#include<vector>

class Sample {

int x = 2;

public:

  //conversion constructor (int to user defined)

  explicit Sample(int p):x(p) {std::cout<<"Sample(int p)"<<std::endl; }


   //Let's consider MyTime class, user define to scalar conversions

   //may be controlled in of the following ways

  //conversion operator: user defined to int

  operator int() {                  //1

    std::cout<<"operator () called"<<std::endl;

    return x*3600;

  }
```

```
//   explicit operator int() {          //2
//     return x*3600;
//   }
//   operator int()=delete;             //3


};


int main() {
Sample s1 = 2;
  return 0;
}
```

```
Compilation failed due to following error(s).
main.cpp: In function 'int main()':
main.cpp:25:13: error: conversion from 'int' to non-scalar type 'Sample' requested
 Sample s1 = 2;
             ^
```

**References:-**

https://en.cppreference.com/w/cpp/language/cast_operator
https://www.learncpp.com/cpp-tutorial/9-13-converting-constructors-explicit-and-delete/
https://isocpp.org/wiki/faq/cpp11-language-classes#explicit-conversion-operators

## User Defined Literals

Bookmark this page

**Usage:-**

- User defined literals allow, custom suffix  for better meaningful literals.
- UDLs can be implemented through operator overloading syntax (non member functions)
- Custom literals implemented on your own can start  with underscore(_) preferably.
- Suffix without underscore(_) are reserved for built-in literal suffix by language standards.

**Example:-**

```
long double operator"" _kg(long double n) {
  return n*1000;
}
long double operator"" _mg(long double n) {
  return n/1000;
}
long double operator"" _km(long double n) {
  return n*1000;
}
long double operator"" _deg(long double n) {
  return n*pi/180.0;
}
long double distance=5.6_km;         //5600 meters
long double weight=2.34_kg+5623_mg;  //2340+5.623 grams
long double nradians=45_deg;
```

```
#include <iostream>

#include <iomanip>

#include <complex>

#include <cmath>


using namespace std;


constexpr float pi = 3.14;


long double operator""_kg(long double n) {

  return n*1000;

}

long double operator""_mg(long double n) {

  return n/1000;

}

long double operator""_km(long double n) {

  return n*1000;

}
```

```
long double operator""_deg(long double n) {
  return n*pi/180.0;
}

int main()
{
    long double distance = 5.6_km;          //5600 meters
    cout<<"distance:"<<distance<<std::endl;

    long double weight = 2.34_kg + 5623.0_mg;  //2340+5.623 grams
    cout<<"weight:"<<weight<<std::endl;

    long double nradians = 45.0_deg;
    cout<<"nradians:"<<nradians<<std::endl;

     // defines the complex number: (10 + 2i)
    std::complex<double> mycomplex(10.0, 2.0);

    // prints the real part using the real function
    std::cout << "Real part: " << real(mycomplex) << std::endl;
    std::cout << "Imaginary part: " << imag(mycomplex) << std::endl;
    std::cout << "The absolute value of " << mycomplex << " is: ";
    std::cout << abs(mycomplex) << endl;
    std::cout << "The complex whose magnitude is " << 2.0;
    std::cout << " and phase angle is " << 0.5;
    std::cout << " is " << polar (2.0, 0.5) << endl;
    //Defined in inline namespace std::literals::complex_literals
```

```
    //operator""if
    //operator""i
    //operator""il
    using namespace std::complex_literals;
    std::cout << std::fixed << std::setprecision(1);

    std::complex<double> z1 = 1i * 1i;      // imaginary unit squared
    std::cout << "i * i = " << z1 << '\n';

    return 0;
}
```

```
distance:5600
weight:2345.62
nradians:0.785
Real part: 10
Imaginary part: 2
The absolute value of (10,2) is: 10.198
The complex whose magnitude is 2 and phase angle is 0.5 is (1.75517,0.958851)
i * i = (-1.0,0.0)
```

## Extern Templates

Bookmark this page

**Problem:-**

- When a template function/class is invoked with same argument in multiple source files, compiler instantiate the definition in each translation unit which causes an unwanted code bloat.

**Solution:-**

- This problem can be addressed by **extern templates** introduced in C++11, which reduces memory footprint of code by instantiating template code exactly once (per type of arguments)
- extern templates ensures that the instantiation doesn't takes place in such translation unit
- We need to ensure that instantiation takes place in of the translation unit

Let's see an example of normal templates which cause code bloats

Let's see an example of normal templates which cause code bloats

```
//fun.h:-
template<typename T>
void swap(T& r1,T& r2) {
  //some code
}

//f1.cpp
#include"fun.h"
void f1() {
  int a=10,b=20;
  myswap<int>(a,b);
}

//2.cpp
#include"fun.h"
void f2() {
  int x=10,y=20;
  myswap<int>(x,y);
}
```

```
//Here myswap definition is instantiated in each source file
//myswap is a weak symbol in each translation unit
//you can check symbol state by nm/objdump
```

Better code with extern templates:-

```
//f1.cpp
extern template void swap(int&,int&);
void f1() {
  int a=10,b=20;
  swap<int>(a,b);
}
//eventually myswap is an undefined symbol
//which is expected to be instantiated elsewhere exactly once

//ensure that myswap is not extern in one of the source files
//to make it available for all other units during linking
//let's add the following in main.cpp without extern prefix
template void myswap(int&,int&);
```

**Another example:-**

```
//f1.cpp,f2.cpp
extern template std::vector<int>;
//in one of the translation unit, say main.cpp
template std::vector<int>
```

//Foo.h

```
template<typename T>
void myswap(T& r1,T& r2) {
  //some code
}
```

//F1.cpp

```
#include"foo.h"

extern template void myswap(int&,int&);

void f1() {
```

```
  int a=10,b=20;
  myswap<int>(a,b);
}
```

//f2.cpp

```
#include"foo.h"

template void myswap(int&,int&);

void f2() {
  int x=10,y=20;
  myswap<int>(x,y);
}
int main()
{
    f1();
    f2();
    cout << "Hello World!" << endl;
    return 0;
}
```

```
g++ -g -std=c++11 f1.cpp f2.cpp
objdump -CS a.out
```

**References:-**

https://en.cppreference.com/w/cpp/language/class_template

https://isocpp.org/wiki/faq/cpp11-language-templates#extern-templates

## Variadic Templates

Bookmark this page

- variadic templates can take variable no.of arguments
- A better replacement for va_list va_arg,va_start,va_end from stdarg.h

**Example:-**

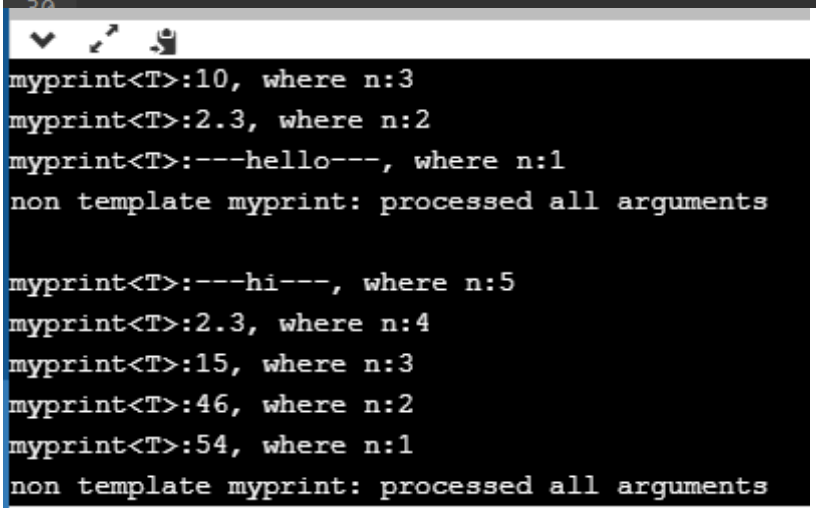
```
void myprint(int n) {          //required on reaching end of argument list
  std::cout << "processed all arguments\n";
}

template<typename T,typename... Args>
void myprint(int n,T val,Args... args) {
    std::cout << val << "\n";
    myprint(n-1,args...);
}
int main() {
    myprint(3,10,2.3f,"hello");
    myprint(5,"hello",2.3f,15,46,54);
}
```

```cpp
void myprint(int n) {          //required on reaching end of argument list
  std::cout << "non template myprint: processed all arguments\n";
}

template<typename T,typename... Args>
void myprint(int n,T val,Args... args) {
    std::cout << "myprint<T>:"<<val << ", where n:"<<n<<"\n";
    myprint(n-1,args...);
}

int main()
{
    myprint(3, 10, 2.3f,"---hello---");
    std::cout<<std::endl;
    myprint(5,"---hi---",2.3f,15,46,54);
    return 0;
}
```



```
myprint<T>:10, where n:3
myprint<T>:2.3, where n:2
myprint<T>:---hello---, where n:1
non template myprint: processed all arguments

myprint<T>:---hi---, where n:5
myprint<T>:2.3, where n:4
myprint<T>:15, where n:3
myprint<T>:46, where n:2
myprint<T>:54, where n:1
non template myprint: processed all arguments
```

Let's consider the following scenario,where one template type is used for other

A space is must prior to C++11

```
std::vector<std::complex<double>  > v1;
```

C++11 allows consecutive > symbols without space for templates

```
std::vector<std::complex<double>> v1;
```

## Aliasing templates

Bookmark this page

Aliasing can be given to template types with the help of **using** keyword as discussed in day-1

Recalling some examples from day-1:-

```
using ivector=std::vector<int>;

template<typename T1, typename T2,typename T3=float>
class Sample {  };

template<typename T>
using isample=Sample<T,int,int>;           //typedef won't work here
```

```
using IPoint=Point<int>
using IPointVector=std::vector<Point<int>>;  //std::vector<IPoint>
```

## noexcept keyword

Bookmark this page

- noexcept specifier - ensures that no exceptions can be thrown from the function.
- noexcept(true) has same effect as empty throw
- noexcept(false) means a function can throw any exceptions
- If any exception is still thrown when no except is specified **std::terminate** will be called

```
void test(args) noexcept;        //noexcept(true)
void test(args) noexcept(true);  //void test(args) throw()
void test(args) noexcept(false); //void test(args);
void test(args) noexcept(expr);  //exceptions can't be thrown if expr is true
```

- noexcept operator - it does a compile time check and returns true if given expression doesn't throw any exception.

```
45 int main()
46 {
47   MyArray arr(10);
48   try
49   {
50       int val=arr.at(5); //callin noexcept, passed value is less that maxlen(10), so no throw
51       int a=10,b=0,c;
52       c = mydiv(a, b);//calling noexcept(true) or noexcept, create throw situation by passing b=0: divide by zero
53                       //noexcept(false)  ==> exception will be thrown and captured in catch block
54   }
55   catch(std::exception& e)
56   {
57       std::cerr << e.what() << "\n";
58   }
59
60   return 0;
61 }
```

input

```
MyArray::at called
mydiv called
erminate called after throwing an instance of 'std::runtime_error'
 what():  div by zero
borted
```

**References:-**

https://en.cppreference.com/w/cpp/language/noexcept_spec
https://en.cppreference.com/w/cpp/language/noexcept

https://isocpp.org/wiki/faq/cpp11-language#noexcept
**Further Reading:-**
**Item 14: Declare functions noexcept if they won't emit exceptions**, Effective Modern C++, Scott Meyers
https://en.wikipedia.org/wiki/Template_metaprogramming
https://en.wikibooks.org/wiki/C%2B%2B_Programming/Templates/Template_Meta-Programming

### Stack/Array

Bookmark this page

Implement your own Stack/Array class with template parameters using following C++11/C++14 features

- Move operations
- deleted copy constructor,operator= to prevent cloning
- In class initialization, e.g. top=-1
- **Push** operations with **r-value** reference
- **Pop** operation which returns **r-value** reference
- Try to instantiate Stack class with non trivial type and check the significance of move semantics
- **emplace** operation as an alternative to regular push (hint:- placement new operator)
- extern templates while instantiating Stack objects
- >> while creating Stack from another template class

# STL Additions

### Initialization

Bookmark this page

- Construction of containers is possible using initializer lists
- This supports uniform initialization concept introduced in C++11

```
std::vector<int> v1{10,20,30,40,50};
std::array<int,5> a1{10,20,30,40,50};
std::list<int> v1{10,20,30,40,50};

std::map<int,std::string> books { {101,"c++"}, {102,"linux"},{103,"Qt5"}};

std::vector<std::string> vs{"red","green","blue","white","black"};

std::vector<Point> points { {0,0}, {1,1}, {2,3}, {3,4}, {1,5} };
```

## Simplified Traversals

Bookmark this page

Using auto keyword for type deduction and range based for loops container traversal is very simple from C++11

**Example:-**

```
std::vector<int> v1{10,20,30,40,50};
for(auto x:v1) {
  sum+=x;
}

for(auto& x:v1) {
  x*=0.1
}

for(auto iter=v1.begin();iter!=v1.end();++v1)
  std:cout << *iter << "\n";

std::for_each(v1.begin(),v1.end(), [](auto x) {
  sum+=x;
});
```

**Further Reading:-**

**Item 13: Prefer const_iterators to iterators.,** Effective Modern C++, Scott Meyers

---

## Move Inserts

Bookmark this page

- With move semantics, containers provide overloaded insert operations with r-value references
- Of course there is no difference between normal insert and move insert, if the underlying objects are of trivial type.
- For non trivial types these move operations allow better optimization

```
std::vector::push_back(T&);           //normal insert
std::vector::push_back(T&&);          //move insert

std::vector<Sample> v1;
Sample s1(10,20);
v1.push_back(s1);                     //normal insert
v1.push_back(Sample(11,12));          //move insert
v1.push_back(std::move(s1));          //move insert
//Of course there is no difference here, as Sample is a trivial type

std::vector<MyString> slist;
MyString s1("abcdxyz");
```

```
slist.push_back(s1);
slist.push_back(std::move(s1));       //move operation
slist.push_back("hello");             //move operation,matches r-value ref
```

## Move Operations

Bookmark this page

Containers are enabled with move constructor, move assignment operator for optimal operations

**Example:-**

```
std::vector<Sensor> getReadings(int len) {
  std::vector<Sensot> vtemp(len);
  for(auto& x:vtemp) v1.setReading(10+std::rand()%30);
  return vtemp;
}

std::vector<Sensor> v1=getValues(20);      //move operation
```

Prior to C++11 large vector need to be returned by value from getReadings to calling function, which involves duplication of underlying array (Assuming RVO not applied). From C++11 underlying array can be moved optimally.

## emplace operations

Bookmark this page

- Though move push operations are far efficient than conventional push, still few computations are required for move operations.Constructing objects in place of container elements can be still efficient than a move operations.
- **emplace**,**emplace_back** forwards given arguments to the constructor of underlying class, and constructs the object in the memory of container element only (using placement new operator)
- This eliminates construction of temporary objects and copying/moving them to container elements.
- emplace operations are designed with the the help of variadic templates internally

**Example:-**

```
std::list<Book> books;
books.push_back(Book(1001,"C++","Scott");
books.emplace_back(1002,"STL","Meyers");

std::vector<std::tuple<int,float,char>> tuples;
tuples.push_back(std::tuple(10,2.3f,'A'));
tuples.emplace_back(11,2.5f,'B'));
```

**Suggested Reading from Effective Modern C++, Scott Meyer:-**
**Item 42: Consider emplacement instead of insertion**
**Hands-on Check:-**

- Implement emplace operation for your own container (Day5- Custom Stack example hints)

**References:-**
https://en.cppreference.com/w/cpp/container/vector/emplace_back
https://www.geeksforgeeks.org/vectoremplace_back-c-stl/

## std::array

Bookmark this page

- std::array is fixed size container introduced in C++11
- It's a thin wrapper to raw arrays
- It gives on par performance with raw array when compared to std::vector

```
std::array<int,10> a1{10,20,30,40,50};
for(int i=0;i<a1.max_size();i++)
  a1[i]=rand()%100;   //a1.at(i)=rand()%100;
for(auto x:a1)
  sum+=x;
std::array<int,10> a2(10);

int *parr=a1.data();     //data() is also available for vectors now
```

**Tutorial:-**
https://thispointer.com/c11-stdarray-tutorial-and-examples/
https://www.geeksforgeeks.org/stdarray-in-cpp/

## unordered maps

Bookmark this page

- Terminology
  - Ordered containers - std::set, std::map, std::multiset, std::multimap
  - Unordered containers - std::unordered_set, std::unordered_map, std::unordered_multiset, std::unordered_multimap
- Unordered containers are implemented using hash tables internally
- Where as ordered containers are implemented using ordered balanced trees)
- Elements of these are sorted in any particular order (you can check through a simple traversal)
- Unordered containers allow faster lookup compared to ordered containers, almost O(1), contrast to O(logn) in case of ordered containers

**Further Reading - Tutorials:-**
https://www.geeksforgeeks.org/unordered_map-in-cpp-stl/
https://www.geeksforgeeks.org/unordered_map-in-stl-and-its-applications/
https://www.geeksforgeeks.org/unordered_multimap-and-its-application/
https://www.geeksforgeeks.org/unordered_multiset-and-its-uses/

## forward list

Bookmark this page

- **std::forward_list** is a single linked list implementation which can be traversed in forward direction only
- **std::forward_list** can minimize space and offers faster insertion, deletions compared to std::list
- **std::list** is a double linked list which can be traversed in both the directions.

**Further Reading:-**
https://www.geeksforgeeks.org/forward-list-c-set-1-introduction-important-functions/

## References

Click to add Bookmark this page

- **std::array**

  https://en.cppreference.com/w/cpp/container/array

  http://www.cplusplus.com/reference/array/array/

- **std::forward_list**

  https://en.cppreference.com/w/cpp/container/forward_list

  http://www.cplusplus.com/reference/forward_list/forward_list/

- **std::unordered_set**

  https://en.cppreference.com/w/cpp/container/unordered_set

  http://www.cplusplus.com/reference/unordered_set/unordered_set/

- **std::unordered_multiset**

  https://en.cppreference.com/w/cpp/container/unordered_multiset

  http://www.cplusplus.com/reference/unordered_set/unordered_multiset/

- **std::unordered_map**

  https://en.cppreference.com/w/cpp/container/unordered_map

  http://www.cplusplus.com/reference/unordered_map/unordered_map/

- **std::unordered_multimap**

  https://en.cppreference.com/w/cpp/container/unordered_multimap

  http://www.cplusplus.com/reference/unordered_map/unordered_multimap/

  **Examples:-**

  http://www.java2s.com/example/cpp/stl/unordered-map-index.html

  http://www.java2s.com/example/cpp/stl/unordered-set-index.html

  http://www.java2s.com/example/cpp/stl/forward-list-index.html

## std::tuple

Bookmark this page

- Tuples can hold 3 or more elements of heterogeneous types, instantiated by std::tuple class
- Tuples allow simple encapsulation without the need of named classes (adhoc packing)

```
std::tuple<int,int,std::string> t1(10,20,"abcd");
```

- std::get is used for accessing elements from tuple and

```
int val1=std::get<0>(t1);
int val2=std::get<1>(t2);
std::string val3=std::get<2>(t1);
```

- Passing invalid template parameters with **std::get** leads to compile time errors

```
auto val=std::get<x>(t1);     //template parameter must be compile time known
                              //error if x is not of a const or constexpr type
auto val=std::get<4>(t1);     //error due to invalid index
```

- std::tie is used for unpacking tuple

---

- std::tie is used for unpacking tuple

```
tie(val1,val2,val3)=t1;
tie(val1,ignore,val3)=t1;
```

- tuple_size returns no.of elements present int the tuples

```
 std::cout << std::tuple_size<decltype(t1)>::value << "\n";
```

- tuple_cat can combine multiple tuples

```
std::tuple<int,std::string> t1(10,"abc");
std::tuple<double,bool> t2(2.3,true);
auto t3=std::tuple_cat(t1,t2);
```

- Tuples allow returning multiple values from functions
- Tuples operations (e..g constructor) are implemented using variadic templates

**Tutorial:-**

https://www.geeksforgeeks.org/tuples-in-c/

https://thispointer.com/c11-stdtuple-tutorial-examples/

## std::make_tuple

Bookmark this page

- std::make_tuple constructs instances of std::tuple with given arguments
- This allows better type deduction using auto keyword

**Example:-**

```
std::tuple<int,std::string,int> t1=std::make_tuple(10,"abcd",20);

auto t2=std::make_tuple(11,"xyz",21);
auto x=std::get<0>(t2);
auto s1=std::get<1>(t2);
auto y=std::get<2>(t2);
```

**Tutorial:-**
https://thispointer.com/c11-make_tuple-tutorial-example/

References

Click to add Bookmark this page

**std::tuple:-**

https://en.cppreference.com/w/cpp/utility/tuple

http://www.cplusplus.com/reference/tuple/tuple/

**std::make_tuple:-**

https://en.cppreference.com/w/cpp/utility/tuple/make_tuple

http://www.cplusplus.com/reference/tuple/make_tuple/

**std::get:-**

https://en.cppreference.com/w/cpp/utility/tuple/get

http://www.cplusplus.com/reference/tuple/get/

## std::regex

Bookmark this page

Regular expressions library provide classes for matching, searching & replacing based on regular expressions.

Simple examples on basic regular expressions:-

**regex_match:-**

main.cpp

```cpp
int main()
{
    std::string str="abcdefxyz";
    std::regex p1("(abcd)(.*)");

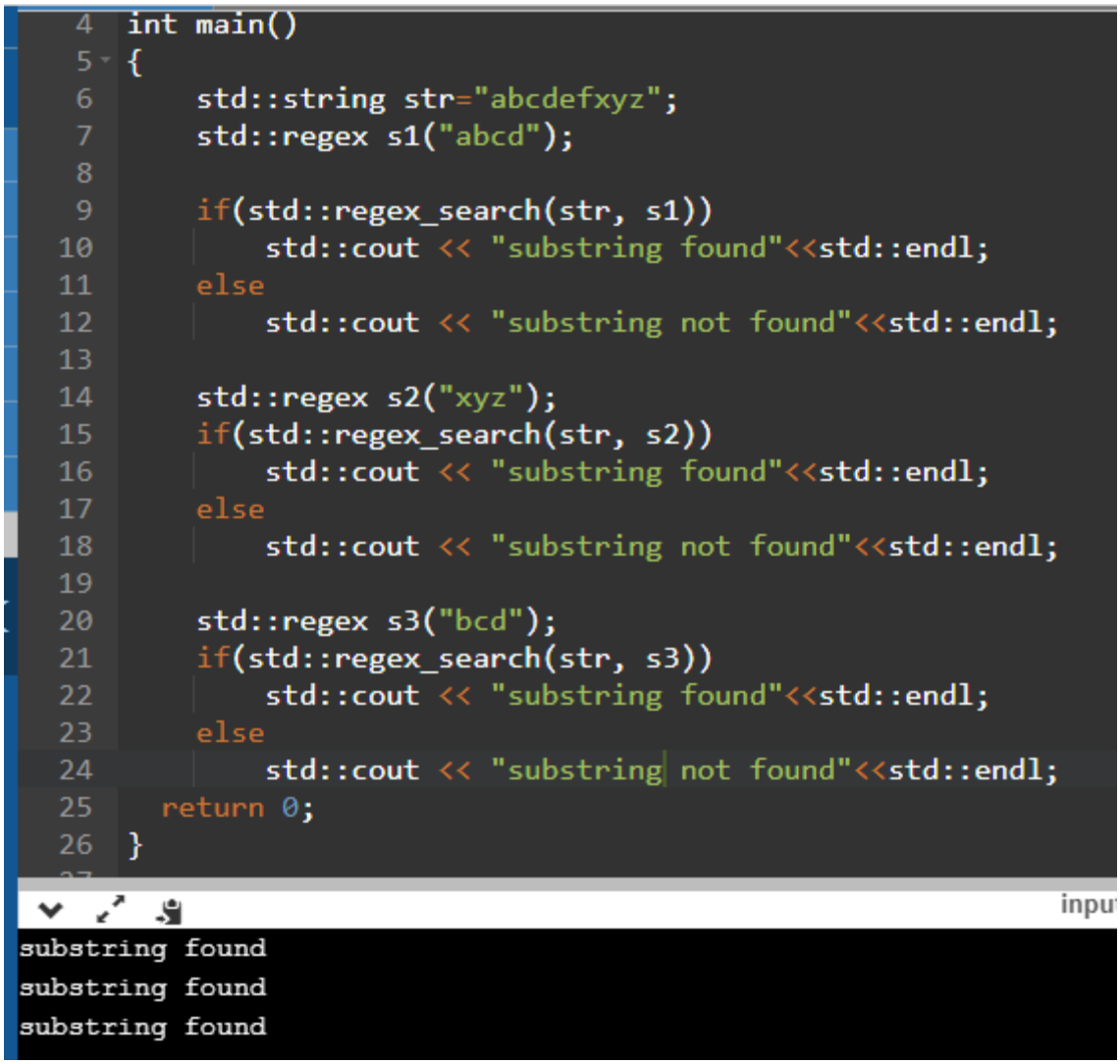
    if(std::regex_match(str, p1))
        std::cout << "pattern found"<<std::endl;
    else
        std::cout << "pattern not found"<<std::endl;

    std::regex p2("(.*)(xyz)");
    if(std::regex_match(str, p2))
        std::cout << "pattern found"<<std::endl;
    else
        std::cout << "pattern not found"<<std::endl;

    std::regex p3("(.*)(bcd)(.*)");
    if(std::regex_match(str, p3))
        std::cout << "pattern found"<<std::endl;
    else
        std::cout << "pattern not found"<<std::endl;
  return 0;
}
```

```
pattern found
pattern found
pattern found
```

**regex_search:-**



```cpp
int main()
{
    std::string str="abcdefxyz";
    std::regex s1("abcd");

    if(std::regex_search(str, s1))
        std::cout << "substring found"<<std::endl;
    else
        std::cout << "substring not found"<<std::endl;

    std::regex s2("xyz");
    if(std::regex_search(str, s2))
        std::cout << "substring found"<<std::endl;
    else
        std::cout << "substring not found"<<std::endl;

    std::regex s3("bcd");
    if(std::regex_search(str, s3))
        std::cout << "substring found"<<std::endl;
    else
        std::cout << "substring not found"<<std::endl;
  return 0;
}
```

```
substring found
substring found
substring found
```

main.cpp

```cpp
#include <iostream>
#include <regex>

int main()
{
    std::string fnames[] = {"libc.a", "libc.so", "libm.so", "sample.so"};
    std::regex pattern("lib[a-z]+\\.so");

    for (const auto &fname : fnames)
    {
      if(std::regex_match(fname, pattern))
        std::cout << fname <<std::endl;
    }
    return 0;
}
```

input

```
libc.so
libm.so
```

**Simple Replacement using regex_replace:-**

Simple Replacement using regex_replace:-

```
std::string str="This is standard C++";
std::regex pat("standard");
std::string rep("modern");
std::cout << std::regex_replace(str, pat, rep) << "\n";
```

The grammar followed for the regular expressions is very close to Java Script ECMA Syntax:-

## Further Reading

Click to add Bookmark this page

**Tutorials & Suggested Reading:-**

https://www.regular-expressions.info/stdregex.html

https://www.geeksforgeeks.org/regex-regular-expression-in-c/

https://www.geeksforgeeks.org/regex_error-in-c/

https://www.geeksforgeeks.org/regex_replace-in-cpp-replace-the-match-of-a-string-using-regex_replace/

https://solarianprogrammer.com/2011/10/12/cpp-11-regex-tutorial/

https://opensourceforu.com/2017/10/regular-expressions-programming-languages-story-c/

Click to add Bookmark this page
**std::regex:-**
https://en.cppreference.com/w/cpp/regex
http://www.cplusplus.com/reference/regex/
**Grammar:-**
https://en.cppreference.com/w/cpp/regex/ecmascript
http://www.cplusplus.com/reference/regex/ECMAScript/
http://ecma-international.org/ecma-262/5.1/#sec-15.10
**Additional Reference - Youtube videos by Jason Turner :-**
https://www.youtube.com/watch?v=IOxKjqC1Ozo
https://www.youtube.com/watch?v=7hfSyxNxFfo

## std::reference_wrapper

Bookmark this page

- reference_wrapper is useful to wrap references into copyable, assignable objects.
- It allows to use references as first class objects
- Overloaded operators on reference wrapper objects allow operations on underlying objects/variables/callables
- This is useful to model containers which can't hold references normally.

**Example:-**

```
std::vector<int> v1{10,20,30,40,50};

std::vector<int&> v2(v1.begin(), v1.end());  //Not allowed, error
std::vector<std::reference_wrapper<int>> v2(v1.begin(), v1.end());  //ok
++v1.at(0);
std::cout << v2.at(0) << "\n";
```

main.cpp

```cpp
int main()
{
    char a1 = 'g', b1 = 'e', c1  = 'e', d1 = 'k', e1 = 's';

    // creating an array of character "references":
    std::reference_wrapper<char> ref[] = {a1, b1, c1, d1, e1};

    for (char& s : ref)
          std::cout << s;

    std::cout<<std::endl;

    int a=10,b=20,c=30,d=40,e=50;
    std::reference_wrapper<int> arr[]={a,b,c,d,e};
    ++c; //==> changed from 30 to 31
    std::cout << arr[2] << "\n";
    d=45; //==> changed from 40 to 45
    std::cout << arr[3] << "\n";
    return 0;
}
```

input

```
geeks
31
45
```

## std::ref, std::cref

Bookmark this page

- std::ref and std::cref are helper functions which can generate std::reference_wrapper instances
- These are helpful even when a template function is expecting parameters to be passed by value
- **std::ref** generates **std::reference_wrapper<T>** from T
- **std::cref** generates **std::reference_wrapper<const T>** from T

**Example:-**

```
template<typename T>
void test(T& x,T& y) {
  ++x;
  ++y;
}
```

```
template<typename T>
void test(const T& x,const T& y) {
  //some code
}

//main
int a=10,b=20;
test(std::ref(a), std::ref(b));            //matches test(T&,T&)
test(std::cref(a), std::cref(b));          //matches test(const T&,const T&)
```

**Applicability of std::ref, std::cref:-**

- Arguments for std::make_pair
- Arguments for std::make_tuple
- Arguments for std::bind
- Arguments for std::thread

## References

Click to add Bookmark this page

- std::reference_wrapper

  https://en.cppreference.com/w/cpp/utility/functional/reference_wrapper

  http://www.cplusplus.com/reference/functional/reference_wrapper/

- std::ref, std::cref

  https://en.cppreference.com/w/cpp/utility/functional/ref

  http://www.cplusplus.com/reference/functional/ref/

  http://www.cplusplus.com/reference/functional/cref/

**Concept Checking:-**

- Identify new members added to the containers, e.g. data, shrink_to_fit of vector class
- Identify move semantics (r-value references) with container operations, e.g. push_back(T&&)
- Complexity analysis between std::vector and std::array
- Complexity analysis between ordered and unordered containers
- Possible implementations of few additions

  a) emplace_back b) std::tuple c) std::reference_wrapper

---

**Hands-on:-**

- Usage of std::for_each
- Write a program to match mobile number patterns using regular expressions
- Write a program to match vehicle registration numbers using regular expressions

### Random Number Generation

Click to add Bookmark this page

Pseudo-random number generation library:-

https://en.cppreference.com/w/cpp/numeric/random

https://en.cppreference.com/w/cpp/header/random

### Scoped Allocators

Click to add Bookmark this page

**References:-**

https://isocpp.org/wiki/faq/cpp11-library#scoped-allocator

## Smart Pointers

# Introduction

Bookmark this page

- Smart pointers simulates pointer types and provide access to actual objects being pointed to, We'll refer these original objects as **managed object** or **underlying object** in the further discussion.
- Overloaded operators like dereference(*), arrow(->) support this simulation
- Smart pointers ensure that managed object will be destroyed when pointer(s) goes out of scope
- C++11 introduced three types of smart pointers
  - std::unique_ptr
  - std::shared_ptr
  - std::weak_ptr
- **C++14** introduced **std::make_unique, std::make_shared** for simplified & safe creation of smart pointers.
- unique pointer ensures exclusive ownership to the managed object by instances of std::unique_ptr
- That means only one instance of unique_ptr can point to the original object at a time
- Shared pointers allows shared ownership to managed object by multiple instanced of std::shared_ptr, managed object will be released when all the owning shared pointers goes out of scope.Shared ownership is tracked with the help of reference/usage count on objects.

- Weak pointers allow access to managed object without taking ownership,i.e. without the increment of reference count
- A weak pointer may exist even when the managed object is released, extended life time of weak pointer doesn't extend life time of managed object

Let's consider the following class for the further discussion on smart pointers

```
class Sample {
  int x;
  int y;
  public:
  Sample()=default;
  Sample(int p,int q):x(p),y(q) { }
  void printxy() const {
    std::cout << x << "," << y << "\n";
  }
  ~Sample() {
    std::cout << "Sample--destr\n";
  }
};
```

# std::unique_ptr

Bookmark this page

- std::unique_ptr creates a smart pointer with exclusive ownership to managed object
- Managed object will be released when unique pointer goes out of scope (with a call to delete on managed object as default deleter, custom deleter can alter this behavior)
- Managed object is accessible through **operator***, **operator->**
- Copy semantics are restricted on unique pointer, justified for exclusive ownership
- When move semantics are applied ownership will be transferred from one object to other

**Simple example:-**

```
  std::unique_ptr<Sample> p1(new Sample(10, 20));
  p1->printxy();
  //std::unique_ptr<Sample> p2(ps);  //error,copy semantics are disabled
  std::unique_ptr<Sample> p2(std::move(ps));

  std::unique_ptr<Sample> p3;
  p3 = std::unique_ptr<Sample>(new Sample(15, 18));
```

```
  std::unique_ptr<Sample> p4;
  p4=p3;                              //error
```

**get** member gives access to raw pointer for underlying object (if any)

```
  std::unique_ptr<Sample> p2(std::move(ps));
  std::cout << p1.get() << "\n";
  std::cout << p2.get() << "\n";
```

**operator==, operator!=, operator bool** are available for comparison (null check of course)

```
std::unique_ptr<Sample> p1(new Sample(10, 20));
std::unique_ptr<Sample> p2=nullptr;
if(p1!=nullptr)    //if(p1) also works
  p1->printxy();
if(p2==nullptr)    //if(!p2) also works
  std::cout << "p2 has already released memory";
```

**reset** can change underlying object, i.e. release the old object and attach a new object

```
std::unique_ptr<Sample> ps(new Sample(10, 20));
ps->printxy();
ps.reset(new Sample(11,21);    //previous object got deleted
std::cout << ps.get() << "\n";
ps->printxy();
```

Invoking **reset(nullptr)** or assigning **nullptr** causes release of managed object

```
std::unique_ptr<Sample> ps(new Sample(10, 20));
ps->printxy();
ps.reset();    //ps=nullptr,ps.reset(nullptr) underlying object is deleted
std::cout << ps.get() << "\n";
```

Invoking **release** causes deletion of managed object and returns the raw pointer

```
std::unique_ptr<Sample> p1(new Sample(10, 20));
Sample *rawptr=p1.release();
```

Increment, Decrement operators are not allowed on unique pointers (any smart pointer), i.e. operator++, operator-- are not implemented on unique pointer

But underlying raw pointer may be incremented (if meaningful)

```
std::unique_ptr<Sample> p1(new Sample(10, 20));
++p1;         //error
Sample *rawptr=p1.get();
++rawptr;     //ok,but not meaningful for here(single object)
```

## std::make_unique

Bookmark this page

std::make_unique allows simplified creation of unique pointers by taking necessary arguments for the constructor.

```
std::unique_ptr<Sample> p3=std::make_unique<Sample>(10,20);
```

This also avoids dangerous heap problems as a resultant of wrapping raw pointers, For e.g. the following pointer creation caused side effects

```
Sample *rawptr=new Sample(10,20);
std::unique_ptr<Sample> up1(rawptr);
std::unique_ptr<Sample> up2(rawptr);
//if up1 goes out of scope, up2 will be dangled
//when up2 goes out of scope, attempt to already deleted object causes one more
issue
```

This problem can be prevented by creating smart pointers wrapped with new operator calls only,i.e. avoid wrapping raw pointers.

Also usage of make_unique guarantees that it'll wrap to new object only.

```
std::unique_ptr<Sample> up(new Sample(10,20);
//(or)
std::unique_ptr<Sample> up=std::make_unique<Sample>(10,20);
```

# Polymorphic Pointers

Bookmark this page

- up casting is possible with unique pointers, i.e.  managed objects can be of derived type.
- But they can access base class members only (similar capabilities of raw pointers of base type)

Let's consider **Account** class inherited by **SavingsAccount**

```
std::unique_ptr<Account> acc1(new SavingsAccount(id,name,bal));
std::unique_ptr<Account> acc2;
acc2=std::unique_ptr<SavingsAccount>(new SavingsAccount(id,name,bal));
std::unique_ptr<Account> acc3;
acc3=std::make_unique<SavingsAccount>(id,name,bal);
```

Let's consider **IMUSensor** class inherited by **Accelerometer**

```
std::unique_ptr<IMUSensor> reading1(new Accelerometer(x,y,z));
std::unique_ptr<IMUSensor> reading2;
reading2 = std::unique_ptr<Accelerometer>(new Accelerometer(x,y,z));
std::unique_ptr<IMUSensor> reading3=std::make_unique<Accelerometer>(x,y,z);
```

**Containers with upcasting:-**

```
std::vector<std::unique_ptr<Account>> accounts;
accounts.push_back(std::unique_ptr<Account>(new SavingsAccount(id,name,bal));
accounts.push_back(std::unique_ptr<SavingsAccount>(new SavingsAccount
(id,name,bal));
accounts.push_back(std::make_unique<SavingsAccount>(id,name,bal));

std::array<std::unique_ptr<IMUSensor>,maxsize> readings;
readings.push_back(std::unique_ptr<IMUSensor>(new Accelerometer(x,y,z));
readings.push_back(std::unique_ptr<Accelerometer>(new Accelerometer(x,y,z));
readings.push_back(std::make_unique<Acceleroeter>(x,y,z));
```

## Container example

Bookmark this page

**Containers holding pointers to heap objects:-**

without smart pointers:-

```
std::vector<Sample*> mylist;
mylist.push_back(new Sample(10, 20));
mylist.push_back(new Sample(11, 22));
//omission of delete on each object leads to memory leaks
```

with smart pointers:-

```
std::vector<std::unique_ptr<Sample>> slist;
slist.push_back(std::unique_ptr<Sample>(new Sample(10, 20)));
slist.push_back(std::make_unique<Sample>(10,20));
//no need to release the vector elements explicitly
```

## Array of Objects & Custom Deleters

Bookmark this page

- unique pointers can also manage array of objects
- Overloaded [] allows access to individual elements
- Default deleter for array of objects is delete[]

**Simple example:-**

```
std::unique_ptr<Sample[]> sparr(new Sample[10]);
std::cout << sparr.get() << "\n";
for(int i=0;i<10;i++)
  sparr[i].printxy();
//TODO: ensure all the objects are released
//      using valgrind or dummy print in destructor

std::unique_ptr<int[]> parr(new int[10]);
for(int i=0;i<10;i++)
  parr[i]=10*i;
```

**Analysis:-**

- Unique pointer for array of managed object vs container holding collection of unique pointers
- std::unique_ptr<T> vs std::unique_ptr<T[]>  -- access to managed objects, default deleter

---

Default deleter for unique pointers are as follows is delete, delete[] for single instance and array of instanced respectively

```
std::unique_ptr<T>(new T(args));           // delete
std::unique_ptr<T[]>(new T[n]);            // delete[]
```

This behavior can be altered using custom deleters which can be passed as second template argument as shown below

```
class MyDeleter {
  public:
   void operator() (Sample* rawptr) {
    delete rawptr;   //free(rawptr) or any other deallocation technique
  }
};
std::unique<Sample,MyDeleter> sp(new Sample(10,20));
//When sp goes out of scope, managed object is released using MyDeleter::operator
()
```

---

**Deleter for array of objects:-**

The following allocation causes wrong deletion

```
std::unique_ptr<Sample> sparr(new Sample[10]);
//Default deleter is delete instead of delete[] here
//Also [] is not allowed on this kind of pointers
//as unique pointer was created by Sample type instead of Sample[]
sparr[0].printxy(); //error

//You may obtain raw pointer to access array elements
Sample *rawptr=sparr.get();
for(i=0;i<n;i++) {
 rawptr->printxy(); //rawptr[i].printxy();
 ++rawptr;
}
```

This can be resolved using custom deleter as follows

```
template<typename T>
 class MyDeleter {
  public:
   void operator() (T* rawptr) {
    delete[] rawptr;
  }
};
std::unique_ptr<Sample,MyDeleter> sparr(new Sample[10]);
//Of course, [] is still not allowed not sparr
//You may go with raw pointers as shown in above snippet
```

## Further Reading

Click to add Bookmark this page

**Suggested Reading from Effective Modern C++ by Scott Meyers :-**

**Item 18: Use std::unique_ptr for exclusive-ownership resource management,**

**Item 21: Prefer std::make_unique and std::make_shared to direct use of new**

**Tutorials & Additional Reading:-**

https://www.learncpp.com/cpp-tutorial/15-5-stdunique_ptr/

https://www.geeksforgeeks.org/auto_ptr-unique_ptr-shared_ptr-weak_ptr-2/

https://thispointer.com/c11-unique_ptr-tutorial-and-examples/

https://www.fluentcpp.com/2017/08/22/smart-developers-use-smart-pointers-smart-pointers-basics/

https://www.fluentcpp.com/2017/08/29/custom-deleters/

https://www.fluentcpp.com/2017/09/01/make-custom-deleters-expressive/

## References

Click to add Bookmark this page

**std::unique_ptr:-**

https://en.cppreference.com/w/cpp/memory/unique_ptr

http://www.cplusplus.com/reference/memory/unique_ptr/

**std::make_unique:-**

https://en.cppreference.com/w/cpp/memory/unique_ptr/make_unique

# std::shared_ptr

Bookmark this page

- Unlike unique pointers, shared pointer allows shared ownership to managed objects, i.e.access to the managed object using multiple shared pointers.
- Managed object will be destructed (delete the raw pointer) when all the shared pointers goes out of scope.
- operator[] is not available with shared pointer to manage array of objects (unlike unique_ptr), i.e. std::shared_ptr<T[]> is anot available, but std::shared_ptr<T> can point to array of objects
- These points are same as unique_ptr
    - **get** member provides access to raw pointer for the managed object
    - Managed object is accessible through **operator***, **operator->**
    - **operator++**, **operator--** are not available

A Simple example:-

```
void test(std::shared_ptr<Sample> ptr) {
  ptr->printxy();
}
std::shared_ptr<Sample> p1(new Sample(10,20));
std::shared_ptr<Sample> p2(p1);
std::shared_ptr<Sample> p3(p1);
std::cout << p1.get() << "," << p2.get() << "," << p3.get() << "\n";
//object will be release when p1,p2,p3,ptr goes out of scope
```

- Reference count is maintained to track the shared ownership, i.e. when a new shared pointer is created for the managed object reference count will be incremented and gets decremented when shared pointer goes out of scope.
- use_count member of std::shared_ptr indicates the reference count
- reset member detaches shared pointer from the object and decrements reference count (same effect when assigned with nullptr)

```
std::shared_ptr<Sample> p1(new Sample(10,20));
std::cout << p1.use_count() << "\n"
std::shared_ptr<Sample> p2(p1);
std::cout << p1.use_count() << "\n";
std::cout << p2.use_count() << "\n";
std::shared_ptr<Sample> p3(p1);
```

```
std::cout << p1.use_count() << "\n";
p2.reset();  //p2=nullptr
std::cout << p1.use_count() << "\n";
```

operator==, operator!=, operator bool are used for comparison

```
std::shared_ptr<Sample> p1(new Sample(10,20));
std::shared_ptr<Sample> p2(p1);
if(p1==p2)
  std::cout << "Both p1,p2 points to same object"
p1.reset();   //p1=nullptr
//if(p1!=nullptr)  //if(p1)
//if(p1==nullptr)  //if(!p1)
```

Another Example to observe reference count as per the scope of shared pointers

```
std::shared_ptr<int> p1(new int());
std::cout << p1.use_count() << "\n";
{
  std::shared_ptr<int> p2(p1);
  std::cout << p1.use_count() << "\n";
  {
    std::shared_ptr<int> p3(p2);
    std::cout << p2.use_count() << "\n";
  }
  std::cout << p2.use_count() << "\n";
  p2.reset();  // p2=nullptr;
  std::cout << p1.use_count() << "\n";
}
std::cout << p1.use_count() << "\n";
```

## std::make_shared

Bookmark this page

Similar to std::make_unique, std::make_shared enables simplified creation of shared pointer instances

```
std::shared_ptr<Sample> sp=std::make_shared<Sample>(10,20);
std::shared_ptr<Account> sp=std::make_shared<SavingsAccount>(id,name,balance);
std::shared_ptr<Sensor> sp=std::make_shared<Accelerometer>(x,y,z);
```

Problem with raw pointers and usage of make_shared:-

As discussed during unique pointer, wrapping raw pointers into shared pointer also lead to dangerous situations.So always directly wrap the objects created by new operator or usage of make_shared is highly recommended.

LinkedlList Example

```
#include <iostream>

#include <memory>

#include <functional>
```

```
class Node {
  int m_value;
  std::shared_ptr<Node> m_next;
  public:
  Node(int val,std::shared_ptr<Node> ptr):m_value(val),m_next(ptr) {
     std::cout<<"Node constructor"<<std::endl;
  }
  ~Node()
  {
     std::cout<<"Node destructor"<<std::endl;
  }
  friend class LinkedList;
};

class LinkedList {
  std::shared_ptr<Node> pstart=nullptr;
  public:
  void insbeg(int val) {
    std::cout << "going to create:" << val <<std::endl;
    std::shared_ptr<Node> ptemp=std::make_shared<Node>(val,nullptr);
    ptemp->m_next=pstart;
    pstart=ptemp;
  }
  void delbeg() {
    std::cout << "going to delete:" << pstart->m_value<<std::endl;
    pstart=pstart->m_next;
```

```
  }
  void display() {
    std::shared_ptr<Node> pcur=pstart;
    for(; pcur!=nullptr; pcur=pcur->m_next) {
      std::cout << "value:"<<pcur->m_value << " ref count:"<<pcur.use_count() <<"\n";
    }
  }
};


int main()
{
    LinkedList list;
    list.insbeg(10);
    list.insbeg(20);
    list.display();
    return 0;
}
```

```
going to create:10
Node constructor
going to create:20
Node constructor
value:20ref count:2
value:10ref count:2
Node destructor
Node destructor
```

Further Reading

Click to add Bookmark this page

**Suggested Reading from Effective Modern C++ by Scott Meyers**

**Item 19: Use std::shared_ptr for shared-ownership resource management**

**Item 21: Prefer std::make_unique and std::make_shared to direct use of new**

**Tutorial:-**

https://www.learncpp.com/cpp-tutorial/15-6-stdshared_ptr/

https://thispointer.com//learning-shared_ptr-part-1-usage-details/

https://thispointer.com//shared_ptr-and-custom-deletor/

https://thispointer.com//how-shared_ptr-object-is-different-from-a-raw-pointer/

https://thispointer.com//create-shared_ptr-objects-carefully/

References

Click to add Bookmark this page

**std::shared_ptr:-**

https://en.cppreference.com/w/cpp/memory/shared_ptr

http://www.cplusplus.com/reference/memory/shared_ptr/

**std::make_shared:-**

https://en.cppreference.com/w/cpp/memory/shared_ptr/make_shared

http://www.cplusplus.com/reference/memory/make_shared/

## Default and Custom Deleters

Bookmark this page

default deleters are same as unique pointers for shared pointers

```
std::shared_ptr<T> (new T(args));            //delete
std::sahred_ptr<Sample> sp(new Sample(10,20));
```

This works well for single instance of managed object

But for array of objects delete[ ] is not the default deleter , as std::shared_ptr<T[]> is not possible with shared pointers.

```
std::shared_ptr<Sample> sp(new Sample[len]);
//default deleter is delete instead of delete[] here
```

This causes memory leaks & heap issues with invalid delete operation on array of objects

This problem can be solved with custom deleters as illustrated below

global functions as custom deleters

```
void mydeleter(Sample *pbase) {
  delete[] base;
}
std::shared_ptr<Sample> sp(new Sample[len],mydeleter);
```

Function objects as custom deleters

```
template<typename T>
 class ArrayDeleter {
  public:
   void operator() (T* rawptr) {
    delete[] rawptr;
  }
};
std::shared_ptr<Sample> p1(new Sample[10],ArrayDeleter<Sample>());
std::shared_ptr<int> p2(new int[len],ArrayDeleter<int>());
```

Lambdas as custom deleters

```
std::shared_ptr<Sample> p1(new Sample[10],[](auto *pbase) {
    delete[] pbase;
});
std::shared_ptr<int> p1(new int[len],[](auto *pbase) {
    delete[] pbase;
});
```

**TODO:-**

Compare memory leaks with and without custom deleter for array of objects

## std::weak_ptr

Bookmark this page

- Weak pointers holds a non owning reference to an object managed by another shared pointer
- Weak pointers doesn't participate in reference counting
- Weak pointers need to be converted into shared pointer to access underlying objects.
- Weak pointer may live beyond the life time of managed object ,but is considered as expired and can't access underlying object

```
};
int main() {

  //std::shared_ptr<Sample> sp(new Sample(10,20));
  std::shared_ptr<Sample> sp=std::make_shared<Sample>(10,20);
  std::weak_ptr<Sample> wp=sp;

  std::cout << "weak ptr use_count: " << wp.use_count() << std::endl;
  std::cout << "shared ptr use_count: " << sp.use_count() << std::endl;

  test_weak_ptr(wp);
  std::cout << "reset shared_ptr " << std::endl;
  sp.reset();              //managed object is released
  test_weak_ptr(wp);

  return 0;
}
```

input

```
Test--weak pointer is alive
stemp.use_count(): 2
10,20
reset shared_ptr
test--wp.expired?: 1
Test -- Weak pointer got expired
```

```
int main() {

 std::weak_ptr<Sample> wp;

 {
  std::shared_ptr<Sample> sp=std::make_shared<Sample>(10,20);
  wp=sp;
  test_weak_ptr(wp);
 } //managed object is released as sp is out of scope
  //But wp still exists in expired state, can't access object

 test_weak_ptr(wp);

  return 0;
}
```

input

```
test--wp.expired?: 0
Test--weak pointer is alive
stemp.use_count(): 2
10,20
test--wp.expired?: 1
Test -- Weak pointer got expired
```

## Further Reading

Click to add Bookmark this page

**Suggested Reading from Effective Modern C++ by Scott Meyers**

**Item 20: Use std::weak_ptr for std::shared_ptr like pointers that can dangle.**

**Tutorial:-**

https://www.geeksforgeeks.org/auto_ptr-unique_ptr-shared_ptr-weak_ptr-2/

http://www.modernescpp.com/index.php/std-weak-ptr

Custom smart pointer implementation (only for understanding internals, not for practical use):-

```
template <typename T>
class smart_ptr {
  T *ptr;
 public:
  smart_ptr() : ptr(nullptr) {}
  smart_ptr(T *temp) : ptr(temp) {}
  smart_ptr(const smart_ptr &) = delete;
  smart_ptr &operator=(const smart_ptr &) = delete;
  smart_ptr(smart_ptr &&temp) : ptr(temp.ptr) { temp.ptr = nullptr; }
  ~smart_ptr() {
    if (ptr != nullptr) delete ptr;
  }
  smart_ptr &operator=(smart_ptr &&temp) {
    ptr = temp.ptr;
    temp.ptr = nullptr;
  }
  T *get() { return ptr; }
  T *operator->() { return ptr; }
  T *operator*() { return ptr; }
};
```

```
47 int main()
48 {
49     smart_ptr<Sample> sp(new Sample(10));
50     sp->printxy(); //-> operator should be called
51     //smart_ptr<Sample> s1(sp); //use of deleted function 'smart_ptr::smart_ptr(const smart_ptr&) [with T = Sample]'
52     smart_ptr<Sample> s1(std::move(sp));
53     smart_ptr<Sample> s2;
54     //s2 = s1; //use of deleted function 'smart_ptr& smart_ptr::operator=(const smart_ptr&) [with T = Sample]'
55     s2 = std::move(s1);
56     s2->printxy();
57     return 0;
58 }
59
```

input

```
Sample(int p)
smart_pt(T*)
operator()
10
smart_pt(smart_ptr &&)
smart_pt()
=(smart_ptr &&)
operator()
10
~Sample
~smart_pt()
~smart_pt()
~smart_pt()
```

## make_smart

Bookmark this page

custom make_unique:-

```
template <typename T, class... Args>
std::unique_ptr<T> my_make_unique(Args &&... args) {
  return std::unique_ptr<T>(new T(args...));
}

std::unique_ptr<Sample> s3=my_make_unique<Sample>(12,24);
s3->printxy();
```

make_smart:-

```
template <typename T, class... Args>
smart_ptr<T> my_make_smart(Args &&... args) {
  return smart_ptr<T>(new T(args...));
}

smart_ptr<Sample> s4 = make_smart<Sample>(11, 22);
s4->printxy();
```

## shared pointer

Bookmark this page

Custom implementation of shared pointer, please extend/improve this with your own ideas for a better implementation

```
template<typename T>
class my_shared_ptr {
  T *m_rawptr;
  int *m_count;
  public:
  my_shared_ptr(T *ptr):m_rawptr(ptr) {
    m_count=new int(1);
  }
  my_shared_ptr(const my_shared_ptr& ref):m_rawptr(ref.m_rawptr),m_count
(ref.m_count) {
    ++(*m_count);
  }
  my_shared_ptr(my_shared_ptr&& rr):m_rawptr(rr.m_rawptr),m_count(rr.m_count) {
    rr.m_rawptr=nullptr;
  }
  ~my_shared_ptr() {
    release();
  }
  my_shared_ptr& operator=(const my_shared_ptr& ref) {
    m_rawptr=ref.m_rawptr;
    m_count=ref.m_count;
    ++(*m_count);
    return *this;
  }
  my_shared_ptr& operator=(my_shared_ptr&& rr) {
    m_rawptr=rr.m_rawptr;
    m_count=rr.m_count;
    rr.m_count=nullptr;
    return *this;
  }
  void release() {
    if(m_rawptr==nullptr) return;
    --(*m_count);
    if(*m_count==0) {
      delete m_rawptr;
      delete m_count;
    }
  }
  void reset() {
    release();
    m_rawptr=nullptr;
  }
  T* operator*() { return m_rawptr; }
  T* operator->() { return m_rawptr; }

  int use_count() const { return *m_count; }
  T* get() const { return m_rawptr; }
};
```

```
102  int main()
103  {
104      my_shared_ptr<Sample> p1(new Sample(10));
105      p1->printxy();
106      std::cout << p1.use_count() << "\n";
107      my_shared_ptr<Sample> p2(p1);
108      std::cout << p1.use_count() << "\n";
109      p1.reset();
110      std::cout << p1.use_count() << "\n";
111      return 0;
112
113  }
114
```

```
1
my_shared_ptr(const my_shared_ptr&)
2
reset()
release()
1
release()
~Sample
~my_shared_ptr()
release()
~my_shared_ptr()
```

## Test Your Skills

Bookmark this page

**Concept Checking:-**

- Memory leaks in absence of smart pointer (check with valgrind and dummy print in destructor)
- Do increment, decrement pointers are allowed on smart pointers?
- Do copy semantics  allowed on unique pointer?
- Smart pointer for array of objects vs container holding collection of smart pointers
- Significance of custom deleters with smart pointers
- Advantages of make_unique, make_shared (over direct use of std::unique_ptr, std:shared_ptr)
- Do move semantics are meaningful for shared pointers?
- Identify the threats with wrapping raw pointers into smart pointers
- Identify the cyclic dependency issues with shared pointer
- How weak pointers can avoid cyclic dependency or dangling issues
- If the life time of a weak pointer is extended, whether the life time of managed object also extends? i.e. do managed object may be deleted even if a weak pointer is pointing to it (via shared pointer).

## Widget Class & Hierarchy

Bookmark this page

Let's consider a widget hierarchy as follows to understand cyclic redundancy problem and a solution with the help of weak pointers.

- Each widget has a label property, you may assume few more properties
- Each widget has a member to track the parent (m_parent)
- Each widget has a member to track the child (m_child)
- Let's assume at most one child widget at any level for simplicity

Let's design simple Widget class without smart pointers initially.

```
class MyWidget {
  std::string m_label;
  MyWidget *m_parent=nullptr;
  MyWidget *m_child=nullptr;
  public:
  MyWidget(std::string label,MyWidget* parent):m_parent(parent),m_label(label) {
    if(m_parent)
      m_parent->m_child=this;
  }
  MyWidget(std::string label):m_label(label) {
  }
  MyWidget* getParent() { return m_parent; }
  MyWidget* getChild() { return m_child; }
  void display () const {
    std::cout << m_label << "\n";
  }
};
int main() {
  MyWidget w1("parent");
  MyWidget w2("child1",&w1);
  MyWidget w3("child2",&w2);
  //Traversing: child to parent, towards the origin
  MyWidget *ptr=&w3;
  while(ptr!=nullptr) {     //no parent for the origin
    ptr->display();
    ptr=ptr->getParent();
  }
  //Traversing : parent to child, from the origin
  ptr=&w1;
  while(ptr!=nullptr) {     //no more child for the last child widget
    ptr->display();
    ptr=ptr->getChild();
  }
  ptr=&w3;
```

```
  return 0;
}
```

# Shared Pointers - One Way

One way, i.e. only child tracks the parent but not vice versa

```
class MyWidget {
  std::string m_label;
  std::shared_ptr<MyWidget> m_parent;
  public:
  MyWidget(std::string label,std::shared_ptr<MyWidget>& parent)
                              :m_label(label),m_parent(parent) {
  }
  MyWidget(std::string label):m_label(label) {
  }
  std::shared_ptr<MyWidget>& getParent() { return m_parent; }
  void display () const {
    std::cout << m_label << "\n";
    std::cout << m_parent.use_count() << "\n";
  }
};
int main() {
  std::shared_ptr<MyWidget> wp1=std::make_shared<MyWidget>("parent");
  std::shared_ptr<MyWidget> wp2=std::make_shared<MyWidget>("child1",wp1);
  std::shared_ptr<MyWidget> wp3=std::make_shared<MyWidget>("child2",wp2);
  wp1->display();
  wp2->display();
  wp3->display();

  std::shared_ptr<MyWidget> ptr(wp3);
  while(ptr!=nullptr) {
    ptr->display();
    ptr=ptr->getParent();
  }
  return 0;
}
```

## Shared Pointers - Two Way

Bookmark this page

- Having shared pointers in both ways, i.e. child to parent and parent to child causes cyclic redundancy.
- That means reference count of shared count never becomes zero, thus underlying objects are not releases.
- This leads to memory leaks (check with valgrind)

So linking child widgets (via setChild)  causes cyclic dependencies as shown below

```
class MyWidget {
  std::string m_label;
  std::shared_ptr<MyWidget> m_parent;
  std::shared_ptr<MyWidget> m_child;
  public:
  MyWidget(std::string label,std::shared_ptr<MyWidget>&
parent)
                                    :m_parent(parent),m_label(labe
l) {
  }
  MyWidget(std::string label):m_label(label) {
  }

  void setChild(const std::shared_ptr<MyWidget>& ptr) {
m_child=ptr; }

  std::shared_ptr<MyWidget>& getParent() { return m_parent; }
  std::shared_ptr<MyWidget>& getChild()  { return m_child; }

  void display () const {
    std::cout << "Display  for " << m_label << "\n";
    std::cout << "parent count: " << m_parent.use_count() <<
"\n";
    std::cout << "pchild count: " << m_child.use_count() <<
"\n";
  }
};
int main() {
  std::shared_ptr<MyWidget> wp1(new MyWidget("parent"));
  std::shared_ptr<MyWidget> wp2(new MyWidget("child1",wp1));
  std::shared_ptr<MyWidget> wp3(new MyWidget("child2",wp2));

  wp1->setChild(wp2);    //TODO:better logic via constructor
  wp2->setChild(wp3);
  wp3->setChild(nullptr);

  wp1->display();
  wp2->display();
  wp3->display();

  std::shared_ptr<MyWidget> ptr(wp1);
  while(ptr!=nullptr) {
    ptr->display();
    ptr=ptr->getChild();
  }

  ptr=wp3;
  while(ptr!=nullptr) {
    ptr->display();
    ptr=ptr->getParent();
  }
```

```
   return 0;
}
```

## Solution with weak pointers

Bookmark this page

- Cyclic redundancy in previous problem can be resolved through weak pointers.
- Here parent tracking is done through strong pointer (shared pointer) and child tracking is done through weak pointers.
- Being weak pointer child tracker doesn't participate in reference counting, which eliminates cyclic redundancy.
- Also please observe the access to child widgets while traversing from origin to the last widget

```
class MyWidget {
  std::string m_label;
  public:
  std::shared_ptr<MyWidget> m_parent;
  std::weak_ptr<MyWidget> m_child;
  MyWidget(std::string label,std::shared_ptr<MyWidget>& parent)
                          :m_parent(parent),m_label(label) {
  }
  MyWidget(std::string label):m_label(label) {
  }
  void setChild(const std::shared_ptr<MyWidget>& ptr) { m_child=ptr; }

  std::shared_ptr<MyWidget>& getParent() { return m_parent; }
  std::weak_ptr<MyWidget>& getChild()  { return m_child; }

  void display () const {
    std::cout << "Display  for " << m_label << "\n";
    std::cout << "parent count: " << m_parent.use_count() << "\n";
    std::cout << "pchild count: " << m_child.use_count() << "\n";
  }
};
int main() {
  std::shared_ptr<MyWidget> wp1(new MyWidget("parent"));
  std::shared_ptr<MyWidget> wp2(new MyWidget("child1",wp1));
  std::shared_ptr<MyWidget> wp3(new MyWidget("child2",wp2));

  wp1->setChild(wp2);
  wp2->setChild(wp3);
  wp3->setChild(nullptr);

  wp1->display();
  wp2->display();
  wp3->display();

  std::weak_ptr<MyWidget> wptr(wp1);
```

```
 std::shared_ptr<MyWidget> sptr;
 while(true) {
  sptr=wptr.lock();
  if(!sptr) break;
  sptr->display();
  wptr=sptr->getChild();
 }

 ptr=wp3;
 while(ptr!=nullptr) {
  ptr->display();
  ptr=ptr->getParent();
 }
 return 0;
}
```

## Placement new operator

Bookmark this page

- Placement new operator allows construction of objects on pre allocated buffers rather than going for a heap block at run time.
- Object memory can be aligned to pre allocated buffers (may be pre filled with desired values,sometimes helpful for array of objects)
- This allows certain optimizations to construct the objects at pre allocated storage.
- This is also helpful to go with custom allocations, if regular new operator is not deterministic due to existing heap policies.

Let's see simple examples on placement new operator for primitive, userdefined types.

```
char mybuffer[128];  //char* mybuffer=new char[128];
//fill the buffer using memset or std::fill
int *parr=new (mybuffer) int[10];
for(int i=0;i<n;i++)
  std::cout << parr[i];

size_t offset=10*sizeof(int);
Sample *ps=new (mybuffer+offset) Sample(10,20);
```

**Think Beyond:-**

- can we call delete on parr, ps?
- can we call delete on mybuffer?

**Note:-** Emplace operations on STL containers introduced in C++11 typically uses placement new operator.We'll see this in further sections.

```
102 int main()
103 {
104     const int n  = 10;
105     char mybuffer[128];  //char* mybuffer=new char[128];
106     //fill the buffer using memset or std::fill
107     int *parr=new (mybuffer) int[10];
108     for(int i=0;i<n;i++)
109       std::cout << "i: "<<i<<", parr[i]:"<<parr[i]<<endl;
110
111     size_t offset=10*sizeof(int);
112     Sample *ps=new (mybuffer+offset) Sample(10);
113 }
```

input

```
i: 0, parr[i]:1
i: 1, parr[i]:0
i: 2, parr[i]:2
i: 3, parr[i]:0
i: 4, parr[i]:6299128
i: 5, parr[i]:0
i: 6, parr[i]:2098795797
i: 7, parr[i]:32731
i: 8, parr[i]:0
i: 9, parr[i]:0
Sample(int p)
```

can we call delete on parr, ps?
NO

```
102 int main()
103 {
104     const int n  = 10;
105     char mybuffer[128];  //char* mybuffer=new char[128];
106     //fill the buffer using memset or std::fill
107     int *parr=new (mybuffer) int[10];
108     for(int i=0;i<n;i++)
109       std::cout << "i: "<<i<<", parr[i]:"<<parr[i]<<endl;
110
111     size_t offset=10*sizeof(int);
112     Sample *ps=new (mybuffer+offset) Sample(10);
113     delete ps;
```

input

```
i: 3, parr[i]:0
i: 4, parr[i]:6299128
i: 5, parr[i]:0
i: 6, parr[i]:-413526763
i: 7, parr[i]:32722
i: 8, parr[i]:0
i: 9, parr[i]:0
Sample(int p)
~Sample
*** Error in `/home/a.out': free(): invalid pointer: 0x00007ffd0195e218 ***
Aborted
```

## Name mangling issues

Bookmark this page

**Discussion:-**

- How C++ supports overloading, but not in C when object code format is same for both the languages
- How to check symbol table from object code.

**Symbol table & nm command usage:-**

nm command is used to display symbol table from object files.

**Symbol:-**

- global variable or function name is considered as symbol, which are visible to linker.
- Local variables are not considered as symbols(converted as offsets within stack), thus not visible to linker

```
nm test.o
nm -C test.o
c++filt test.o

nm -C account.o
```

For following overloaded sum functions

```
int sum(int, int);
int sum(int, int, int);
float sum(float, float);
```

We can find the mangled names as (actual mangled name may vary according to compiler being used)

```
000000000000013c T _Z3sumff
000000000000010c T _Z3sumii
0000000000000120 T _Z3sumiii
```

For same functions, demangled names can be (using nm -C or c++filt)

```
000000000000013c T sum(float, float)
000000000000010c T sum(int, int)
0000000000000120 T sum(int, int, int)
```

**Hands-on:-**

**Terminal users:-**

Analyze symbol names for various types of functions.

- global function or member function
- const member functions
- const prefix on parameters
- generic code with templates

```
#include <memory>

constexpr double tmax=24;
const int constdata = 3;
int nonconstdata = 5;
static int staticdata = 5;


int sum(int, int) {}
int sum(int, int, int) {}
float sum(float, float){}

int main() {
    constexpr double pi=22.0/7.0;
    constexpr int maxsize=10;
  return 0;
}
```

```
$ nm test.o
00000000 b .bss
00000000 d .data
00000000 r .eh_frame
00000000 r .rdata
00000000 r .rdata$zzz
00000000 t .text
         U ___main
00000004 d __ZL10staticdata
00000010 r __ZL4tmax
00000018 r __ZL9constdata
00000004 r __ZN9__gnu_cxxL21__default_lock_policyE
```

```
00000008 r __ZStL13allocator_arg
00000000 r __ZStL19piecewise_construct
00000009 r __ZStL6ignore
00000000 T _main
00000000 D _nonconstdata
```

## extern "C" block

Bookmark this page

- extern "C" blocks are helpful to bring interoperability between C & C++ layers.
- The symbols in extern "C" blocks will undergo C coding conventions.
- This is especially helpful for designing callbacks where some C code in middle-ware trying to invoke C++ functionality in application layer.

```
extern "C" {
  void sum(int x,int y) {
     int z;
     z=x+y;
     cout << "z=" << z << "\n"
  }
}
```

A technique followed for mixed language (C & C++) headers

```
#ifdef __cplusplus
extern "C" {
#endif

//some code


#ifdef __cplusplus
}
#endif
```

**Hands-on:-**

- What is the symbol name generated for above sum function, check with nm tool.

**Further Reading :** - *(Mixing C and C++, extern "C" blocks)*
http://www.oracle.com/technetwork/articles/servers-storage-dev/mixingcandcpluspluscode-305840.html
https://www.geeksforgeeks.org/extern-c-in-c/
https://en.cppreference.com/w/cpp/language/language_linkage
https://isocpp.org/wiki/faq/mixing-c-and-cpp
https://yosefk.com/c++fqa/mixing.html

**Think Beyond:-**

- Do C++ features are allowed in extern "C " block?
- Do overloading allowed in extern "C" block?

# Return value mechanisms

Bookmark this page

**Return mechanisms:-**

- Return by value
- Return by address
- Return by reference

**Think Beyond:-**

- Impact of returning local variables, parameters (which are passed by value) by address/reference.
- Do return by value is efficient and safe always?
- Do return by address/reference is unsafe always?
- Converting return mechanisms into parameter passing.
- Do function calls are having l-value context?

Example : l-value context of functions returning reference:-

```
int& max(int& r1,int& r2)
{
    if(r1>r2)
        return r1;
    else
        return r2;
}

max(a,b)++;

max(a,b)=30;
```

**Note:-** This technique is useful while overloading some special operators like [], * w.r.t containers, iterators.

# Trivial, POD types

Bookmark this page

**Trivial class:-**

A class is known to be trivial,if it meets following conditions

- No virtual functions or virtual base class
- All data members are trivial without the need for any external representations or custom initialization (for e.g. no dynamic memory, file handling, network connections, database connections etc. are involved)
- All base classes(if any) must be trivial
- Objects of trivial class can be managed with compiler provided copy constructor, destructor, operator= as they can be copied by compiler implicit rules and no destructor code is required.
- You can observe that constructor body can be blank for trivial class, as all members can be initialized through initializer list trivially.
- C++11 provides **std::is_trivial** from type traits to check if a given class is trivial or not.

Example of a trivial class:-

```
class Simple
{
  int x;
  int y;
  public:
  Sample(int p,int q):x(p),y(q) {
  }
  void printxy() {
    std::cout << x << "," << y << "\n";
  }
};

//Check:-
std::cout << std::is_trivial<Simple>::value << "\n";
```

Example of a non trivial class:-

```
class Stack
{
  int *arr;
  int top;
  const int maxlen;
  public:
  Stack(int len):maxlen(len),top(-1) {
     arr=new int[maxlen];
  }
  ~Stack() {
     delete[] arr;
  }
  //remaining code
};

//Check:-
std::cout << std::is_trivial<Stack>::value << "\n";
```

**Suggested Reading:-**
https://www.geeksforgeeks.org/trivial-classes-c/
https://en.cppreference.com/w/cpp/types/is_trivial
https://en.cppreference.com/w/cpp/named_req/TriviallyCopyable
**Additional Reading:-**
**Std Layout:-**
https://en.cppreference.com/w/cpp/named_req/StandardLayoutType
https://en.cppreference.com/w/cpp/types/is_standard_layout
**Plain Old Data Structure (POD) Type:-**
http://www.drdobbs.com/c-made-easier-plain-old-data/184401508
https://en.cppreference.com/w/cpp/types/is_pod
**Note:-** POD Types are fully compatible with C structures

## Copy elison

Bookmark this page

Let's see an example on returning unnamed objects from a function (global function here)

```
#include<iostream>
class Sample {
  int x;
  int y;
public:
  Sample(int p,int q):x(p),y(q) {
    std::cout << "param ctor-->" << x << "," << y << "\n";
  }  //Also used for conversion,but not preferred
  Sample(const Sample& ref):x(ref.x),y(ref.y) {
    std::cout << "copy ctor-->" << x << "," << y << "\n";
  }
  ~Sample() {
    std::cout << "Destructor-->" << x << "," << y << "\n";
  }
  void printx() const {
    std::cout << x << "\n";
  }
};
Sample getSample(int a,int b) {
  return Sample(a,b);
}
int main() {
  Sample s1=getSample(10,20);
  Sample s2(getSample(11,12));
};
```

**Code Analysis:-**

- **Copy elison** is an optimization technique applied implicitly by the compiler.
- This elides(eliminates) multiple copies of unnamed objects using Return Value Optimization(RVO) technique.
- Here temporary object creation and intermediate copies are eliminated and compiler is permitted to construct the object directly where these expressions are otherwise copied to
- In above example objects are directly constructed at storage of s1,s2 eliminating temporary objects & intermediate copies.
  - unnamed object Sample(a,b) using parameterized constructor
  - return value from getSample based on unnamed object (copy constructor)
- Finally return value is copies to s1 or s2 using copy constructor.
- Compiler options can disable the elison technique, where above temporary objects are created.

**Hands-on Activity:-**

Prevent elison with compiler option **-fno-elide-constructors** and check the behavior of above code

```
22  int main() {
23    Sample s1=getSample(10,20);
24    Sample s2(getSample(11,12));
25    return 0;
26  };
27
```

```
param ctor-->10,20
param ctor-->11,12
Destructor-->11,12
Destructor-->10,20
```

**Further Reading:-**
https://en.cppreference.com/w/cpp/language/copy_elision
https://en.wikipedia.org/wiki/Copy_elision
https://www.geeksforgeeks.org/copy-elision-in-c/

**https://web.archive.org/web/20140205194657/http://cpp-next.com/archive/2009/08/want-speed-pass-by-value/**

## Basic RAII examples

Bookmark this page

Composition Example - parts as direct members:-

```
class Car
{
    int m_wheels;
    int m_speed;
    Engine m_engine;
    public:
    Car(int wheels,int speed,int strokes,int rpm):
    m_wheels(wheels),m_speed(speed),m_engine(strokes,rpm)
    {

    }
    Car(const Car& ref):m_wheels(ref.m_wheels),
            m_speed(ref.m_speed),m_engine(ref.m_engine)
    {

    }
    void display()
    {
        m_engine.display();
        std::cout << m_wheels << "," << m_speed << "\n";
    }
};
int main()
{
    Car c1(4,0,4,0);
    return 0;
}
```

Composition Example - parts using pointers:-

```
class Car
{
    int m_wheels;
    int m_speed;
    Engine *m_engine;
    public:
    Car(int wheels,int speed,int strokes,int rpm):
                                        m_wheels(wheels),m_speed(speed)
    {
        m_engine=new Engine(strokes,rpm);
    }
    Car(const Car& ref):m_wheels(ref.m_wheels),m_speed(ref.m_speed)
    {
        m_engine=new Engine(*ref.m_engine);
    }
    ~Car()
    {
        delete m_engine;
    }
```

```
    void display()
    {
        m_engine->display();
        std::cout << m_wheels << "," << m_speed << "\n";
    }
};
int main()
{
    Car c1(4,0,4,0);
    return 0;
}
```

## Conversion Constructors and elison

Bookmark this page

A constructor which takes single parameter can also act like conversion constructor (scalar of parameter type to class type conversion) in absence of explicit prefix.

```
class Sample {
  int x;
public:
  Sample(int p):x(p) {  }  //Also used for conversion,but not preferred
  Sample(const Sample& ref):x(ref.x) { }
  void printx() const {
    std::cout << x << "\n";
  }
};
int main() {
  Sample s1(10);
  Sample s2=15;   //works if single param ctor is not explicit
  s1=15;          //works if single param ctor is not explicit
  Sample s2(s1);
  Sample s3=s1;   //works if copy constructor is not explicit
};
```

Run above code by disabling constructor elison,i.e. by adding the g++ option ***-fno-elide-constructors***

You'll observe some interesting facts like, when constructor elison is prevented.

```
Sample s2=15;       ==> Sample s2(Sample(15));
s1=18;              ==> s1=Sample(18);
```

- Conversion constructors are not recommended by most of the coding standards, this can be prevented using explicit keyword prefix as follows.
- If conversion is desired implement overloaded operator=(T) , where T is a scalar type

```
class Sample {
  int x;
public:
  explicit Sample(int p):x(p) {  }  //Also used for conversion,but not preferred
  explicit Sample(const Sample& ref):x(ref.x) { }
  void printx() const {
    std::cout << x << "\n";
  }
};
```

```
int main() {
  Sample s1(10);
  Sample s2=15;   //error
  s1=15;          //error,not allowed
  Sample s2(s1);
  Sample s3=s1;   //error
};
```

## Overloading subscript operator

Bookmark this page

In this session we'll see an example of  implementing our own containers and demonstrating subscript operator (similar to STL containers):-

```
class MyArray {
    int *m_array;
    int *m_cursor;
    int m_len;
  public:
    MyArray(int len):m_len(len) {
        m_array=new int[m_len];
        m_cusror=m_array;
    }
    MyArray(const MyArray& ref):m_len(ref.m_len) {
        m_array=new int[m_len];
        m_cusror=m_array;
        //copy each element from ref.m_array to m_array
    }
```

```
    int& operator[](int index) {
        return m_array[index];
    }
    int& at(int index) {
        return m_array[index];
    }
};
int main() {
    MyArray a1(10);
    int i=5;
    a1[i]=56;                    //a1.operator[](i)=56, Also a1.at(i)=56
    std::cout << a1[i] << "\n";  //a1.operator[](i),
    for(int i=0;i<a1.size();i++)
       std::cout << a1.at(i) << "\n";
    return 0;
}
```

Overloading subscript, dereference operators is the basis for containers and iterators

**Further Reading:-**
http://www.learncpp.com/cpp-tutorial/98-overloading-the-subscript-operator/

```
#include <iostream>

template <typename T>
class MyIterator {
  T *m_cursor;

public:
  MyIterator() = default;
  MyIterator(T *ptr) : m_cursor(ptr) {}
  T &operator*() { return *m_cursor; }
  T *operator->() { return m_cursor; }
  MyIterator<T> &operator++() {
    ++m_cursor;
    return *this;
  }
  MyIterator<T> &operator--() {
    --m_cursor;
    return *this;
  }
  bool operator!=(const MyIterator<T> &ref) { return m_cursor != ref.m_cursor; }
  bool operator==(const MyIterator<T> &ref) { return m_cursor == ref.m_cursor; }
  /*MyIterator<T>& operator=(const MyIterator<T>& ref) {
    m_cursor=ref.m_cursor;
    return *this;
  }*/};

template <typename T>
class MyArray {
  T *m_array;
  int m_len;
```

```
public:
  MyArray(int len) : m_len(len) { m_array = new T[m_len]; }
  MyArray(const MyArray &ref) : m_len(ref.m_len) {
    m_array = new int[m_len];
    // copy each element from ref.m_array to m_array
  }
  T &operator[](int index) { return m_array[index]; }
  T &at(int index) { return m_array[index]; }
  MyIterator<T> begin() { return MyIterator<T>(m_array); }
  MyIterator<T> end() { return MyIterator<T>(m_array + m_len); }
};

int main() {
  MyArray<int> a1(10);
  MyIterator<int> iter;
  for (iter = a1.begin(); iter != a1.end(); ++iter)
    *iter = rand() % 100;
  for (auto x : a1)
    std::cout << x << "\n";
}
```

## Implementing Custom Iterators

Let's see an example of implementing custom iterators with **overloading of dereference, arrow** operators.

```
#include <iostream>

template <typename T>
class MyIterator {
  T *m_cursor;

public:
  MyIterator() = default;
  MyIterator(T *ptr) : m_cursor(ptr) {}
  T &operator*() { return *m_cursor; }
  T *operator->() { return m_cursor; }
  MyIterator<T> &operator++() {
    ++m_cursor;
    return *this;
  }
  MyIterator<T> &operator--() {
    --m_cursor;
    return *this;
  }
  bool operator!=(const MyIterator<T> &ref) { return m_cursor != ref.m_cursor; }
  bool operator==(const MyIterator<T> &ref) { return m_cursor == ref.m_cursor; }
  /*MyIterator<T>& operator=(const MyIterator<T>& ref) {
    m_cursor=ref.m_cursor;
    return *this;
```

```
 }*/};

template <typename T>
class MyArray {
  T *m_array;
  int m_len;

public:
  MyArray(int len) : m_len(len) { m_array = new T[m_len]; }
  MyArray(const MyArray &ref) : m_len(ref.m_len) {
    m_array = new int[m_len];
    // copy each element from ref.m_array to m_array
  }
  T &operator[](int index) { return m_array[index]; }
  T &at(int index) { return m_array[index]; }
  MyIterator<T> begin() { return MyIterator<T>(m_array); }
  MyIterator<T> end() { return MyIterator<T>(m_array + m_len); }
};

int main() {
  MyArray<int> a1(10);
  MyIterator<int> iter;
  //MyIterator<std::remove_reference<decltype(a1.at(0))>::type> iter;
  //also works, we'll discuss this technique in later chapters
  for (iter = a1.begin(); iter != a1.end(); ++iter)
    *iter = rand() % 100;
  for (auto x : a1)
    std::cout << x << "\n";
}
```

## Function call overloading

Bookmark this page

- If a class implements overloaded function call operator,instances of such class can be invoked as if they were normal functions.These are known as function call objects or functors
- Like regular functions overloaded function call operator can also work on different parameter combination (overloading)
- Function call objects are highly useful to provide custom logic for STL algorithms,also for binding few parameters.

Here are some examples on overloading function call operator,

```
template<class T>
class Sum {
    T operator()(T x,T y) {
        return x+y;
    }
    T operator()(T x,T y,T z) {
```

```
        return x+y+z;
    }
};
template<class T>
class Biggest {
    T operator()(T x,T y) {
        return x>y?x:y;
    }
    T operator()(T x,T y,T z) {
        return x>y?(x>z:x):(y>z?y:z);
    }
};
class Hello {
  std::string m_greeting;
  public:
  Hello(std::string greeting):m_greeting(greeting) { }
  void operator()()
    std::cout << "Hello " << m_greetings << "\n";
  }
  void operator()(std::string str)
    std::cout << "Hello " << str << "\n";
  }
}
int main() {
    Sum<int> s1;
    std::cout << s1(10,20) << "\n";
    std::cout << s1(10,20,30) << "\n";
    Sum<float> s2;
    std::cout << s2(2.3f,4.5f) << "\n";
    Biggest<int> b1;
    std::cout << b1(10,20);

    Hello h1("c++");
    h1();
    h1("Linux");

    return 0;
}
```

Function call overloading is the basis for function objects(functors),provided by STL

**Further Reading:-**

http://www.learncpp.com/cpp-tutorial/99-overloading-the-parenthesis-operator/

https://en.cppreference.com/w/cpp/utility/functional/function

https://www.geeksforgeeks.org/functors-in-cpp/

## Overloading new, delete

Bookmark this page

Overloading new, delete operators as member functions of a class.

```
class Sample {
  int x;
  int y;
 public:
  Sample(int p,int q):x(p),y(q) { }
  Sample(const Sample& ref):x(ref.x),y(ref.y) {}
  void* operator new(std::size_t nbytes) {
    void *pbase;
    pbase=std::malloc(nbytes);
    //pbase = ::operator new(nbytes);
    return pbase;
  }
  void* operator new[](std::size_t sz) {
    return std::malloc(sz);
    //return ::operator new(sz);
  }
  void operator delete(void* pbase) {
    std::free(pbase);
    //::operator delete(pbase);
  }
  void operator delete[](void* pbase) {
    std::free(pbase);
    //::operator delete(pbase);
  }
  void printxy() const {
    std::cout << x << "," << y << "\n";
  }
};
int main() {
  Sample *ptr;
  ptr=new Sample(10,20);
  ptr->printxy();
  delete ptr;

  Sample *psarr=new Sample[5];
  delete[] psarr;

  Sample *ps;
  ps= ::new Sample(10,20);
  ps->printxy();
  ::delete ps;
  return 0;
```

**Further Reading:-**

https://en.cppreference.com/w/cpp/memory/new/operator_new
https://thispointer.com/overloading-new-and-delete-operators-at-global-and-class-level/
https://www.geeksforgeeks.org/overloading-new-delete-operator-c/

Overloading global new,delete operators:-

```
#include<cstdlib>
void* operator new(size_t nbytes)
{
  void* pbase = std::malloc(nbytes);
  return pbase;
}
void operator delete(void* pbase)
{
  std::free(pbase);
}
void* operator new[](size_t nbytes)
{
  void* pbase = std::malloc(nbytes);
  return pbase;
}
void operator delete[](void* pbase)
{
  std::free(pbase);
}
int main() {
  int *ptr;
  int *parr;
  ptr=new int(12);          //::new int(12);
  parr=new int[len];        //::new int[len];
  //usage of *ptr, parr[i]
  delete ptr;               //::delete ptr;
  delete[] parr;            //::delete[] parr;
  return 0;
}
```

Passing arguments to overloaded new operator:-

```
void* operator new(size_t nbytes, int8_t val)
{
  void *pbase=std::malloc(nbytes);
  memset(pbase,nbytes,val);
  return pbase;
}
//Usage
parr=new ('A')char [20];
```

## Template Specialization

Bookmark this page

Templates can be specialized for specific data type, i.e. when functions or classes to be applied for such particular type, specialized version will be used instead of instantiating from general version.

Let's see different examples on

- Function template specialization
- Class template specialization
- Partial specialization
- Specialization vs Overloading

**Additional Reading:-**
https://www.geeksforgeeks.org/template-specialization-c/

**Function Template Specialization:-**

```
template<typename T>
void myswap(T& r1,T& r2)
{
   T temp=r1;
   r1=r2;
   r2=temp;
}
template<>
void myswap<char>(char& r1,char& r2)
{
   char temp=r1;
   r1=r2;
   r2=temp;
}

int a=10,b=20;
float x=5,y=15;
char c1='A',c2='B';
myswap(a,b);   //myswap<int>(a,b);
myswap(x,y);   //myswap<float>(x,y);
myswap(c1,c2); //myswap<char>(c1,c2);
```

**Additional Reading:-**
http://www.learncpp.com/cpp-tutorial/13-5-function-template-specialization/

Template class specialization:-

**Template Class Specialization**

```
template<typename T>
class Stack
{
  T *arr
  int top;
  //....
};
template<>
class Stack<char>
{
  char *parr;
  int top;
  //....
};
//main
Stack<int> s1(10);
Stack<float> s2(8);
Stack<char> s3(5);
```

**Additional Reading:-**
http://www.learncpp.com/cpp-tutorial/136-class-template-specialization/

---

```
template<typename T>
class Sample {
  T x;
  T y;
  //....
};
template<typename T>
class Sample<T*>
{
  T *p1;
  //...
}
```

**Additional Reading:-**
https://en.cppreference.com/w/cpp/language/partial_specialization
https://en.wikipedia.org/wiki/Partial_template_specialization
https://www.fluentcpp.com/2017/08/15/function-templates-partial-specialization-cpp/
http://www.learncpp.com/cpp-tutorial/137-partial-template-specialization/
http://www.learncpp.com/cpp-tutorial/13-8-partial-template-specialization-for-pointers/

**Overloading vs Specialization:-**

```
template<typename T>
void myswap(T& r1,T& r2) {
  std::cout << "myswap(T,T)\n";
}
template<typename T>
void myswap(T* p1,T* p2) {
  std::cout << "myswap(T*,T*)\n";
}
```

Now to implement myswap for char* type we have two options.

```
//Method1:- Overloading for char*
void myswap(char* p1,char* p2) {
  std::cout << "myswap(char*,char*)\n";
}
//(or)
//Method-2Specialization for char*
template<>
void myswap<char>(char* p1,char* p2) {
  std::cout << "myswap(char*,char*)\n";
}
```

## Additional Reading:-

http://www.gotw.ca/publications/mill17.htm

### Non-type parameters

A template non-type parameter is a special type of parameter that does not substitute for a type, but is instead replaced by a value. A non-type parameter can be any of the following:

- A value that has an integral type or enumeration
- A pointer or reference to a class object
- A pointer or reference to a function
- A pointer or reference to a class member function
- std::nullptr_t

In the following example, we create a non-dynamic (static) array class that uses both a type parameter and a non-type parameter. The type parameter controls the data type of the static array, and the non-type parameter controls how large the static array is.

```
#include <iostream>


template<typename T>

class Stack

{

  T *arr;
```

```
  int top;
public:
        Stack(int a): top(a) {}
  //....
};

template<>
class Stack<char>
{
  char *parr;
  int top;
public:
        Stack(int a): top(a) {}
  //....
};
```

//Using non type parameters

```
/*Stack<int, 5> s1;
Stack<int, 10> s2;
Stack<int, 15> s3;
Stack<int, 20> s4;
Stack<int, 25> s5;
Stack<int, 30> s6;
Stack<int, 35> s7;
Stack<int, 40> s8;
Stack<int, 45> s9;
Stack<int, 50> s10;
*/
```

// Without using non type parameters

```
Stack<int> s1(5);

Stack<int> s2(10);

Stack<int> s3(15);

Stack<int> s4(20);

Stack<int> s5(25);

Stack<int> s6(30);

Stack<int> s7(35);

Stack<int> s8(40);

Stack<int> s9(45);

Stack<int> s10(50);


int main()

{

        return 0;

}
```

// Without using non type parameters

```
PRIYARANJANS@L-5321 /cygdrive/d/C++/KPIT-Training/testproject
$ g++ -c test1.cpp

PRIYARANJANS@L-5321 /cygdrive/d/C++/KPIT-Training/testproject
$ size test1.o
   text    data     bss     dec     hex filename
    536       4      84     624     270 test1.o

PRIYARANJANS@L-5321 /cygdrive/d/C++/KPIT-Training/testproject
$ ls -lh test1.o
-rw-rw-r--+ 1 PRIYARANJANS Domain Users 2.3K Dec  5 18:01 test1.o

PRIYARANJANS@L-5321 /cygdrive/d/C++/KPIT-Training/testproject
$ du -sh test1.o
4.0K    test1.o
```

```
#include <iostream>

template <class T, int size> // size is the non-type parameter
class Stack
{
private:
   // The non-type parameter controls the size of the array
   T m_array[size];

public:
```

```
    T* getArray();

    T& operator[](int index)
    {
        return m_array[index];
    }
};

// Showing how a function for a class with a non-type parameter is defined outside of the class
template <class T, int size>
T* Stack<T, size>::getArray()
{
    return m_array;
}

Stack<int, 5> s1;
Stack<int, 10> s2;
Stack<int, 15> s3;
Stack<int, 20> s4;
Stack<int, 25> s5;
Stack<int, 30> s6;
Stack<int, 35> s7;
Stack<int, 40> s8;
Stack<int, 45> s9;
Stack<int, 50> s10;

int main()
{

        return 0;
}
```

```
PRIYARANJANS@L-5321 /cygdrive/d/C++/KPIT-Training/testproject
$ size test1.o
   text    data     bss     dec     hex filename
    256       4    1248    1508     5e4 test1.o

PRIYARANJANS@L-5321 /cygdrive/d/C++/KPIT-Training/testproject
$ ls -lh test1.o
-rw-rw-r--+ 1 PRIYARANJANS Domain Users 1.6K Dec  5 18:33 test1.o

PRIYARANJANS@L-5321 /cygdrive/d/C++/KPIT-Training/testproject
$ du -sh test1.o
4.0K    test1.o
```

One noteworthy thing about the above example is that we do not have to dynamically allocate the m_array member variable! This is because for any given instance of the StaticArray class, size is actually constant. For example, if you instantiate a StaticArray<int, 12>, the compiler replaces size with 12. Thus m_array is of type int[12], which can be allocated statically.

This functionality is used by the standard library class std::array. When you allocate a std::array<int, 5>, the int is a type parameter, and the 5 is a non-type parameter!

# Analysing Virtual Tables

Bookmark this page

Virtual Tables & Offsets of virtual functions:-

**Code Scenario-1:-**

```
class A {
 public:
  virtual void f1();
  virtual void f2();
};
class B:public A {
 public:
  virtual void f1();
  virtual void f3();
};
```

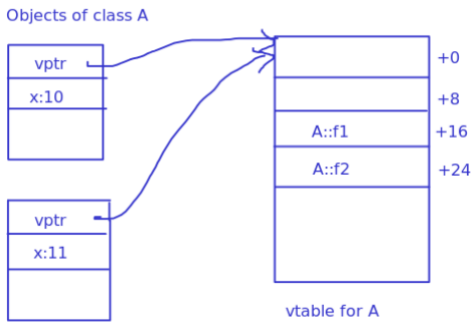
**Hands-on Activity:-**

Compile the above code with ***-fdump-class-hierarchy*** option of g++ which dumps virtual tables in a file with extension of **.002t.class*. In the generated dump file, vtable entries for class A & B looks like

```
Vtable for A
A::_ZTV1A: 4u entries
0     (int (*)(...))0
8     (int (*)(...))(& _ZTI1A)
16    (int (*)(...))A::f1
24    (int (*)(...))A::f2
```

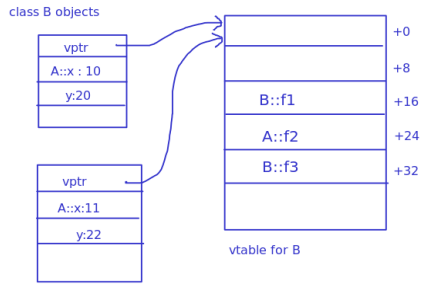
```
Vtable for B
B::_ZTV1B: 5u entries
0     (int (*)(...))0
8     (int (*)(...))(& _ZTI1B)
16    (int (*)(...))B::f1
24    (int (*)(...))A::f2
32    (int (*)(...))B::f3
```

You may visualize above entries with the help of following diagrams




Vtable for A
A::_ZTV1A: 4 entries
0 (int (*)(...))0
4 (int (*)(...))(& _ZTI1A)
8 (int (*)(...))A::f1
12 (int (*)(...))A::f2

Class A
  size=4 align=4
  base size=4 base align=4
A (0x0xfeb102a0) 0 nearly-empty
   vptr=((& A::_ZTV1A) + 8)




Vtable for B
B::_ZTV1B: 5 entries
0     (int (*)(...))0
4     (int (*)(...))(& _ZTI1B)
8     (int (*)(...))B::f1
12    (int (*)(...))A::f2
16    (int (*)(...))B::f3

Class B
  size=4 align=4
  base size=4 base align=4
B (0x0xfedaa740) 0 nearly-empty
   vptr=((& B::_ZTV1B) + 8)
 A (0x0xfeb10380) 0 nearly-empty
     primary-for B (0x0xfedaa740)

---

**Code Scenario-2:-**

```
class A {
 public:
  virtual void f1();
  virtual void f2();
  virtual void f3();
};
class B:public A {
 public:
  virtual void f1();
  virtual void f2();
  virtual void f4();
  virtual void f5();
};

class C:public B {
 public:
  virtual void f1();
  virtual void f4();
  virtual void f6();
};
```

Vtable for A

```
A::_ZTV1A: 5 entries
0     (int (*)(...))0
4     (int (*)(...))(& _ZTI1A)
8     (int (*)(...))A::f1
12    (int (*)(...))A::f2
16    (int (*)(...))A::f3

Class A
   size=4 align=4
   base size=4 base align=4
A (0x0xfeb10380) 0 nearly-empty
    vptr=((& A::_ZTV1A) + 8)
```

=➔ Just by adding virtual destructor in A will change like below

```
class A {
 public:
  virtual void f1();
  virtual void f2();
  virtual void f3();
  virtual ~A();
};

Vtable for A
A::_ZTV1A: 7 entries
0     (int (*)(...))0
4     (int (*)(...))(& _ZTI1A)
8     (int (*)(...))A::f1
12    (int (*)(...))A::f2
16    (int (*)(...))A::f3
20    (int (*)(...))A::~A
24    (int (*)(...))A::~A

Class A
   size=4 align=4
   base size=4 base align=4
A (0x0xfeb00460) 0 nearly-empty
    vptr=((& A::_ZTV1A) + 8)



Vtable for B
B::_ZTV1B: 7 entries
0     (int (*)(...))0
4     (int (*)(...))(& _ZTI1B)
8     (int (*)(...))B::f1
12    (int (*)(...))B::f2
16    (int (*)(...))A::f3
20    (int (*)(...))B::f4
24    (int (*)(...))B::f5

Class B
   size=4 align=4
   base size=4 base align=4
B (0x0xfedaa740) 0 nearly-empty
    vptr=((& B::_ZTV1B) + 8)
  A (0x0xfeb10460) 0 nearly-empty
      primary-for B (0x0xfedaa740)
Change due to virtual destructor in parent A
```

```
Vtable for B
B::_ZTV1B: 9 entries
0     (int (*)(...))0
4     (int (*)(...))(& _ZTI1B)
8     (int (*)(...))B::f1
12    (int (*)(...))B::f2
16    (int (*)(...))A::f3
20    (int (*)(...))B::~B
24    (int (*)(...))B::~B
28    (int (*)(...))B::f4
32    (int (*)(...))B::f5

Class B
   size=4 align=4
   base size=4 base align=4
B (0x0xfedaa740) 0 nearly-empty
    vptr=((& B::_ZTV1B) + 8)
  A (0x0xfeb00540) 0 nearly-empty
      primary-for B (0x0xfedaa740)

Vtable for C
C::_ZTV1C: 8 entries
0     (int (*)(...))0
4     (int (*)(...))(& _ZTI1C)
8     (int (*)(...))C::f1
12    (int (*)(...))B::f2
16    (int (*)(...))A::f3
20    (int (*)(...))C::f4
24    (int (*)(...))B::f5
28    (int (*)(...))C::f6

Class C
   size=4 align=4
   base size=4 base align=4
C (0x0xfedaa7c0) 0 nearly-empty
    vptr=((& C::_ZTV1C) + 8)
  B (0x0xfedaa800) 0 nearly-empty
      primary-for C (0x0xfedaa7c0)
    A (0x0xfeb10508) 0 nearly-empty
        primary-for B (0x0xfedaa800)
```

change due to adding virtual destructor in parent A

```
Vtable for C
C::_ZTV1C: 10 entries
0     (int (*)(...))0
4     (int (*)(...))(& _ZTI1C)
8     (int (*)(...))C::f1
12    (int (*)(...))B::f2
16    (int (*)(...))A::f3
20    (int (*)(...))C::~C
24    (int (*)(...))C::~C
28    (int (*)(...))C::f4
32    (int (*)(...))B::f5
36    (int (*)(...))C::f6

Class C
   size=4 align=4
   base size=4 base align=4
C (0x0xfedaa800) 0 nearly-empty
```

```
    vptr=((& C::_ZTV1C) + 8)
  B (0x0xfedaa840) 0 nearly-empty
      primary-for C (0x0xfedaa800)
    A (0x0xfeb005e8) 0 nearly-empty
        primary-for B (0x0xfedaa840)
```

```
class D: public A {
            int i;
};

Vtable for D
D::_ZTV1D: 7 entries
0     (int (*)(...))0
4     (int (*)(...))(& _ZTI1D)
8     (int (*)(...))A::f1
12    (int (*)(...))A::f2
16    (int (*)(...))A::f3
20    (int (*)(...))D::~D
24    (int (*)(...))D::~D

Class D
   size=8 align=4
   base size=8 base align=4
D (0x0xfedaa8c0) 0
    vptr=((& D::_ZTV1D) + 8)
  A (0x0xfeb00690) 0 nearly-empty
      primary-for D (0x0xfedaa8c0)
```

```
class E
{
            int j;
};

Class E
   size=4 align=4
   base size=4 base align=4
E (0x0xfeb00738) 0
```

```
class F
{
};

Class F
   size=1 align=1
   base size=0 base align=1
F (0x0xfeb00770) 0 empty
```

**Additional Reading:-**

https://thispointer.com/when-should-i-use-virtual-functions-in-c/

https://thispointer.com//how-virtual-functions-works-internally-using-vtable-and-vpointer/

## Virtual Destructors

Bookmark this page

**Code Scenario:-**

```
class A {
 protected:
  int m_len;
 public:
  A(int len):m_len(len) { }
  ~A() { }    //virtual ~A()
  virtual void fill() { }
  virtual void print() { }
};
class B:public A {
  int *m_buf;
 public:
  B(int len):A(len) {
    m_buf=new int[m_len];
  }
  ~B() {
    delete[] m_buf;
  }
  virtual void fill() { }
  virtual void print() { }
};
int main() {
  A* ptr=new B(10);
  ptr->fill();
  ptr->print();
  delete ptr;
}
```

**Hands-on Activity:-**

- Using **valgrind** detect the memory leaks in absence of virtual destructor for above code scenario
- Analyze the offset for virtual destructor in vtables

**Suggested Reading:-**
http://www.learncpp.com/cpp-tutorial/123-virtual-destructors-virtual-assignment-and-overriding-virtualization/

# RTTI & Type Conversions

```
20  int main()
21  {
22  int x;
23  float y;
24  const int max=10;
25
26  Box b1(10,12,5);
27  const Box *pb=&b1; //new Box(10,12,5);
28  const Box &rb=b1;
29
30  std::cout << "typeid(x).name():" <<typeid(x).name() << "\n";
31  std::cout << "typeid(y).name():" <<typeid(y).name() << "\n";
32  std::cout << "typeid(b1).name():"<<typeid(b1).name() << "\n";
33  std::cout << "typeid(pb).name():"<<typeid(pb).name() << "\n";
34  std::cout << "typeid(*pb).name():"<<typeid(*pb).name() << "\n";
35  std::cout << "typeid(rb).name():"<<typeid(rb).name() << "\n";
36
37  return 0;
```

input | stdout

Compiled Successfully. memory: 2684 time: 0.01 exit code: 0

```
typeid(x).name():i
typeid(y).name():f
typeid(b1).name():3Box
typeid(pb).name():PK3Box
typeid(*pb).name():3Box
typeid(rb).name():3Box
```

```cpp
int main()
{
//let's assume B is derived by A and maintains virtual tables
A a1(10);
B b1(12,18);

A *p1=&a1;  //new A(10);
A *p2=&b1;  //new B(12,18);
B *p3=&b1;  //new B(15,25);

A& r1=a1;
A& r2=b1;
B& r3=b1;

std::cout <<"typeid(a1).name():"<< typeid(a1).name() << "\n";
std::cout << "typeid(b1).name():"<<typeid(b1).name() << "\n";

std::cout << "typeid(*p1).name():"<<typeid(*p1).name() << "\n";
std::cout << "typeid(*p2).name():"<<typeid(*p2).name() << "\n";
std::cout << "typeid(*p3).name():"<<typeid(*p3).name() << "\n";

std::cout << "typeid(r1).name():"<<typeid(r1).name() << "\n";
std::cout << "typeid(r2).name():"<<typeid(r2).name() << "\n";
std::cout << "typeid(r3).name():"<<typeid(r3).name() << "\n";

return 0;
```

Compiled Successfully. memory: 2764 time: 0.08 exit code: 0

```
typeid(a1).name():1A
typeid(b1).name():1B
typeid(*p1).name():1A
typeid(*p2).name():1B
typeid(*p3).name():1B
typeid(r1).name():1A
typeid(r2).name():1B
typeid(r3).name():1B
```

**Suggested Reading:-**
https://en.wikipedia.org/wiki/Run-time_type_information
http://www.cplusplus.com/doc/tutorial/typecasting/
https://en.cppreference.com/w/cpp/language/typeid
http://www.cplusplus.com/reference/typeinfo/type_info/
**Examples:-**
http://www.java2s.com/example/cpp/class/typeid-index.html

# dynamic_cast

Bookmark this page

**Code Scenario:-**

```
A a1(10);
B b1(10,20);

A *p1,*p2;
B *p3;
p1=&a1;
p2=&b1;
p3=&b1;

A* pa1=dynamic_cast<A*>(&a1);    //&a1
A* pa2=dynamic_cast<A*>(p1);     //p1:&a1
A* pa3=dynamic_cast<A*>(&b1);    //&b1
A* pa4=dynamic_cast<A*>(p2);     //p2:&b1
A* pa5=dynamic_cast<A*>(p3);     //&b1
```

```
B* pb1=dynamic_cast<B*>(&a1);    //NULL
B* pb2=dynamic_cast<B*>(&b1);    //&b1
B* pb3=dynamic_cast<B*>(p1);     //NULL
B* pb4=dynamic_cast<B*>(p2);     //&b1
B* pb5=dynamic_cast<B*>(p3);     //&b1
B* pb6=dynamic_cast<B*>(pa1);    //NULL
B* pb7=dynamic_cast<B*>(pa2);    //NULL
B* pb8=dynamic_cast<B*>(pa3);    //&b1
B* pb9=dynamic_cast<B*>(pa4);    //&b1
B* pb10=dynamic_cast<B*>(pa5);    //&b1

A& r1=a1;
A& r2=b1;
B& r3=b1;

A& ra1=dynamic_cast<A&>(a1);
A& ra2=dynamic_cast<A&>(b1);
A& ra3=dyanmic_cast<A&>(r1);
A& ra4=dynamic_cast<A&>(r2);
A& ra5=dynamic_cast<A&>(r3);
```

```
B& rb1=dynamic_cast<B&>(a1);    //throws std::bad_cast exception
B& rb2=dynamic_cast<B&>(b1);    //rb:b1
B& rb3=dynamic_cast<B&>(r1);    //throws std::bad_cast exception
B& rb4=dynamic_cast<B&>(r2);    //rb:b1
B& rb5=dynamic_cast<B&>(r3);    //rb:b1
B& rb6=dynamic_cast<B&>(ra1);   //std::bad_cast
B& rb7=dynamic_cast<B&>(ra2);   //ok
B& rb8=dynamic_cast<B&>(ra3);   //std::bad_cast
B& rb9=dynamic_cast<B&>(ra4);   //ok
B& rb10=dynamic_cast<B&>(ra5);   //ok
```

**Hands-on Activity:-**

- Extend the above code by adding another class C, which inherits from B and test the dynamic_cast with various combinations of pointers & references.

**Suggested Reading:-**

- http://www.learncpp.com/cpp-tutorial/12-9-dynamic-casting/
- https://en.cppreference.com/w/cpp/language/dynamic_cast

**static_cast**

static_cast can perform down casting, which will be unsafe if underlying object is not compatible with pointer/reference type.

```
A* p1=new A(10);
A* p2=new B(10,20);
B *p3=new B(11,22);

B* p4 = static_cast<B*>(p1);          //no error, but unsafe
                                      //if we access any derived members through
p4
B* p5 = static_cast<B*>(p2);          //safe
B* p6 = static_cast<B*>(p3);

B& r1 = static_cast<B&>(*p1);         //unsafe
B& r2 = static_cast<B&>(*p2);
B& r3 = static_cast<B&>(*p3);
```

**Another Scenario:-**

```
//Let's consider two derived classes of Employee, which are Engineer, Manager
Manager *mptr=new Engineer; //error

//this can be achieved by (it could be unsafe without proper care)
Employee *base_ptr=new Engineer;
Manager *mptr=static_cast<Manager*>(base_ptr);
```

**Suggested Reading:-**
https://en.cppreference.com/w/cpp/language/static_cast
https://en.wikipedia.org/wiki/Static_cast
http://www.learncpp.com/cpp-tutorial/4-4a-explicit-type-conversion-casting/
http://www.learncpp.com/cpp-tutorial/44-implicit-type-conversion-coercion/

# Concurrency & Chrono Library

- Chrono library provides three types of elements of broadly, which are
  - **std::chrono::duration** -- to specify the time interval at desired precision
  - Clocks - three clock types are defined
    - **std::chrono::system_clock** - system wide real time clock
    - **std::chrono::steady_clock** - monotonic clock
    - **std::chrono::high_resolution_clock** - Granular precision with shortest tick period possible on the machine
  - **std::chrono::time_point** - time elapsed from clock specific epoch (as *std::chrono::duration*)

Apart from chrono library,few C-Style date & time library utilities are also available.

- std::time_t type
- std::time
- std::ctime
- std::localtime
- std::asctime
  std::difftime

## std::chrono::duration

Bookmark this page

Basic usage:-

```
std::chrono::duration<int,std::ratio<3600>> nh(2);      //2 hours
std::chrono::duration<int,std::ratio<60>> nmins(15);    //15 mins
std::chrono::duration<int,std::ratio<1>> ns(40);        //40 seconds
//std::chrono::duration<int> ns(40);                    //40 seconds
std::chrono::duration<int,std::milli> nmilli(500);      //500 milli seconds
//std::chrono::duration<int,std::ratio<1,1000>> nmilli(500);
std::chrono::duration<int,std::micro> nmicro(25);
std::chrono::duration<int,std::nano> nanos(525);
```

The following helper types provide convenient types for various precision levels

- std::chrono::hours ==> std::chrono::duration<T,std::ratio<3600>
- std::chrono::minutes ==> std::chrono::duration<T,std::ratio<60>

- std::chrono::seconds ==> std::chrono::duration<I,std::ratio<1> (or) std::chrono::duration<I>
- std::chrono::milliseconds ==> std::chrono::duration<T,std::milli> (or) std::chrono::duration<T, std::ratio<1,1000>>
- std::chrono::microseconds ==> std::chrono::duration<T,std::micro>, std::micro == std::ratio<1,1000000>
- std::chrono::nanoseconds ==> std::chrono::duration<T,std::nano>, std::nano == std::ratio<1,1000000000>

Basic usage of helper types,specifying intervals, implicit conversions, arithmetic

```
std::chrono::seconds ns(5);
std::cout << ns.count() << "\n";
std::cout << std::chrono::milliseconds(ns).count() << "\n";

std::chrono::hours nhours(2);
std::chrono::minutes nmins(25);
auto total = nhours + nmins;
std::cout << std::chrono::minutes(total).count() << "\n";
```

std::duration_cast can convert the intervals from one precision to other.

```
std::chrono::milliseconds tms(500);
auto tmicro=std::chrono::microseconds(tms);     //ok
auto tmicro=std::chrono::seconds(tms);          //error
```

```
40 int main()
41 {
42 std::chrono::seconds ns(5);
43 std::cout << ns.count() << "\n";
44 std::cout << std::chrono::milliseconds(ns).count() << "\n";
45
46 std::chrono::hours nhours(2);
47 std::chrono::minutes nmins(25);
48 auto total = nhours + nmins;
49 std::cout << std::chrono::minutes(total).count() << "\n";
50
51 return 0;
52 }
53
```

input stdout

Compiled Successfully. memory: 2828 time: 0 exit code: 0

```
5
5000
145
```

```
auto tmicro=std::chrono::seconds(tms);          //error
auto tmicro=std::chrono::duration_cast<std::chrono::seconds>(tms);     //ok

std::chrono::duration<double,std::milli> tval(5000);
auto nmicro=std::chrono::duration<double,std::micro>(tval);
auto nsecs=std::chrono::duration<double>(tval);

auto nsecs=std::chrono::seconds(tval);    //error
auto nsecs=std::chrono::duration_cast<std::chrono::seconds>(tval);    //ok
```

From C++14 literals can have suffix ,added through user defined literals

Suffix :- h for hours, min for minutes, s for seconds, ms for milli, us for micro, ns for nanoseconds

These are implemented through operator""h, operator""min, operator""s, operator""ms,operator""us,operator""ns

```
using namespace std::chrono_literals;
std::chrono::hours nhours=2h;  //auto nhours=2h;
std::chrono::hours nmins=25min;  //auto nmins=25min;
std::chrono::duration<double> period(2h+25min);  //auto period=2h+20min;
std::cout << period.count() << "\n";
```

main.cpp

```
int main()
{
std::chrono::milliseconds tms(500);
std::cout<<"tms:"<<tms.count()<<std::endl;
auto tmicro=std::chrono::microseconds(tms);    //ok
std::cout<<"tmicro:"<<tmicro.count()<<std::endl;
//auto tmicro1=std::chrono::seconds(tms);         //compilation error
auto tmicro2=std::chrono::duration_cast<std::chrono::seconds>(tms);    //ok
std::cout<<"tmicro2:"<<tmicro2.count()<<std::endl;

std::chrono::duration<double,std::milli> tval(5000);
std::cout<<"tval:"<<tval.count()<<std::endl;
auto nmicro=std::chrono::duration<double,std::micro>(tval);
std::cout<<"nmicro:"<<nmicro.count()<<std::endl;
auto nsecs1=std::chrono::duration<double>(tval);
std::cout<<"nsecs1:"<<nsecs1.count()<<std::endl;

//auto nsecs2=std::chrono::seconds(tval);   //compilation error
auto nsecs3=std::chrono::duration_cast<std::chrono::seconds>(tval);   //ok
std::cout<<"nsecs3:"<<nsecs3.count()<<std::endl;
```

Compiled Successfully.

```
tms:500
tmicro:500000
tmicro2:0
tval:5000
nmicro:5e+06
nsecs1:5
nsecs3:5
```

From C++14 literals can have suffix ,added through user defined literals

Suffix :- h for hours, min for minutes, s for seconds, ms for milli, us for micro, ns for nanoseconds

These are implemented through operator""h, operator""min, operator""s, operator""ms,operator""us,operator""ns

```
using namespace std::chrono_literals;
std::chrono::hours nhours=2h;  //auto nhours=2h;
std::chrono::hours nmins=25min;  //auto nmins=25min;
std::chrono::duration<double> period(2h+25min);  //auto period=2h+20min;
std::cout << period.count() << "\n";
```

```
40 int main()
41 {
42 using namespace std::chrono_literals;
43 std::chrono::hours nhours=2h;  //auto nhours=2h;
44 std::cout<<"nhours:"<<nhours.count()<<std::endl;
45 //std::chrono::hours nmins=25min;  //auto nmins=25min;
46 //std::cout<<"nmins:"<<nmins.count()<<std::endl;
47 std::chrono::duration<double> period(2h+25min);  //auto period=2h+20min;
48 std::cout << period.count() << "\n";return 0;
49 }
50
```

input | stdout

Compiled Successfully. memory: 2880 time: 0 exit code: 0

```
nhours:2
8700
```

**System clock (std::chrono::system_clock):-**

- It is associated with usual real time clock of the current system, this measures time elapsed from epoch of the system.
- This clock may be affected by any manual changes by system administrators or NTP synchronization.
- This clock is typically compatible with time_t type followed by C-Style time APIs
- Typically UNIX Epoch is the epoch for C++ system clock (not necessarily all the times)

Here is a small example on measuring time points using system clock

```
std::vector<int> v1(1000);
std::srand(std::time(nullptr));
std::generate(v1.begin(),v1.end(), [](){ return std::rand()%100;});
```

```
int sum=0;
auto tstart=std::chrono::system_clock::now();
//std::chrono:time_point<std::chrono::system_clock> tstart;
for(auto x:v1)
  sum+=x;
auto tend=std::chrono::system_clock::now();
std::chrono::duration<double,std::micro> elapsed=tend-tstart;
```

```
//auto elapsed=tend-tstart;
std::cout << elapsed.count() << "\n";
std::cout << std::duration<double,std::milli>(elapsed).count() << "\n";
```

Compatibility with c-style APIs:-

```
auto tnow=std::chrono::system_clock::now();
std::time_t t1=std::chrono::system_clock::to_time_t(&tnow);
constc char* tstr=std::ctime(&t1);

time_t t2=std::time(nullptr);
std::chrono::system_clock::time_point tp=std::from_time_t(t2);
//auto tp=std::from_time_t(t2)
```

---

**std::chrono::steady_clock:-**

This is associated with monotonic clock of the system, typically w.r.t machine startup

This clock is not affected by any changes to system time (manual or ntp sync)l0huo

It's most suitable for measuring intervals between time points

Example:-

```
//std::chrono::timepoint<std::chrono::steady_clock> tstart;
auto tstart=std::chrono::steady_clock::now();
//some computations
auto tend=std::chrono::steady_clock::now();
```

**Further reading on Real Time clock vs Monotonic clock:-**

- clock_gettime man page

---

High Resolution Clock(std::chrono::high_resolution_clock):-

This is associated with most accurate clock supported by the system, which can measure smallest tick period supported by the system

Example:-

```
auto tstart=std::chrono::high_resolution_clock::now();
//some computations
auto tend=std::chrono::high_resolution_clock::now();
```

**Tutorial:-**
https://thispointer.com/how-to-put-a-thread-to-sleep-in-c11-sleep_for-sleep_until/
https://www.geeksforgeeks.org/chrono-in-c/
**From Books:-**
The C++ Standard Library : A Tutorial and Refernce, 2/e ==> 5.7) Clocks and Timers

**std::chrono:-**
https://en.cppreference.com/w/cpp/chrono
http://www.cplusplus.com/reference/chrono/
**Duration:-**

https://en.cppreference.com/w/cpp/chrono/duration
http://www.cplusplus.com/reference/chrono/duration/

**std::ratio:-**

https://en.cppreference.com/w/cpp/numeric/ratio/ratio
https://en.cppreference.com/w/cpp/header/ratio
http://www.cplusplus.com/reference/ratio/ratio/

**Clocks:-**

https://en.cppreference.com/w/cpp/chrono/system_clock
http://www.cplusplus.com/reference/chrono/system_clock/
https://en.cppreference.com/w/cpp/chrono/steady_clock
http://www.cplusplus.com/reference/chrono/steady_clock/
https://en.cppreference.com/w/cpp/chrono/high_resolution_clock
http://www.cplusplus.com/reference/chrono/high_resolution_clock/

**Time Point:-**

https://en.cppreference.com/w/cpp/chrono/time_point
http://www.cplusplus.com/reference/chrono/time_point/

## Simple Thred Creation

Bookmark this page

C++11 allows multi threading  using std::thread class, which are built on top of pthread APIs in *nix.

Simple usage of std::thread which starts on a entry function which doesn't take any arguments

```
void doSomething() {
   //some computations
}
std::thread t1(doSomething);
t1.join();
```

join member function makes current thread to block for the completion of target thread.

In absence of join main thread won't wait for the completion of running thread(t1), as a resultant all running threads terminate abnormally when main thread hits return statement.(Recall **exit vs pthread_exit** at low level system programming)

Here is an example on using normal functions(function pointers) as callbacks(entry code):-

```
#include<thread>
using namespace std::chrono_literals;

constexpr int max=10;

void do_service1() {
  std::cout << "A--welcome\n";
  for(int i=1;i<=max;i++) {
    std::cout << "A--" << i << "\n";
    std::this_thread::sleep_for(1s);
  }
}

void do_service2() {
  std::cout << "B--welcome\n";
  for(int i=1;i<=max;i++) {
    std::cout << "B--" << i << "\n";
    std::this_thread::sleep_for(1s);
  }
}
```

```
int main() {
  std::thread t1(do_service1);
  std::thread t2(do_service2);
  t1.join();
  t2.join();
  std::cout << "main--thank you\n";
  return 0;
}
```

**Code Analysis:-**

- Identify  the significance of join?
- List all running threads using following command

```
ps -eL -o pid,ppid,lwp,nlwp,cmd
```

- Do above code conforms to task driven or data driven parallelism?

---

Thread entry functions can take arguments

Passing arguments to entry function is very elegant in C++, as no void pointer & conversions are required unlike pthread_create

This allows same function as callback (entry code) for multiple threads:-

```
void do_service(std::string label) {
  std::cout << "Started thread with id : " << std::this_thread::get_id() << "\n";
  std::cout << label << "--welcome\n";
  for(int i=1;i<=20;i++) {
    std::cout << label << "--" << i << "\n";
    usleep(100);
  }
}

int main() {
  std::thread t1(do_service,"A1");
  std::thread t2(do_service,"B2");
  t1.join();
  t2.join();
  return 0;
}
```

Retrieving thread id using get_id:-

```
std::thread::id tid1=t1.get_id();
```

**Note:-** get_id  output is not same as gettid system call

std::this_thread namespace provides following members applicable on current thread:-

- std::this_thread::sleep_for
- std::this_thread::sleep_until
- std::this_thread::get_id
- std::this_thread::yield

sleep_for can block the thread for a particular duration

```
std::this_thread::sleep_for(interval);
//interval can be any instance or literals supported by std::chrono::duration
for e.g.
std::this_thread::sleep_for(std::chrono::seconds(5));
std::this_thread::sleep_for(5s);
std::this_thread::sleep_for(std::chrono::duration<int,std::milli>(5000));
```

sleep_until blocks the thread till a future timepoint, this is helpful for absolute delays where as sleep_for is helpful for relative delays

```
std::chrono::time_point<std::chrono::system_clock> tbase;
tbase=std::chrono::system_clock::now();
std::chrono::duration<int,std::milli> period=2000ms;

std::chrono::time_point<std::chrono::system_clock> tnext=tbase+period;
while(true) {
  std::this_thread::sleep_until(tnext);
  tnext+=period;
}
```

**std::thread::get_id** returns id associated with current thread

**std::this_thread::yield** yields current thread, i.e. current thread forces scheduling by leaving running state and comes to ready state

**Using lambdas as callbacks:-**

```
int main() {
  int max=10;
  std::thread t1( [=] () {
    std::cout << "Welcome\n";
    for(int i=1;i<=max;i++)
      std::cout << "Hello--%d\n",i);
  });
  std::thread t2( []() {
    do_service("C3");
  });
  auto do_compute = [](std::string label) {
    do_service(label);
  };
  std::thread t2(do_compute,"D4");
  t1.join();
  t2.join();
  t3.join();
}
```

**Using function objects as callbacks:-**

```
class DoService {
  std::string m_label;
  public:
  DoService(std::string label):m_label(label) { }
  void operator()() {
    std::cout << m_label << "--welcome\n";
    for(int i=1;i<=20;i++) {
      std::cout << m_label << "--" << i << "\n";
      std::this_thread::sleep_for(std::chrono::duration<int,std::micro>(100));
    }
  }
};
int main() {
 std::thread t1(DoService("A1"));
 DoService s2("B2");
 std::thread t2(s2);
 t1.join();
 t2.join();
}
```

```
class DoService {
  public:
  void operator()(std::string label) {
    std::cout << label << "--welcome\n";
    for(int i=1;i<=20;i++) {
      std::cout << label << "--" << i << "\n";
      std::this_thread::sleep_for(std::chrono::duration<int,std::micro>(100));
    }
  }
};
int main() {
  std::thread t1(DoService(),"A1");
  DoService s2;
  std::thread t2(s2,"B2");
  t1.join();
  t2.join();
}
```

## Parallel sum of large array

Bookmark this page

```
#include<thread>
#include<iostream>
#include<cstdlib>

constexpr int maxsize=1000;
int arr[maxsize];
int parallelSum=0;

void do_sum(int start,int end) {
  int tsum=0;
  for(int i=start;i<end;i++)
    tsum+=arr[i];
  parallelSum+=tsum;                //???
}
```

```
int main() {
  std::srand(std::time(nullptr));
  for(int i=0;i<maxsize;i++)
    arr[i]=std::rand()%100;
  int seqSum=0;
  for(int i=0;i<maxsize;i++)
    seqSum+=arr[i];
  std::cout << "sequential sum:" << seqSum << "\n";
  /*********************************************/
  std::thread t1(do_sum,0,200);        //0-199
  std::thread t2(do_sum,200,400);    //200-399
  std::thread t3(do_sum,400,600);    //400-599
  std::thread t4(do_sum,600,800);    //600-799
  std::thread t5(do_sum,800,1000);    //800-999

  t1.join();
  t2.join();
  t3.join();
  t4.join();
  t5.join();
  std::cout << "Final sum is :" << parallelSum << "\n";
  return 0;
}
```

Another Approach:-

```
int n=10;
std::thread *tarr[n];
range=maxsize/n;
for(int i=0;i<n;i++) {
  tarr[i]=new std::thread(do_sum,start,start+range);
  start+=range;
}
for(int i=0;i<n;i++) {
  tarrr[i]->join();
  delete tarr[i];
}
```

## Member Functions as callbacks

Bookmark this page

std::thread instances can even take member functions as callbacks by binding calling object address.

If psum is member function of MyData class, we can create thread as follows

```
void MyDataSet::psum(int start,int end);

MyDataSet *ds=new MyDataSet(100);
std::thread t2(&MyDataSet::psum,ds,200,400);
//(or)
using namespace std::placeholders;
auto tentry=std::bind(&MyDataSet::psum,ds,_1,_2);
std::thread t1(tentry,0,200);
```

Let's rewrite previous example using member functions as callback

```
#include<iostream>
class MyDataSet {
  int *arr;
  int maxlen;
  int total=0;
  public:
  MyDataSet(int len):maxlen(len) {
    arr=new int[maxlen];
  }
  MyDataSet(const MyDataSet&)=delete;
  void fill() {
    for(int i=0;i<maxlen;i++)
      arr[i]=std::rand()%100;
  }
  void psum(int start,int end) {
    int tsum=0;
    for(int i=start;i<end;i++)
      tsum+=arr[i];
    total+=tsum;
  }
  void printTotal() {
    std::cout << "Final sum is:" << total << "\n";
  }
```

```
  void seqSum() {
    int seqSum=0;
    for(int i=0;i<maxlen;i++)
      seqSum+=arr[i];
    std::cout << "sequential sum:" << seqSum << "\n";
  }
};
int main() {
  MyDataSet ds(1000);
  ds.fill();
  std::thread t1(&MyDataSet::psum,&ds,0,200);
  std::thread t2(&MyDataSet::psum,&ds,200,400);
  std::thread t3(&MyDataSet::psum,&ds,400,600);
  std::thread t4(&MyDataSet::psum,&ds,600,800);
  std::thread t5(&MyDataSet::psum,&ds,800,1000);
  t1.join();
  t2.join();
  t3.join();
  t4.join();
  t5.join();
  ds.printTotal();
  ds.seqSum();
  return 0;
}
```

## Parallel Sum - Container

Bookmark this page

Re writing above code using containers & iterators:-

```
constexpr int maxsize=1000;
std::array<int,maxsize> arr;
int total(0);

//template<typename T>
using myiter_t=std::array<int,maxsize>::iterator;
void psum(myiter_t pstart, myiter_t pend) {
  int tsum=0;
  for(auto iter=pstart;iter!=pend;++iter)
    tsum += *iter;
  total+=tsum;
}
```

```
int main() {
  int lsum=0,i;
  //for(auto iter=arr.begin();iter!=arr.end();++iter)
  //  *iter=std::rand()%100;
  for(auto& x:arr)
    x=std::rand()%100;
  for(auto x:arr)
    lsum+=x;
  std::cout << "Linear sum is " << lsum << "\n";
  int n=10;
  std::thread *tarr[n];
  int range=maxsize/n;
  std::array<int,maxsize>::iterator tstart=arr.begin();
  for(i=0;i<n;i++) {
    tarr[i]=new std::thread(psum,tstart,tstart+range);
    tstart+=range;
  }
  for(i=0;i<n;i++) {
    tarr[i]->join();
    delete tarr[i];
  }
  std::cout << "Final sum is " << total << "\n";
  return 0;
}
```

## Detached threads

Bookmark this page

Sometimes it is not possible to join the main thread (by which new threads get created ) till the completion of target thread, for e.g. in a concurrent server example main thread can never reach join calls as it's waiting for the acceptance of more client connects.

Pseudo-code for concurrent server:-

```
void service_the_client(int client_id) {
}

//start the server (socket, bind, listen as per BSD socket API)
while(true) {
  //accept the connection from a new client (blocking call)
  //create a new thread to service the client
}
//join code   //never reachable
```

In above scenario, server can never reach the join calls.So these kind of threads can be put in detached state to allow independent execution, i.e. these are exempted from join calls

A thread can execute independently in detachable state.

To detach a thread

```
void doSomething() {
   //heavy/busy code or infinite loop
}
std::thread t1(doSomething);
t1.detach();
```

joinable member of thread class checks if given thread is joinable (not detached) or not

```
std::thread t1(doSomething);
std::thread t2(doSomething);
t1.deatch();
bool state1=t1.joinable();    //true or false
bool state2=t2.joinable();    //true or false
```

**Concept Check - System Programming:-**

- Who cleans up resources for detached threads?
- If any one thread calls exit (may be via return) do detached threads are allowed to run?

## Detached threads

Bookmark this page

Sometimes it is not possible to join the main thread (by which new threads get created ) till the completion of target thread, for e.g. in a concurrent server example main thread can never reach join calls as it's waiting for the acceptance of more client connects.

Pseudo-code for concurrent server:-

```
void service_the_client(int client_id) {
}

//start the server (socket, bind, listen as per BSD socket API)
while(true) {
  //accept the connection from a new client (blocking call)
  //create a new thread to service the client
}
//join code   //never reachable
```

In above scenario, server can never reach the join calls.So these kind of threads can be put in detached state to allow independent execution, i.e. these are exempted from join calls

A thread can execute independently in detachable state.

To detach a thread

```
void doSomething() {
   //heavy/busy code or infinite loop
}
std::thread t1(doSomething);
t1.detach();
```

joinable member of thread class checks if given thread is joinable (not detached) or not

```
std::thread t1(doSomething);
std::thread t2(doSomething);
t1.deatch();
bool state1=t1.joinable();    //true or false
bool state2=t2.joinable();    //true or false
```

**Concept Check - System Programming:-**

- Who cleans up resources for detached threads?
- If any one thread calls exit (may be via return) do detached threads are allowed to run?

### Further Reading from Effective Modern C++ by Scott Meyers

### Item 37: Make std::threads unjoinable on all paths

### Item 38: Be aware of varying thread handle destructor behavior

---

### Tutorials:-

https://www.geeksforgeeks.org/multithreading-in-cpp/

from *thispointer.com:-*

- Part 1: Three Ways to Create Threads
- Part 2: Joining and Detaching Threads
- Part 3: Passing Arguments to Threads

---

- C++11 : Start thread by member function
- C++11 : How to put a thread to sleep
- C++11 : How to get a Thread ID ?
- C++11: Vector of Thread Objects
- C++11 : std::thread as a member variable in class
- C++11 : How to Stop a Thread
    - **std::thread:-**
    - https://en.cppreference.com/w/cpp/thread/thread
    - http://www.cplusplus.com/reference/thread/thread/

- **members of std::this_thread namespace :-**
- yield:- https://en.cppreference.com/w/cpp/thread/yield
- get_id:- https://en.cppreference.com/w/cpp/thread/get_id
- sleep_for:- https://en.cppreference.com/w/cpp/thread/sleep_for
- sleep_until:- https://en.cppreference.com/w/cpp/thread/sleep_until

## std::async & std::future

Bookmark this page

- Async task is a high level implementations for threads for simplified execution of concurrent tasks
- They allow to run a piece of code (callable elements) in background as a separate thread and fetch the return value.
- The thread can be picked up from a thread pool or by creating a new one
- Futures wait for the completion of Async and collect the results from their execution
- This is beneficial over threads to fetch results from computations, other wise we need to manage with some shred data like global variables in case of low level threads.

**Simple Example:-**

```
#include<future>
int doSumofArray(int *arr,int max) {
  int tsum=0;
  for(int i=0;i<max;i++)
    tsum+=arr[i];
  return tsum;
}
```

```
int main() {
  int arr[100];
  for(int i=0;i<100;i++)
    arr[i]=rand()%100;
  std::future<int> sum;
  sum=std::async(doSumofArray,arr,100);
  //do some work while async task is running
  std::cout << sum.get() << "\n";
  return 0;
}
```

std::async can have one of the following launch policies

- `std::launch::async` ==> execute the code asynchronously

  (whenever scheduler permits irrespective of result is requested or not)
  `std::launch::deferred` ==> execute only on result request

```
template <class Fn, class... Args>
future<typename result_of<Fn(Args...)>::type> async (launch policy, Fn&& fn, Args&&... args);
```

**std::async** returns a **std::future<T>,** that stores the value returned by function object executed by **std::async()**. Arguments expected by function can be passed to std::async() as arguments after the function pointer argument.

First argument in std::async is launch policy, it control the asynchronous behaviour of std::async. We can create std::async with 3 different launch policies i.e.

- **std::launch::async**
  - It guarantees the asynchronous behaviour i.e. passed function will be executed in seperate thread.
- **std::launch::deferred**
  - Non asynchronous behaviour i.e. Function will be called when other thread will call get() on future to access the shared state.
- **std::launch::async | std::launch::deferred**
  - Its the default behaviour. With this launch policy it can run asynchronously or not depending on the load on system. But we have no control over it.

If we do not specify an launch policy. Its behaviour will be similar to **std::launch::async | std::launch::deferred**.

Let's rewrite parallel sum code using async and futures

```
constexpr int maxsize=1000;
int arr[maxsize];

int do_async_sum(int start,int end) {
  int tsum=0;
  for(int i=start;i<end;i++)
    tsum+=arr[i];
  return tsum;
}
```

```
int main() {
  std::srand(std::time(nullptr));
  for(int i=0;i<maxsize;i++)
    arr[i]=std::rand()%100;
  /*********************************************/
  std::future<int> f1;
  std::future<int> f2;
  std::future<int> f3;
  std::future<int> f4;
  std::future<int> f5;
```

```
  f1=std::async(do_async_sum,0,200);      //0-199
  f2=std::async(do_async_sum,200,400);    //200-399
  f3=std::async(do_async_sum,400,600);    //400-599
  f4=std::async(do_async_sum,600,800);    //600-799
  f5=std::async(do_async_sum,800,1000);   //800-999

  int total=f1.get()+f2.get()+f3.get()+f4.get()+f5.get();
  std::cout << "Final sum is :" << total << "\n";
  return 0;
}
```

**Suggested Reading from Effective Modern C++, Scott Meyers**
**Item 35: Prefer task-based programming to thread-based**
**Item 36: Specify std::launch::async if asynchronicity is essential**
**Item 39: Consider void futures for one-shot event communication**
**Tutorial:-**
https://thispointer.com/c11-multithreading-part-9-stdasync-tutorial-example/
https://thispointer.com/c11-multithreading-part-10-packaged_task-example-and-tutorial/

**std::async:-**
https://en.cppreference.com/w/cpp/thread/async
http://www.cplusplus.com/reference/future/async/
**std::future:-**
https://en.cppreference.com/w/cpp/thread/future
http://www.cplusplus.com/reference/future/future/

Recursive sum example:-

```
int recursive_sum(int lb,int ub) {
  std::cout << lb << "," << ub << "\n";
  if(lb>ub) return 0;
  if(lb==ub) return arr[lb];
  std::future<int> first;
  std::future<int> second;
  int mid=(lb+ub)/2;
  first=std::async(recursive_sum,lb,mid);
  second=std::async(recursive_sum,mid+1,ub);
  return first.get()+second.get();
}
```

```
std::future<int> sum;
sum=std::async(recursive_sum,0,n);
int total=sum.get();
```

**Concept Checking:-**

- Compare  system_clock, steady_clock, high_resolution_clock
- Simplicity & Advantages of async over threads

---

**Hands-on:-**

- Compare the complexity of following operations using suitable clock APIs on a vector holding objects of non trivial types
  - push_back(T&), push_back(T&&), emplace_back
- Compare the lookup complexity of **std::map** and **std::unordered_map** for a quite large no.of elements
- Identify some conversions on std::chrono::duration which are only possible by duration_cast, i.e. those conversions which are not implicit
- Compare the difference between time points with sleep_for, sleep_until
- Find the min, max values in a large container with parallel computations using
  - thread
  - async and futures

# call_once and once_flag

call_onces ensures that some code is executed only once, though it's invoked by multiple threads
https://en.cppreference.com/w/cpp/thread/call_once
https://en.cppreference.com/w/cpp/thread/once_flag

## Argument Passing - Dos and Don'ts

Bookmark this page

Ensure that arguments passed to thread callbacks(entry functions) are stored on stack frame of the callback only.

If the callback parameters are pointing out of its stack frame the results are unpredictable.

So avoid callback parameters pointing to

- Stack frame of other functions like main thread
- Heap

For example the behavior of following code is not predictable, as *ptr may lead to side effects. i.e. when one thread trying to access *ptr, id could be incremented by other threads

```
void do_service1(int *ptr) {
  std::cout << *ptr << "--welcome\n";
  for(int i=1;i<=max;i++) {
    std::cout << *ptr << "--" << i << "\n";
    std::this_thread::sleep_for(1s);
  }
```

```
int main() {
  int n=10;
  std::thread *tarr[n];
  int id=101;
  for(int i=0;i<10;i++) {
    tarr[i]=new std::thread(do_service,&id);
    id++;
  }

  for(int i=0;i<10;i++)
    tarr[i]->join();
  std::cout << "main--thank you\n";
  return 0;
}
```

**Tutorial:-**
https://thispointer.com/c11-multithreading-part-10-packaged_task-example-and-tutorial/
**References:-**
https://en.cppreference.com/w/cpp/thread/packaged_task
http://www.cplusplus.com/reference/future/packaged_task/

## Race Conditions - Example

Bookmark this page

Let's consider an example to demonstrate race conditions, in this example final value of global(shared) variable is unpredictable due to concurrent increment/decrement operations.

```
int val=100;
constexpr int max=100000;

void do_inc() {
  for(int i=0;i<max;i++) {
    val++;
  }
}
void do_dec() {
  for(int i=0;i<max;i++) {
    val--;
  }
}
```

```
int main() {
  std::thread t1(do_inc);
  std::thread t2(do_dec);
  t1.join();
  t2.join();
  std::cout << "Final val:" << val << "\n";
}
```

**Recall some IPC Concepts:-**

- Race conditions - Concurrent access to shared resources by two or more threads lead to inconsistency, in other words the final state of the resources will be undefined.
- Critical Section - piece of code accessing shared resource
- Mutual Exclusion - allowing shared resource access to only one thread at a time, in other words only one thread can be in their critical section at a time.
- OS/Library support for Mutual Exclusion
  - Semaphores
  - Mutex
  - Spinlocks

C++11 provides std::mutex class for mutual exclusion, let's consider an example to prevent race conditions in previous example.

```
int val=100;
constexpr int max=100000;
std::mutex m1;

void do_inc() {
  for(int i=0;i<max;i++) {
    m1.lock();
    val++;
    m1.unlock();
  }
}
void do_dec() {
  for(int i=0;i<max;i++) {
    m1.lock();
    val--;
    m1.unlock();
  }
}
```

```
int main() {
  std::thread t1(do_inc);
  std::thread t2(do_dec);
  t1.join();
  t2.join();
  std::cout << "Final val:" << val << "\n";
}
```

---

Working Principle of Mutex:-

- Mutex is a system/library level data structure which ensures non concurrent access to shared resources, i.e. allowing only one thread to execute their related critical sections at a time.
- Mutex operations - lock & unlock
- Lock rules
  - If the mutex is in **Unlocked** state, lock it and allow thread execution,i.e. this thread is first person trying to access shared resource
  - If the mutex is in **Locked** state, block the current thread and add to mutex queue,i.e. already another thread is in middle of shared resource access,thus blocking current thread.
- Unlock rules
  - If the mutex queue is not empty, remove one thread from queue and resume (unblocking from blocked state )
  - If the queue is empty, change mutex state back to **Unlocked**
- Salient features of Mutex and differences from semaphore operations
  - Mutex comes with ownership, i.e. whichever thread locks the mutex can only unlock
  - Unlocking before locking is not allowed with mutex
  - Unlocking more than once us not allowed with mutex (non recursive mutex)
- Thus we can say that  mutex operations are purely meant for mutual exclusion, not for synchronization,

Recall the example on threading which demonstrates concurrent execution of respective for loops.In this example such concurrent execution is prevented through mutex

```
#include<thread>
#include<mutex>
#include<iostream>
#include<chrono>

std::mutex m1;
std::chrono::duration<int,std::micro> interval(100));

void do_service1() {
  std::cout << "A--welcome\n";
  m1.lock();
  for(int i=1;i<=10;i++) {
    std::cout << "A--" << i << "\n";
    std::this_thread::sleep_for(interval)
  }
  m1.unlock();
}
void do_service2() {
  std::cout << "B--welcome\n";
  m1.lock();
```

```
  for(int i=1;i<=10;i++) {
    std::cout << "B--" << i << "\n";
    std::this_thread::sleep_for(interval)
  }
  m1.unlock();
}

int main() {
  std::thread t1(do_service1);
  std::thread t2(do_service2);
  t1.join();
  t2.join();
  return 0;
}
```

**Code Analysis:-**

- In this example you can observe that only one thread can enter the for loop (Let's consider this as dummy critical section,stdout as shared resource) at a time
- Second thread can enter for loop only when the first thread release the mutex after completion of for loop.

Need for lock objects:-

- What'll happen if you omit the unlock statement by mistake or intentionally in on of the threads?
- All other threads waiting for this mutex will undergo starvation.
- This problem can be solved by std::unique_lock/std::lock_guard instance as shown in following example
- **std::unique_lock/std::lock_guard** ensures that underlying mutex will be unlocked when the lock instance goes out of the scope.

```
void do_service1() {
  std::cout << "A--welcome\n";
  std::unique_lock<std::mutex> ulck(m1);
  //std::lock_guard<std::mutex> glck(m1);
  for(int i=1;i<=10;i++) {
    std::cout << "A--" << i << "\n";
    sleep(1);
  }
  //m1 will be unlocked automatically when ulck/glck goes out of scope
}
```

```
void do_service2() {
  std::cout << "B--welcome\n";
  std::unique_lock<std::mutex> ulck(m1);
  //std::lock_guard<std::mutex> glck(m1);
  for(int i=1;i<=10;i++) {
    std::cout << "B--" << i << "\n";
    sleep(1);
  }
  //m1 will be unlocked automatically when ulck/glck goes out of scope
}
```

**Note:-**
In this example, either of std::unique_lock, std::lock_guard can be used with similar functionality.We'll see the additional features of std::unique_lock during further discussion.

## Parallel sum of large array - concurrency issues

Bookmark this page

- Recall the parallel sum example discussed during thread topic
- Identify potential race conditions, Hint:- adding local sum to final sum at end of each thread
- Prevent the race conditions using
  - Mutex operations - surrounding this code with lock and unlock
  - Atomic variables - defining total as atomic type

**Tutorial:-**
https://thispointer.com//c11-multithreading-part-4-data-sharing-and-race-conditions/

https://thispointer.com//c11-multithreading-part-5-using-mutex-to-fix-race-conditions/

**References:-**

https://en.cppreference.com/w/cpp/thread/mutex
https://en.cppreference.com/w/cpp/thread/mutex/lock
https://www.cplusplus.com/reference/mutex/mutex/
http://www.cplusplus.com/reference/mutex/mutex/lock/

## Atomic Operations

Bookmark this page

Operations on atomic variables ensure that no race condition by intrinsic mutual exclusion.

```
#include<iostream>
#include<atomic>

std::atomic<int> val(100);
constexpr int max=1000000;

void do_inc() {
  for(int i=0;i<max;i++) {
    val++;                      //atomic operation
  }
}
void do_dec() {
  for(int i=0;i<max;i++) {
    val--;                      //atomic operation
  }
}
```

**Suggested Reading from Effective Modern C++ by Scott Meyers**

**Item 40: Use std::atomic for concurrency, volatile for special memory,**

**References:-**

https://en.cppreference.com/w/cpp/atomic/atomic
http://www.cplusplus.com/reference/atomic/

## Deadlock Scenario

Bookmark this page

Let's consider a scenario,where we need two mutex operations to protect two shared resources.Arbitrary mutex operations can lead to dead lock as shown in following example.i.e. if switching occurs in middle of m1.lock, m2.lock bot the threads will be infinitely blocked waiting for other mutex by locking a mutex each

```
std::mutex m1;
std::mutex m2;

void do_service1() {
  std::cout << "A--welcome\n";
  m1.lock();     //let's assume this thread executes initially and switch a here
  m2.lock();
  for(int i=1;i<=10;i++) {
    std::cout << "A--" << i << "\n";
    sleep(1);
  }
  m2.unlock();
  m1.unlock();
}
```

```
void do_service2() {
  std::cout << "B--welcome\n";
  m2.lock();    //arbitrary sequence of locking mutexes
  m1.lock();    //may lead to dead lock
  for(int i=1;i<=10;i++) {
    std::cout << "B--" << i << "\n";
    sleep(1);
  }
  m1.lock();
  m2.lock();}
```

## Deadlock avoidance

Bookmark this page

The deadlock problem in previous example can be resolved with safe locking operations as shown in following example, these locking techniques ensures that both the  mutexes will be locked or none.If one mutex is locked but other is not available, locked mutex will be released and trying to lock both again.

Since the mutex shouldn't be locked independently(one at a time), deferred lock option is specified so that constructor of unique_lock won't lock mutex immediately and deferred for safe locking together.

```
std::mutex m1;
std::mutex m2;

void do_service1() {
  std::cout << "A--welcome\n";
  std::unique_lock<std::mutex> ulck1(m1,std::defer_lock);
  std::unique_lock<std::mutex> ulck2(m2,std::defer_lock);
  std::lock(ulck1,ulck2);   //safe locking with deadlock avoidance algorithms
  for(int i=1;i<=10;i++) {
    std::cout << "A--" << i << "\n";
    sleep(1);
  }
}
```

```
void do_service2() {
  std::cout << "B--welcome\n";
  std::unique_lock<std::mutex> ulck2(m2,std::defer_lock);
  std::unique_lock<std::mutex> ulck1(m1,std::defer_lock);
  std::lock(ulck1,ulck2);
  for(int i=1;i<=10;i++) {
    std::cout << "B--" << i << "\n";
    sleep(1);
  }
}
```

## std::unique_lock vs std::lock_guard

Bookmark this page

- std::unique_lock objects must be used in conjunction with std::condition_variable
- std::lock_guard locks the underlying mutex on construction and unlocks on destruction.Since locking can't be deferred safe locking of multiple mutexes is not possible
- std::unique_lock can defer the locking for safe locking of multiple mutexes
- So lock_guard can be used in following context
  - For simple/independent locking operations
  - Synchronization not required,i.e. there is no need for condition variables
- In rest all the cases unique_lock is preferred.

**Suggested Reading from Effective Modern C++, Scott Meyers**
**Item 16: Make const member functions thread safe.**,
std::timed_mutex:-

https://en.cppreference.com/w/cpp/thread/timed_mutex
std::recursive_mutex:-
https://en.cppreference.com/w/cpp/thread/recursive_mutex
std::recursive_timed_mutex:-
https://en.cppreference.com/w/cpp/thread/recursive_timed_mutex
std::unique_lock:-
https://en.cppreference.com/w/cpp/thread/unique_lock
std::lock_guard:-
https://en.cppreference.com/w/cpp/thread/lock_guard
std::shared_lock:-
https://en.cppreference.com/w/cpp/thread/shared_lock
std::shared_timed_mutex:-
https://en.cppreference.com/w/cpp/thread/shared_timed_mutex
std::try_lock:-
https://en.cppreference.com/w/cpp/thread/try_lock
std::try_lock:-
https://en.cppreference.com/w/cpp/thread/try_lock

## Producer Consumer Problem

Bookmark this page

Let's recall producer consumer scenario

- Common data set accessible to multiple threads (shared resource)
- Producers add elements to data set
- Consumers remove elements from data set
- Rules

1. Consumer should block on empty data set
2. Producer should block on full data set (Applicable in case of finite capacity)
3. On partial data set, either producer or consumer (only one) should access the data set

- Here Rule-3 can be implemented through mutual exclusion (Semaphores/Mutex/Spinlocks)
- Rule 1 & 2 comes under synchronization/sequencing/dependencies
- Condition variables are of best choice to implement such synchronization
- Since they closely work with mutex, deadlocks can be prevented.

## condition variables

Bookmark this page

A simple example,which demonstrates synchronization between two threads using condition variables as follows

- First thread allowed executed initially
- No thread is allowed to execute for more than 5 iterations at once, i.e. on executing 5 instructions each thread should block and allow other thread to run.
- Both the threads shouldn't execute the iterations concurrently

```
#include<thread>
#include<mutex>
#include<iostream>
#include<unistd.h>
#include<atomic>
#include<condition_variable>

std::mutex m1;
std::condition_variable cv;
std::atomic<int> k(0);


void do_service_producer() {
  std::cout << "do_service_producer called\n";
  for(int i=1;i<=25;i++) {
    std::unique_lock<std::mutex> lk(m1);
    cv.wait(lk, [] () { return k<5; });
    std::cout << "[PRODUCER], produce i:" << i << "\n";
    k++;
    if(k==4) {
      lk.unlock();
      cv.notify_one();
      std::cout << "[PRODUCER], produce data count 5"<< "\n";
      std::cout << "[PRODUCER], release lock for consumer"<< "\n";
    }
  }
}
void do_service_consumer() {
  std::cout << "do_service_consumer called\n";

  for(int i=1;i<=25;i++) {
    std::unique_lock<std::mutex> lk(m1);
    cv.wait(lk, [] () { return k>0; });
    k--;
    std::cout << "[CONSUMER], consme i:" << i << "\n";
    if(k==0) {
      lk.unlock();
      cv.notify_one();
      std::cout << "[CONSUMER], consume data count 5"<< "\n";
      std::cout << "[CONSUMER], release lock for producer"<< "\n";
    }
```

```
  }
}

int main() {
  std::thread t1(do_service_producer);
  std::thread t2(do_service_consumer);
  t1.join();
  t2.join();
  return 0;
}
```

```
do_service_producer called
[PRODUCER], produce i:1
[PRODUCER], produce i:2
[PRODUCER], produce i:3
[PRODUCER], produce i:4
[PRODUCER], produce data count 5
[PRODUCER], release lock for consumer
[PRODUCER], produce i:5
do_service_consumer called
[CONSUMER], consme i:1
[CONSUMER], consme i:2
[CONSUMER], consme i:3
[CONSUMER], consme i:4
[CONSUMER], consme i:5
[CONSUMER], consume data count 5
[CONSUMER], release lock for producer
[PRODUCER], produce i:6
[PRODUCER], produce i:7
[PRODUCER], produce i:8
[PRODUCER], produce i:9
[PRODUCER], produce data count 5
[PRODUCER], release lock for consumer
[PRODUCER], produce i:10
[CONSUMER], consme i:6
[CONSUMER], consme i:7
[CONSUMER], consme i:8
[CONSUMER], consme i:9
[CONSUMER], consme i:10
```

## Stack Example

Bookmark this page

Here is another example on synchronized stack operations as follows

- Pushing thread will block if stack is full, until some space is made by some pop operations.
- Popping thread will block if stack is empty, until some elements are added by push operations
- Concurrent push, pop operations are not allowed as they can lead to corruption of underlying data set and cusror (i.e. array and top index).

```
#include<iostream>
#include<thread>
#include<mutex>
#include<condition_variable>
#include<unistd.h>
class Stack {
  int *arr;
  int maxlen;
  int top;
  std::mutex m1;
  std::condition_variable cv;
```

```
  public:
  Stack(int len):maxlen(len),top(-1) {
    arr=new int[maxlen];
  }
  ~Stack() {
    delete[] arr;
  }
  void push(int val) {
    std::cout << "pushing : " << val << "\n";
    std::unique_lock<std::mutex> lk(m1);
    cv.wait(lk, [this] { return !isfull(); });
    arr[++top]=val;
    cv.notify_one();
  }
  int pop() {
    std::unique_lock<std::mutex> lk(m1);
    cv.wait(lk, [this] { return !isempty(); });
    int val=arr[top--];
    cv.notify_one();
    return val;
  }
  bool isempty() const { return top==-1; }
  bool isfull() const { return top==maxlen-1; }
};
```

```
int main() {
  Stack s1(5);
  std::thread tprod ( [&s1]() {
    for(;;) {
      s1.push(std::rand()%100);
      usleep(100);
    }
  });
  std::thread tcons ( [&s1]() {
    for(;;) {
      std::cout << "popped val:" << s1.pop() << "\n";
      usleep(100);
    }
  });
  tprod.join();
  tcons.join();
  return 0;
}
```

**Tutorials:-**
https://thispointer.com//c11-multithreading-part-7-condition-variables-explained/
https://thispointer.com//c11-multithreading-part-6-need-of-event-handling/
**References:-**
https://en.cppreference.com/w/cpp/thread/condition_variable
http://www.cplusplus.com/reference/condition_variable/condition_variable/

# Future & Promise

Bookmark this page

- std::promise instances are used to pass arguments or handle exceptions across the threads
- A thread will create std::promise object and get the future of it (**get_future** member). So it's a promise to update the object state in future.
- Promise object can be passed to another thread, who is supposed to update the state.
- First thread can wait for the update on promise object state via **set_value**
- Once the object is updated first thread can retrieve the updated state **via** get member
- So this can be considered as high level implementation of condition variables and mutex when object state to be reflected across the threads.
- An empty void future/promise can be used to signalling between the threads (one shot event communication)

Simple example on object state update:-

```
void doSomething (int a,int b,std::promise<int> p) {
  int res;
  res=a/b;
  //std::this_thread::sleep_for(std::chrono::milliseconds(5000));
  p.set_value(res);
}
int main() {
  std::promise<int> p;
  std::future<int> f=p.get_future();
  std::thread t1(doSomething, 10,2,std::move(p));
  t1.detach();
  std::cout << "result is " << f.get() << "\n";
  //t1.join();     //required if not detached
  return 0;
}
```

- Here **f.get** blocks till the completion of **p.set_value,** you can check this by un commenting sleep statement intentionally introduced for some delay
- This even works on detached threads

```
void doSomething (int a,int b,std::promise<int> p) {
  int res;
  res=a/b;
  std::cout << "[CHILD THREAD], Going to sleep"<< "\n";
  std::this_thread::sleep_for(std::chrono::milliseconds(5000));
  std::cout << "[CHILD THREAD], complete the promise for future object"<< "\n";
  p.set_value(res);
}
int main() {
  std::promise<int> p;
  std::future<int> f=p.get_future();
  std::cout << "[MAIN THREAD], create a thread to fullfil promise "<< "\n";
  std::thread t1(doSomething, 10,2,std::move(p));
  //t1.detach();
  std::cout << "[MAIN THREAD], Wait for promised to be fullfilled "<< "\n";
  std::cout << "[MAIN THREAD] promise fullfilled, result is " << f.get() << "\n";
  std::cout << "[MAIN THREAD], Waiting for child thread to complete "<< "\n";
  t1.join();     //required if not
  return 0;
}
```

```
[MAIN THREAD], create a thread to fullfil promise
[MAIN THREAD], Wait for promised to be fullfilled
[CHILD THREAD], Going to sleep
[CHILD THREAD], complete the promise for future object
[MAIN THREAD] promise fullfilled, result is 5
[MAIN THREAD], Waiting for child thread to complete
```

## Exception Handling Example

Bookmark this page

Futures can also pass the exceptions occurred at thread level

```
#include<thread>
#include<future>
#include<iostream>
#include<chrono>

void doSomething (int a,int b,std::promise<int> p) {
  int c;
  try {
    if(b==0)
       throw std::runtime_error("div by zero");
      c=a/b;
    p.set_value(c);
  }catch(std::exception& e) {
    p.set_exception(std::current_exception());
  }
}
```

```
int main() {
  try {
    std::promise<int> p;
    std::future<int> f=p.get_future();
    std::thread t1(doSomething, 10,0,std::move(p));
    t1.detach();
    std::cout << "result is " << f.get() << "\n";
    //t1.join();     //what if thread got terminated due to exception?
  }catch(const std::exception& e) {
    std::cerr << e.what() << "\n";
  }catch(...) {
    std::cerr << "generic exception\n";
  }
  return 0;
}
```

## void futures for event signalling

Bookmark this page

An empty promise & future can be used to synchronize between threads (one shot event signalling)

```
#include<thread>
#include<future>
#include<iostream>
#include<chrono>

void do_work(std::promise<void> barrier)
{
  std::cout << "do some work\n";
  std::this_thread::sleep_for(std::chrono::milliseconds(2000));
  barrier.set_value();
  std::this_thread::sleep_for(std::chrono::milliseconds(3000));
  std::cout << "some more work\n";
}
```

```
int main()
{
  std::promise<void> barrier;
  std::future<void> barrier_future = barrier.get_future();
  std::thread t1(do_work, std::move(barrier));
  barrier_future.wait();                //???
  std::cout << "barrier crossed\n";
  t1.join();
  std::cout << "Thread got completed\n";
  return 0;
}
```

Click to add Bookmark this page

**Suggested Reading from Effective Modern C++ by Scott Meyers**

**Item 39: Consider void futures for one-shot event communication**

**Tutorial:-**

https://thispointer.com//c11-multithreading-part-8-stdfuture-stdpromise-and-returning-values-from-thread/

http://www.modernescpp.com/index.php/promise-and-future

**References:-**
https://en.cppreference.com/w/cpp/thread/promise
http://www.cplusplus.com/reference/future/promise/

std::timed_mutex:-
https://en.cppreference.com/w/cpp/thread/timed_mutex
std::recursive_mutex:-
https://en.cppreference.com/w/cpp/thread/recursive_mutex
std::recursive_timed_mutex:-
https://en.cppreference.com/w/cpp/thread/recursive_timed_mutex
std::shared_lock:-
https://en.cppreference.com/w/cpp/thread/shared_lock
std::shared_timed_mutex:-
https://en.cppreference.com/w/cpp/thread/shared_timed_mutex
std::try_lock:-
https://en.cppreference.com/w/cpp/thread/try_lock
std::shared_future:-
http://www.modernescpp.com/index.php/promise-and-future

## clang-format

Bookmark this page

Install clang-format on a ubuntu machine

```
sudo apt install clang-format
```

Format source code as per google style guidelines (default rules)

```
clang-format -i *.cpp *.h
#-style=Google is implicit for clang-format
```

Source formatting with custom style

```
clang-format  -style=Google -dump-config  > .clang-format
#Customize .clang-format and keep  in your project dir
#Now clang-format follows custom rules
```

## Static Analysis - cppcheck

Bookmark this page

Install cppcheck on a ubuntu machine:-

```
sudo apt install cppcheck
```

Cppcheck rules:-

Ref:- https://sourceforge.net/p/cppcheck/wiki/ListOfChecks/

We can also list these rules as follows

```
cppcheck --doc
```

Source code validation with cppcheck

```
cppcheck --enable=all --std=c++11 *.cpp
#use -I option if header files are in a different location
#Usage of --check-config, --xml options
```

## Heap Analysis - Valgrind

Bookmark this page

Recall the possible memory leaks & heap issues in various examples we encountered in earlier sections, Analyze these examples with valgrind.

- Omission of delete on dynamically allocated objects
- Default destructor (compiler provided) for non trivial classes.
- Destructor is not virtual and the derived class is a non trivial class, delete is called on upcasted pointer.
- Direct vs Indirect memory leaks
- Mismatch in new/new[] vs delete/delete[]
- Read/Write access beyond heap block size or after delete
- Double free problem , e.g. No user defined copy constructor for MyString class.
- STL container holding normal pointers (not a smart pointer) and cleanup code is missing towards end.
- No memory leaks when smart pointers are involved.

### Hint:-

- Append **-g** option to compilation step for debug friendly analysis

## Simple Example

Bookmark this page

**Scenario-1:-**

Assume your project has three files *account.h, account.cpp, bank.h, bank.cpp, test.cpp*

Manual build steps to compile individual source files and link them

```
g++ bank.cpp -c            #includes bank.h, account.h
g++ account.cpp -c         #includes account.h
g++ main.cpp -c            #includes bank.h, if required account.h
g++ bank.o account.o main.o -o bank-demo
```

Makefile approach:-

```
bank-demo:bank.o account.o main.o
     g++ bank.o account.o main.o -o bank-demo
bank.o:bank.cpp bank.h account.h
     g++ bank.cpp -c
account.o:account.cpp account.h
     g++ account.cpp -c
main.o:main.cpp bank.h          #account.h
     g++ main.cpp -c
clean:
     rm -rf *.o demo
```

To build the code

```
make
make clean
```

---

Improved Makefile

```
TARGET=bank-demo
OBJS=bank.o account.o main.o
$(TARGET):$(OBJS)
     g++ $^ -o $@
# main.o, bank.o, account.o can be built using implicit rules
clean:
     rm -rf $(OBJS) $(TARGET)
```

**Some observations:-**

- Makefile is a collection of rules
- Each rule consists of minimum two lines, where first line holds target/outcome and dependencies seperated by : and second line holds actual command to generate the target
- Second line of each rule must start with tab strictly
- Build starts with first rule by default, unless an explicit argument is passed to make command, for e.g. *make clean* jumps to rule whose target is *clean*
- Special variables used - $@, $^, $< etc.
- Pattern based rules, for e.g. %.o

**Scenario-2:-**

Let's assume we have following files in a project with certain header file dependencies.

- main.cpp
- weather.cpp, weather.h
- sensor.cpp, sensor.h
- temperature.cpp, temperature.h
- humidity.cpp, humidity.h
- pressure.cpp, pressure.h

---

**Makefile:-**

```
TARGET =  weather-demo
OBJS   := sensor.o temperature.o humidity.o pressure.o
OBJS   += main.o weather.o

all:$(TARGET)

$(TARGET):$(OBJS)
     g++ $^ -o $@
%.o:%.cpp %.h sensor.h
     g++ $< -c
weather.o:weather.cpp weather.h sensor.h temperature.h humidity.h pressure.h
     g++ $< -c
main.o:main.cpp weather.h
     g++ $< -c
clean:
     rm -rf $(TARGET) $(OBJS)
```

**TODO:-**

- Identify rebuilding when certain source file is modified
- Identify rebuilding when a common header file like sensor.h is modified

## Static Libraries

Bookmark this page

Discussion : Comparison of Static vs Dynamic libraries:-

- Linking approach
- Performance overhead/Execution speed
- Memory footprint, sharable code
- Versioning of libraries, flexible upgrades

Steps for static library creation and linking:-

**Scenario-1:-**

```
#generate object files by compiling specific source files
ar rc libbanking.a bank.o account.o main.o
g++ -L. main.o -o s1.out -lbanking
g++ -L. main.o -o s2.out -lbanking -static
```

**Scenario-2:-**

```
#let's assume necessary object files are generated
ar rc libweather.a weather.o sensor.o temperature.o humidity.o pressure.o
g++ -L. main.o -o s1.out -lweather
g++ -L. main.o -o s2.out -lweather -static
```

**TODO/Hands-on:-**

- Writing Makefiles to generate static libraries and linking
- Usage of -L option if .a files are stored in a different directory (other than current dir)
- Comparison of s1.out s2.out using following commands/utils
  - file, size, ls, ldd

## Shared Libraries

Bookmark this page

Dynamic Linking - Shared Object Files

**Scenario-1:-**

```
g++ -shared -o libbanking.so bank.o account.o
g++ -L. main.o -o d1.out -lbanking
LD_LIBRARY_PATH=.  ./d1.out
```

**Scenario-2:-**

```
g++ -shared -o libweather.so *.o      #except main.o
g++ -L. main.o -lweather -o d1.out
LD_LIBRARY_PATH=.  ./d1.out
```

## TODO/Hands-on:-

- Makefile approach for above steps
- Usage of commands/utils
  - file, ls, size, ldd
- Versioning of .so files
- Adding entries to /etc/ld.so.conf
- ldconfig tool

## Comparison

Bookmark this page

**TODO/Hands-on:-**

Check whether given executable is statically or dynamically linked.

```
file s1.out
file s2.out
file d1.out
```

Compare the footprint of statically & dynamically linked executables

```
ls -lh s1.out s2.out d1.out
(or)
du -sh s1.out s2.out d1.out
```

Listing the shared object dependencies for dynamically linked executables

```
ldd s1.out
ldd d1.out
ldd s2.out            //???
```

ELF Header Analysis:-

```
readelf s1.out s2.out d1.out
```

If necessary so files are in ~/dlibs, How do you run dynamically linked executables

```
#Method-1
LD_LIBRARY_PATH=~/dlibs    ./d1.out

#Method-2
#Adding ~/dlibs to /etc/ld.so.conf and updating so cache with ldconfig
```

# API Documentation - Doxygen

Bookmark this page

Install the ubuntu packages:-

```
sudo apt install doxygen doxygen-gui
```

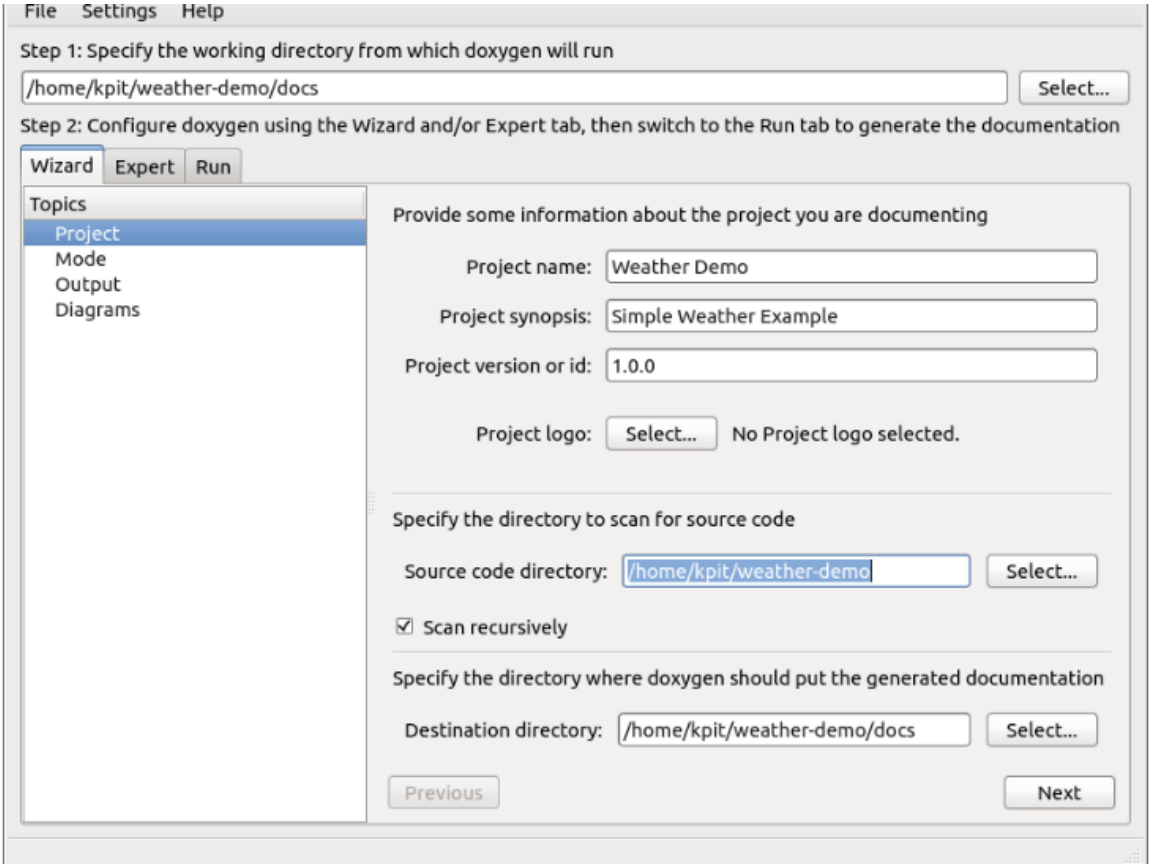
If you have a hand written or auto generated configuration file

```
doxygen my-config-file
doxygen                 #default config file will be Doxyfile
```

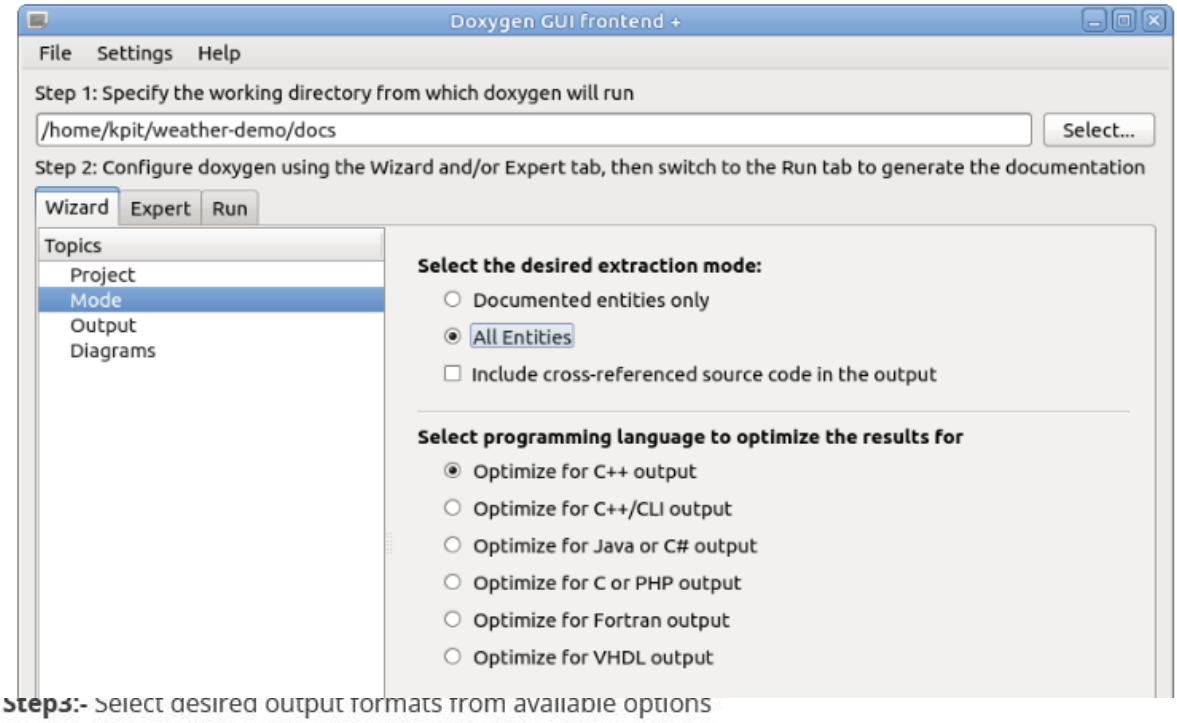
To generate the configuration file through a friendly wizard:-

```
doxywizard
```

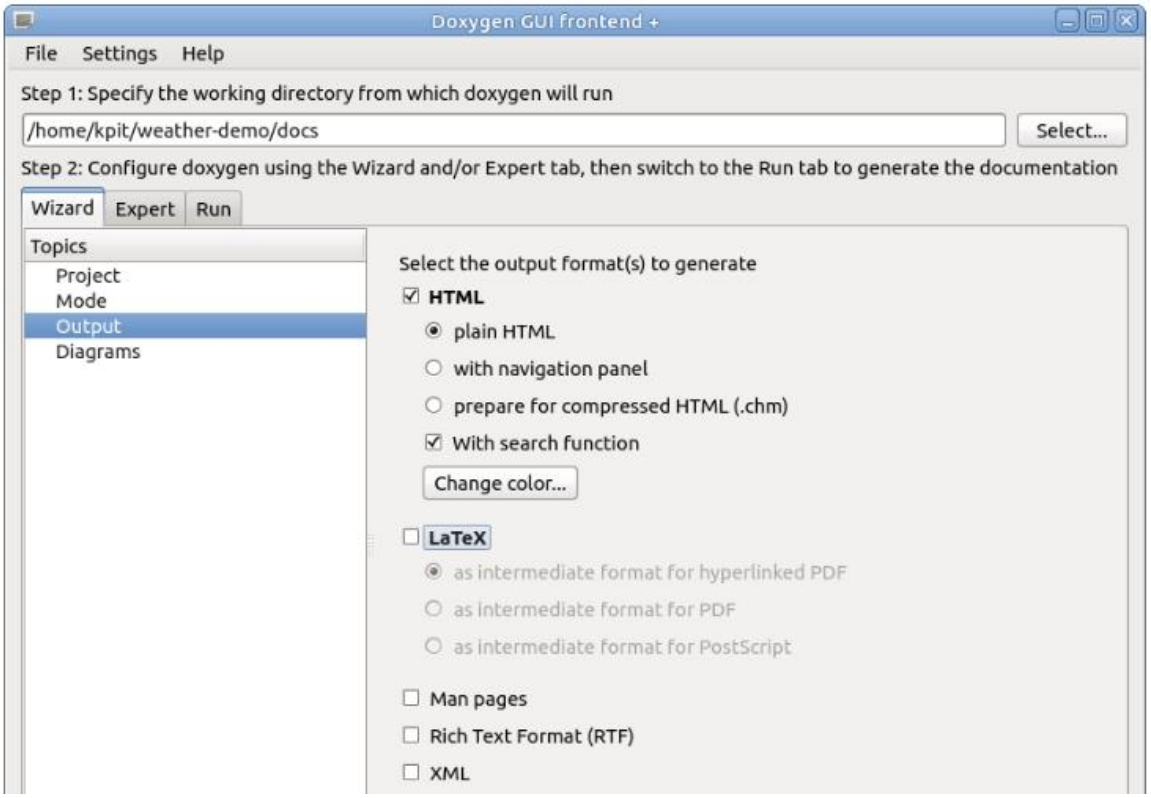
**Step1:-** Select working directory, source directory, destination directory. You can also add Project name, synopsis and version id.



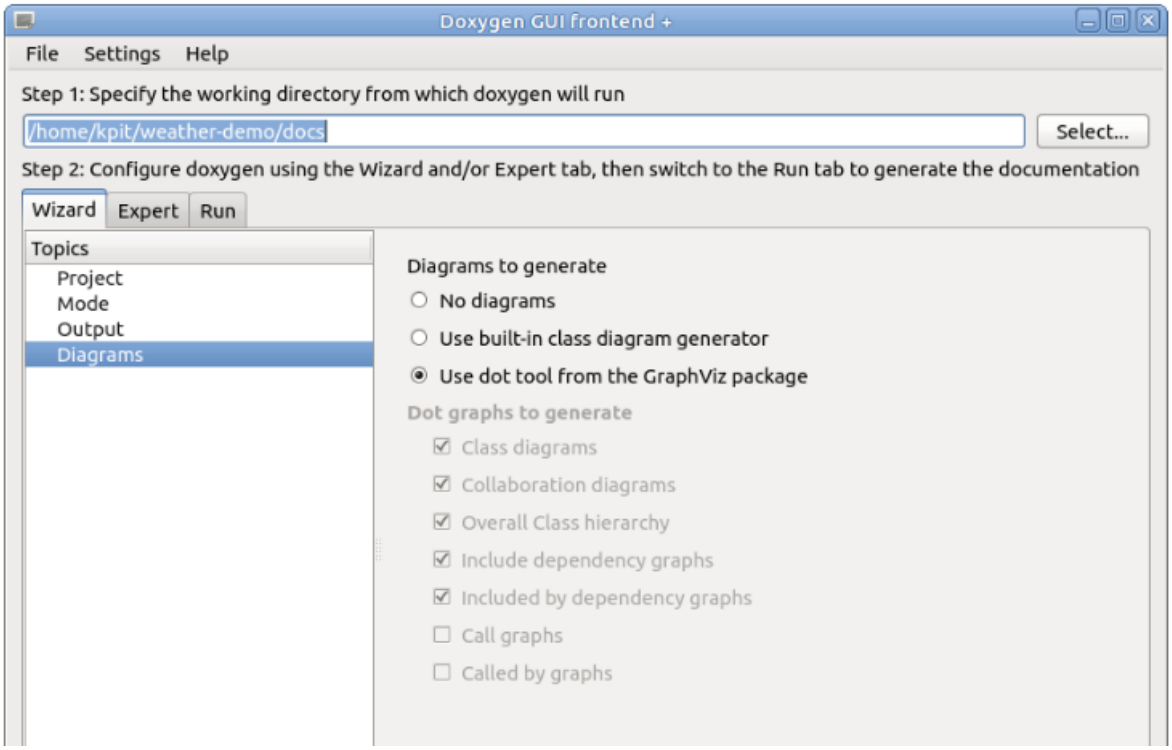
Step2:- Select the extract mode to "All Entities", leave optimize options to default




**Step3:-** Select desired output formats from available options



Step4:- Options for diagram generation




Step5:- Here you can go any of the following three options

- Hit show configuration and manually copy the content
- Hit save log, to store the config in a file under suitable dir, say docs/Doxyfile
- Hit Run doxygen to generate output without storing configuration file, in this case you can also go for "Show HTML output" to view generated pages.

**Writing C++ Comments for doxygen:-**

https://en.wikipedia.org/wiki/Doxygen
https://www.stack.nl/~dimitri/doxygen/manual/docblocks.html#cppblock
https://www.cs.cmu.edu/~410/doc/doxygen.html

# Overview

Bookmark this page

In this section i assume that you are familiar with basic build process in Jenkins

Suggested Plugins for C/C++ development:-

- cppcheck
- valgrind
- Warnings
- xUnit
- cccc
- Doxygen
- CMake

Ref:- https://jenkins.io/solutions/c/

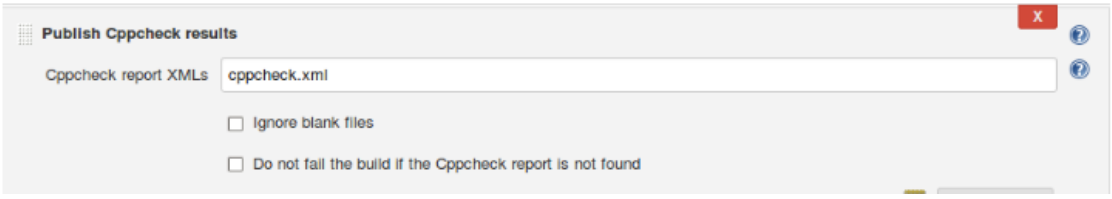
## cppcheck

Bookmark this page

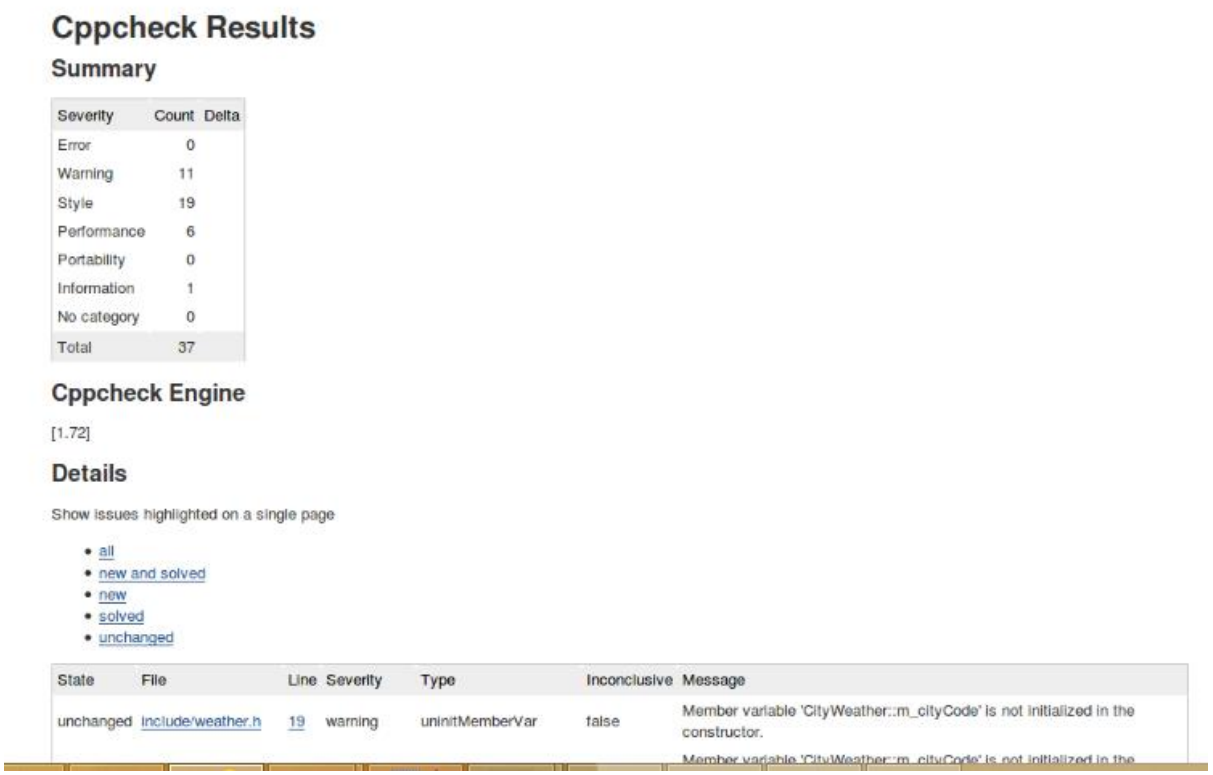
Add following build step




Add post build action



Cppcheck results looks like



## Cppcheck Results

### Summary

| Severity | Count | Delta |
|---|---|---|
| Error | 0 | |
| Warning | 11 | |
| Style | 19 | |
| Performance | 6 | |
| Portability | 0 | |
| Information | 1 | |
| No category | 0 | |
| Total | 37 | |

### Cppcheck Engine

[1.72]

### Details

Show issues highlighted on a single page

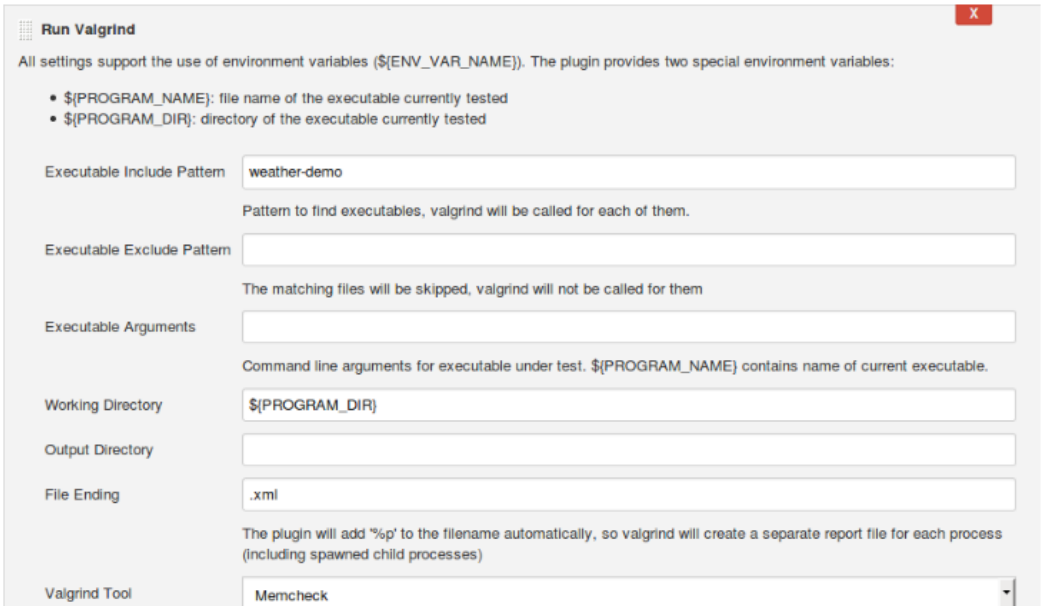
- all
- new and solved
- new
- solved
- unchanged

| State | File | Line | Severity | Type | Inconclusive | Message |
|---|---|---|---|---|---|---|
| unchanged | include/weather.h | 19 | warning | uninitMemberVar | false | Member variable 'CityWeather::m_cityCode' is not initialized in the constructor. |

---

**Build/Pre Step:-**

Please fill final target name under ***Executable Include Pattern***, say ***weather-demo*** for the sample project, all valgrind results will be stored in a xml file.



**Run Valgrind**

All settings support the use of environment variables (${ENV_VAR_NAME}). The plugin provides two special environment variables:

- ${PROGRAM_NAME}: file name of the executable currently tested
- ${PROGRAM_DIR}: directory of the executable currently tested

Executable Include Pattern: weather-demo

Pattern to find executables, valgrind will be called for each of them.

Executable Exclude Pattern:

The matching files will be skipped, valgrind will not be called for them

Executable Arguments:

Command line arguments for executable under test. ${PROGRAM_NAME} contains name of current executable.

Working Directory: ${PROGRAM_DIR}

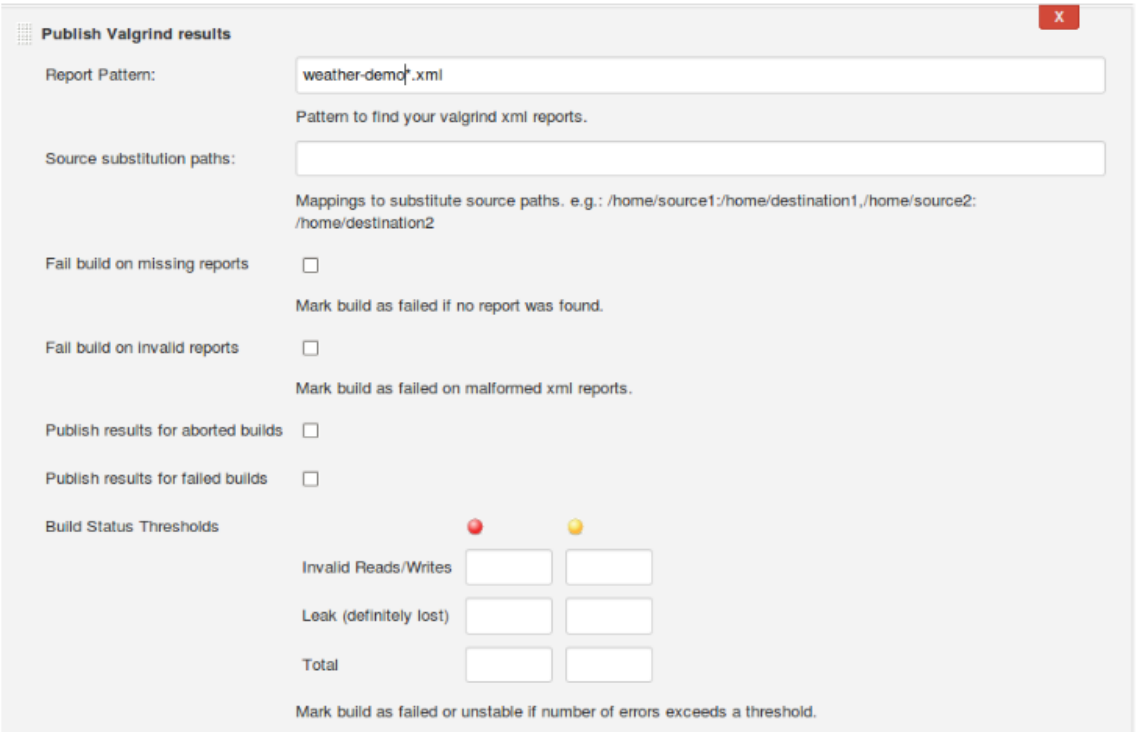
Output Directory:

File Ending: .xml

The plugin will add '%p' to the filename automatically, so valgrind will create a separate report file for each process (including spawned child processes)

Valgrind Tool: Memcheck

**Post Build Action Step:-**




## Lambdas in place of slots

Bookmark this page

Example 1:-

```
connect(timer, SIGNAL(timeout()), this, SLOT(onTimedout());
void MyController::onTimedout() {
  this->lcdDisplay->display(QString::from(k));
}
```

Qt5 introduced new connect syntax using function pointers and callback need not be defined as slot in class.This allows lambdas as callback for signal as shown below

```
connect(timer, &QTimer::timeout, [this]() {
  this->lcdDisplay->display(now.toString("hh:mm:ss"));
});
```

Example 2:-

```
connect(slider, SIGNAL(valueChanged()), progressBar, SLOT(setValue(int));
```

Using C++11:-

```
connect(slider,&QSlider::valueChanged,[this](int val) {
   progressBar->setValue(val);
});
(or)
connect(slider,&QSlider::valueChanged,[&](int val) {
   progressBar->setValue(val);
});
```

Example 3:-

```
connect(timer,&QTimer::timeout,MyDialog::updateDisplay());
void MyDialog::updateDisplay() {
  ui->lcdNumber->display(QString::number(k++);
}
```

This can be rewritten as

```
connect(timer,&QTimer::timeout,[this]() {
  ui->mylcdNumber->display(QString::number(k));
  k++;
});
//(or)
connect(timer,&QTimer::timeout,[&]() {
  ui->mylcdNumber->display(QString::number(k));
  k++;
});
```

Suggested Reading:-
https://wiki.qt.io/New_Signal_Slot_Syntax
https://woboq.com/blog/new-signals-slots-syntax-in-qt5.html

## New Macros

Bookmark this page

Qt5 introduced some new macros in align with c++11 keywords, here are few examples.From Qt 5.7 most of these macros can be ignored as C++11 support was added to Qt by default.

**Q_DECL_CONSTEXPR:-**

Qt5 introduced  to specify expressions which can be constructed at compile time.

This macro expands to constexpr as per the compiler support

**Q_DECL_RELAXED_CONSTEXPR:-**

To support relaxed constexpr rules for inline functions introduced in C++14

**Q_DECL_OVERRIDE, Q_DECL_FINAL:-**

To represent override, final keywords

**Q_DECL_NOEXCEPT, Q_DECL_NOEXCEPT_EXPT:-**

to represent noexcept, noexcept(expr)

**Q_FOREACH:-**

this macro was used in prior to Qt5.7 as a replacement for range based for loops(when C++11 wasn't supported by default).

**Containers & Algorithms:-**

Usage of STL is recommended in newer versions of Qt due to default support for C++11.

**Reference:-**

http://doc.qt.io/qt-5/qtglobal.html

# GENIVI Projects

**vsomeip library**

https://github.com/GENIVI/vsomeip/

Identify following modern cpp features in following examples

- Examples
  - examples/hello_world/hello_world_service.cpp
  - examples/hello_world/hello_world_client.cpp
  - examples/request-sample.cpp

- examples/response-sample.cpp
- examples/notify-sample.cpp
- examples/subscribe-sample.cpp
- Modern C++ Features
- Usage of std::bind and std::placeholders
- std::thread instances
- std::this_thread::sleep_for, std::chrono::duration
- std::mutex,std::condition_variable
- std::lock_guard, std::unique_lock
- std::shared_ptr
- nullptr
- **TODO**
- Is it possible to replace bind & placeholders with lambdas while registering callbacks

#### Common API libs

- Examples / test Cases
- https://github.com/GENIVI/capicxx-dbus-runtime ==> src/test
- Identify following C++ Features
- auto keyword
- Nested templates >>
- lambdas, std::bind and std::placeholders
- std::shared_ptr, std::make_shared
- std::tuple, std::make_tuple, std::get
- std::thread, std::mutex, std::unique_lock, std::condition_variable
- std::chrono::*
- std::future, std::promise (std::async at backend)
- std::move
- Google Test Macros
- In Source code
- https://github.com/GENIVI/capicxx-dbus-runtime ==> src/CommonAPI/DBus
- TODO

#### JSON for Modern C++

You can use following json library for encoding decoding.
https://github.com/nlohmann/json
This is a header only library, so it's enough to include single_include/nlohmann/ json.hpp from above repository.
For usage of this library please follow the documentation available at above github repository

**Disclaimer:-** The steps provided in these links may not work out of the box due to variations in google test versions and IDE being used, please do necessary fine tuning and consider the content as hints only.

- https://www.ibm.com/developerworks/aix/library/au-googletestingframework.html
- http://www.yolinux.com/TUTORIALS/Cpp-GoogleTest.html
- http://www.bogotobogo.com/cplusplus/google_unit_test_gtest.php
- https://blog.jetbrains.com/rscpp/unit-testing-google-test/
  (Video:- https://www.youtube.com/watch?v=IsvbXMoG_ew)

- https://www.ics.uci.edu/~pattis/common/modules46/googletestpc.html

**Videos:-**

By Bi Qian (boqian.weebly.com)

- https://www.youtube.com/watch?v=TS2CTf11k1U (1-setup:netbeans)
- https://www.youtube.com/watch?v=n1OL800IcXI (setup:visual studio)
- https://www.youtube.com/watch?v=CQ4TBwewKU4 (Basic Macros)
- https://www.youtube.com/watch?v=iyDAFpcEy4c (Test Fixtures)
- https://www.youtube.com/watch?v=GvBPPkOsseE (Controlling test cases)
- https://www.youtube.com/watch?v=HNu-Rsno_wU (Death & Exception cases)

Other videos:-

- https://www.youtube.com/watch?v=nbFXI9SDfbk
- https://www.youtube.com/watch?v=x5_UxQ9wrH4
- https://www.youtube.com/watch?v=vYjGRzIAipQ
- https://www.youtube.com/watch?v=y9sGAF1k63o
- https://www.youtube.com/watch?v=16FI1-d2P4E

# Setting up Google Test Framework

Google Test is not a header only framework, a static library need to be linked with the applications along with header file inclusion

**Building the library:-**

Downlaod googletest source archive from https://github.com/google/googletest

```
git clone https://github.com/google/googletest
cd googletest
mkdir build && cd build
cmake ..
make
#Identify generated static libs in build/googlemock/gtest,
#which are libgtest.a, libgtest_main.a
```

**Setting up first example:-**

Place the files as follows

- Copy **gtest** subdir from **googletest/googletest/include** into **include** subdir (i.e. copy whole include subidr from googletest/googletest to current dir,which holds **gtest** subdir)

- Create a new dir **lib** and copy the generated **libgtest.a**, **libgtest_main.a** (from googletest/build/googlemock/gtest)
- Place your c++ code to be tested in src subdir
- Place your c++ code, with test cases in test subdir
- Place any  header files supporting your source in include, i.e. parallel to gtest which is just copied.
- Create a simple Makefile with following contents

```
TARGET: hello_gtest
all:hello_gtest
hello_gtest:hello.o hello_test.o lib/libgtest.a
    g++ -isystem ./include -g -Wall -Wextra -pthread -lpthread $^ -o $@
hello.o:src/hello.cc
    g++ -isystem ./include -g -Wall -Wextra -pthread -c $<
hello_test.o:test/hello_test.cxx
    g++ -isystem ./include -g -Wall -Wextra -pthread -c $<
clean:
    rm -rf *.o ${TARGET}
```

- Run the generate executable hello_gtest

You can compare the above directory structure with this sample code.Here is the sample output from above code

```
[==========] Running 3 tests from 1 test case.
[----------] Global test environment set-up.
[----------] 3 tests from LeapYearTest
[ RUN      ] LeapYearTest.InvalidInputs
[       OK ] LeapYearTest.InvalidInputs (0 ms)
[ RUN      ] LeapYearTest.Leap
[       OK ] LeapYearTest.Leap (0 ms)
[ RUN      ] LeapYearTest.NonLeapYears
[       OK ] LeapYearTest.NonLeapYears (0 ms)
[----------] 3 tests from LeapYearTest (1 ms total)

[----------] Global test environment tear-down
[==========] 3 tests from 1 test case ran. (1 ms total)
[  PASSED  ] 3 tests.
```

Along with sample code with zip file, please refer the code used in this example.

**hello.h(include/hello.h):-**

```
#ifndef GTEST_SAMPLES_HELLO_H_
#define GTEST_SAMPLES_HELLO_H_

int IsLeapYear(int);

#endif  // GTEST_SAMPLES_HELLO_H_
```

**hello.cc(src/hello.cc):-**

```

#include "hello.h"

// Returns 1 if given year is a leap year, 0 other wise,
// Returns -1 if input is negative
int IsLeapYear(int yy) {
  if(yy < 0) return -1;
  if(yy % 400 == 0 || (yy % 100 != 0 && yy % 4 == 0))
    return 1;
  else
    return 0;
}
```

**hello_test.cc(test/hello.cc):-**

```
#include <limits.h>
#include "hello.h"
#include "gtest/gtest.h"
namespace {   //anonymous
  TEST(LeapYearTest, InvalidInputs) {
    EXPECT_EQ(-1, IsLeapYear(-1500));
    EXPECT_EQ(-1, IsLeapYear(-1996));
    EXPECT_TRUE(-1==IsLeapYear(-1000));
  }
  TEST(LeapYearTest, LeapYears) {
    EXPECT_EQ(1, IsLeapYear(1996));
    EXPECT_EQ(1, IsLeapYear(2400));
    EXPECT_TRUE(1==IsLeapYear(2016));
  }
  TEST(LeapYearTest, NonLeapYears) {
    EXPECT_EQ(0, IsLeapYear(2018));
    EXPECT_EQ(0, IsLeapYear(1995));
    EXPECT_NE(1, IsLeapYear(1900));
    EXPECT_FALSE(1==IsLeapYear(2100));
  }
}
```

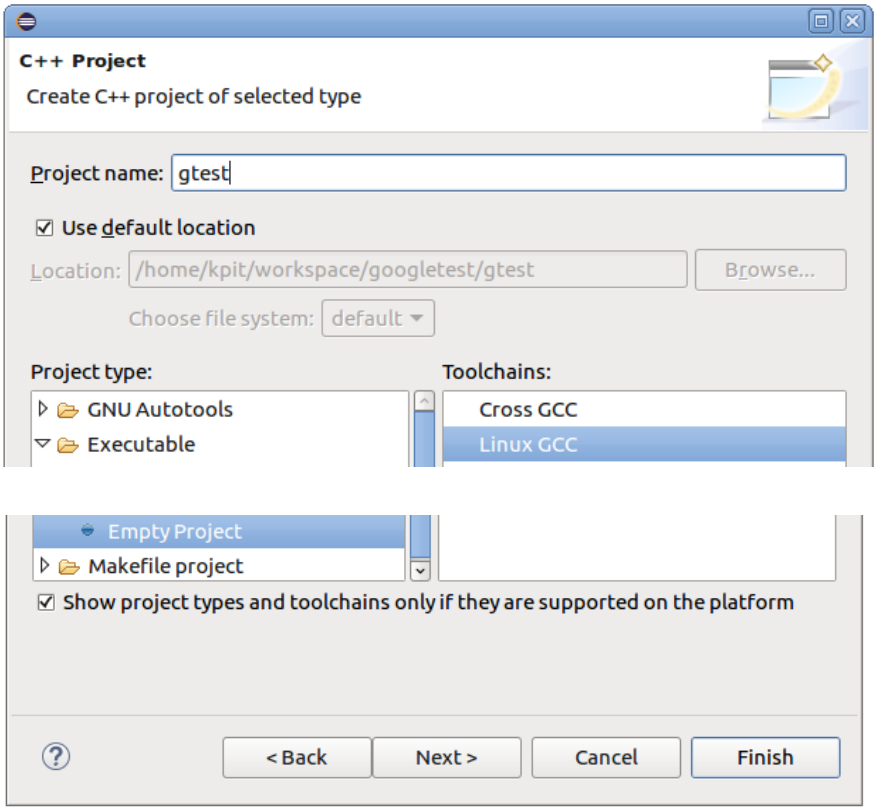
# Eclipse IDE

Bookmark this page

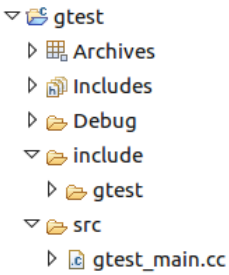
**Disclaimer:-** These steps assume that you have basic familiarity with developing C/C++ applications in eclipse.

**Preparing static library project:-**

- Create a new C++ Project with the name **gtest,** (Choose Static Library in first wizard)




- Copy **include** subdir from googletest/googletest to current project location (which includes sub dir **gtest** )
- Copy **src** subdir from googletest/googletest to current project location
- With this you can expect the project structure as follows



- gtest-death-test.cc
- gtest-filepath.cc
- gtest-internal-inl.h
- gtest-port.cc
- gtest-printers.cc
- gtest-test-part.cc
- gtest-typed-test.cc

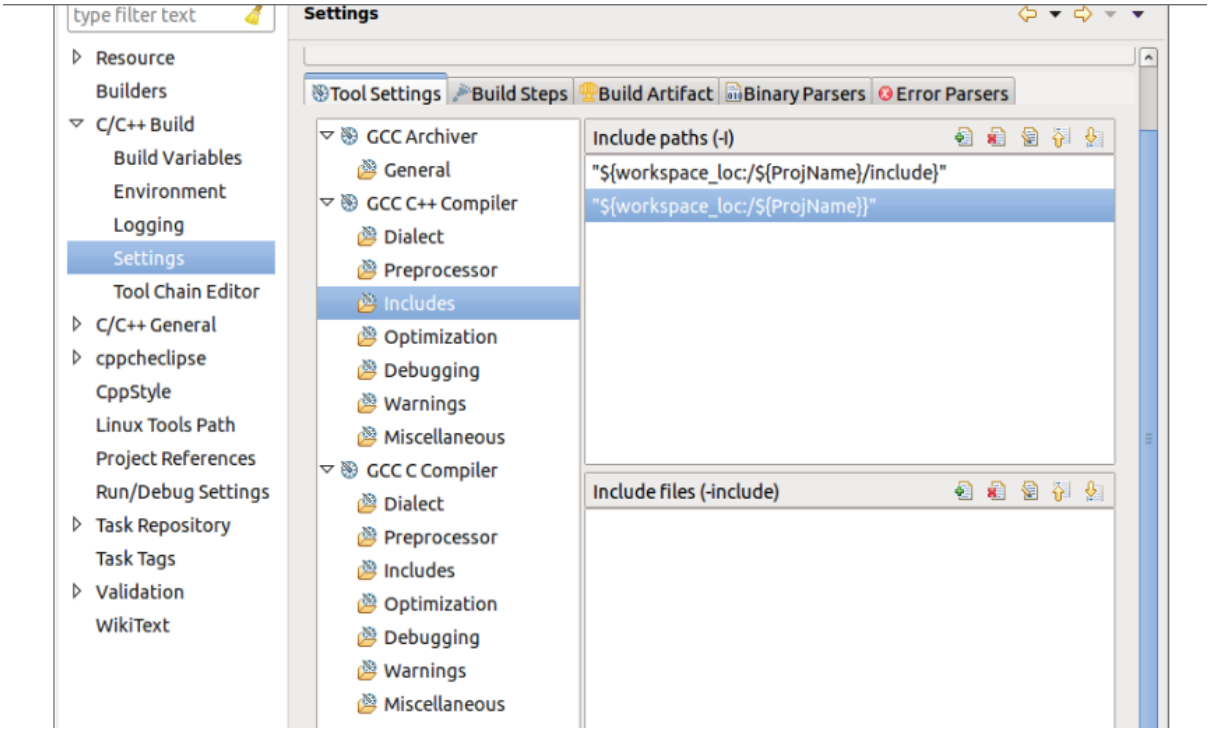
- Go to C/C++ Build Settings of the project as follows

  *Right Click on Project ==> Properties ==> C/C++ Build ==> Settings ==> Tool Settings*

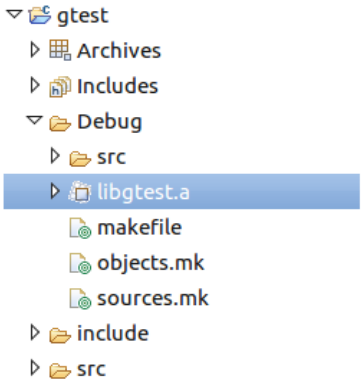
- Add include directories as follows (Under Tool Settings)

  GCC C++ Compiler ==> Include

```
"${workspace_loc:/${ProjName}/include}"
"${workspace_loc:/${ProjName}}"
```

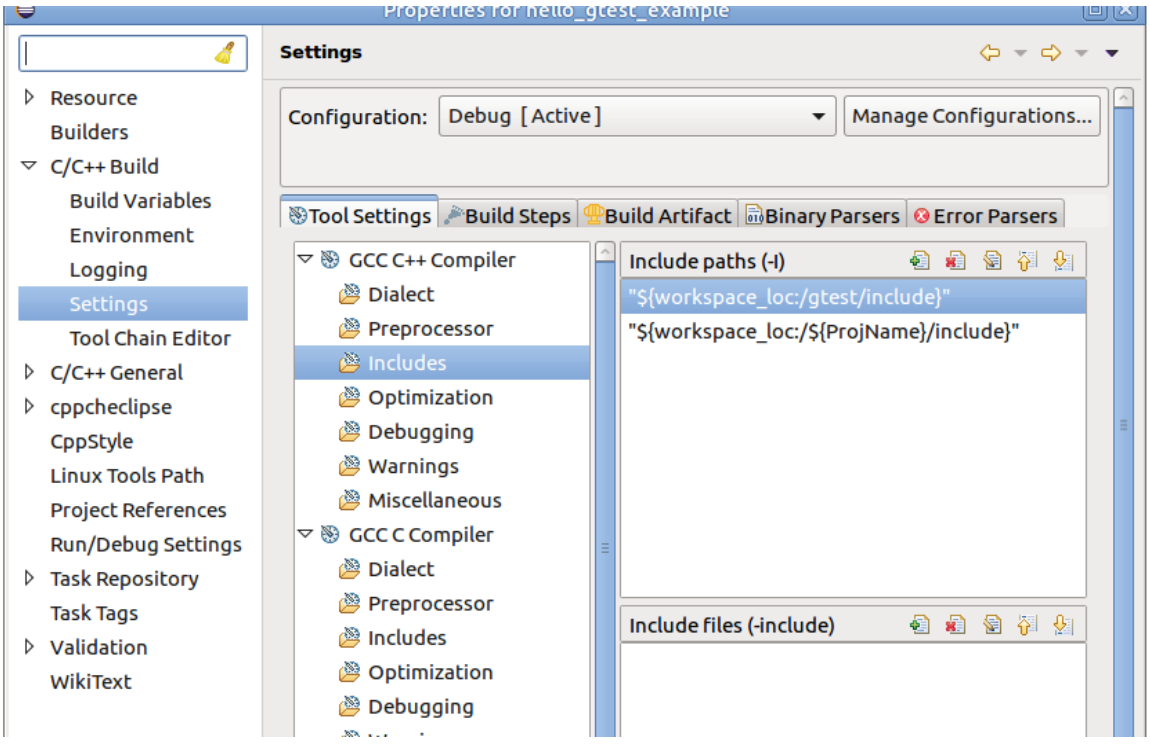


- Build the project and identify generated **libgtest.a** under **Debug** dir



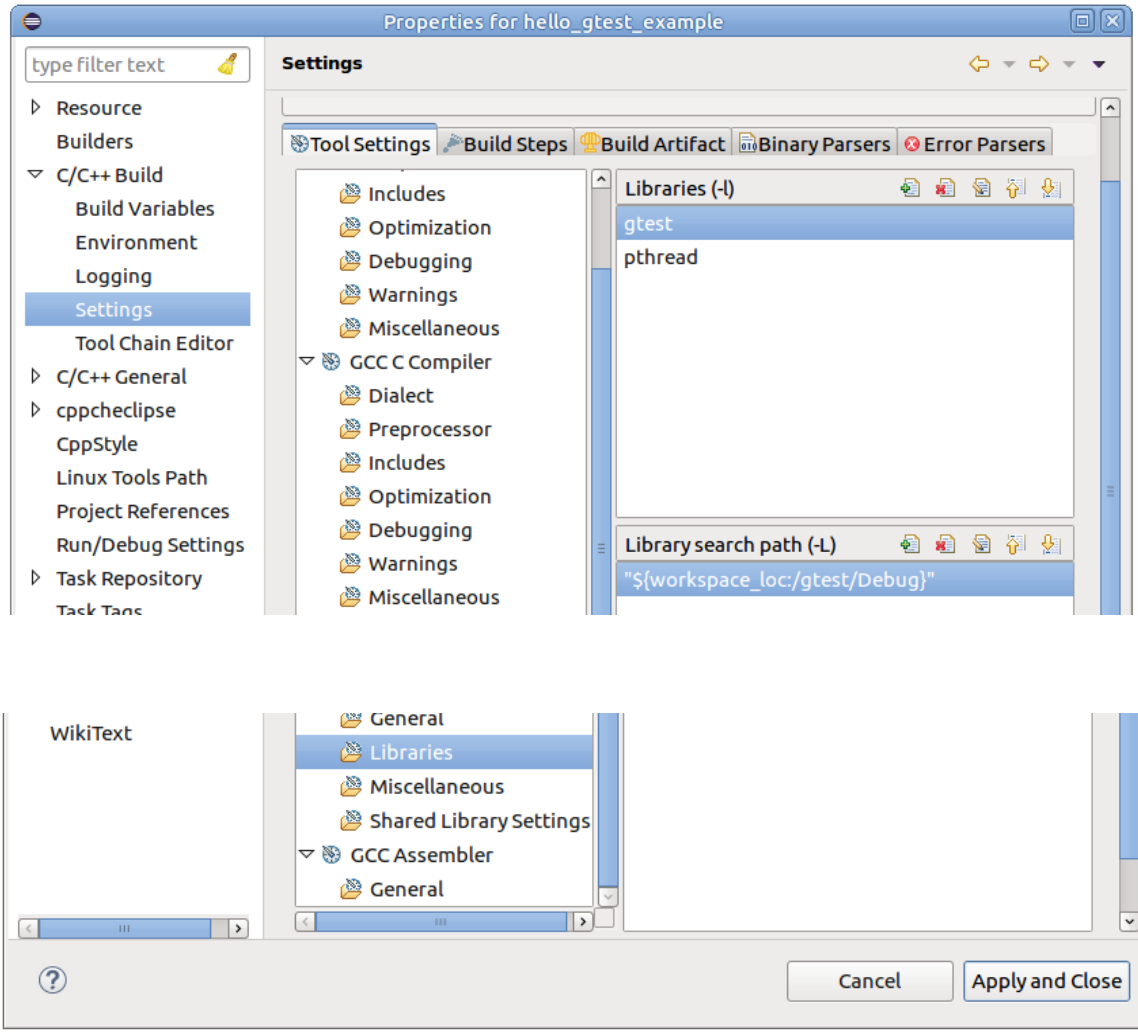

**Creating Application Project (with test cases):-**

- Create a C/C++ application project (Executable in first wizard)
- Place *hello.h, hello.cc, hello_test.cc* under *include, src, test* subdirs respectively
- In C/C++ settings ==> GCC C++ Compiler ==> Includes



- In C/C++ settings ==> GCC C++ Linker ==> Libraries

Add *gtest, pthread* under Libraries(-l), i.e. *libgtest.a, libpthread.**




Add *"${workspace_loc:/gtest/Debug}"* as Library Search Path(-L)

- Build and run project, see the test results in console window

```
Problems  Tasks  Console  Properties  Call Graph
<terminated> (exit value: 0) functions_demo [C/C++ Application] /home/kpit/workspace/
Running main() from ../src/gtest_main.cc
[==========] Running 3 tests from 1 test case.
[----------] Global test environment set-up.
[----------] 3 tests from LeapYearTest
[ RUN      ] LeapYearTest.InvalidInputs
[       OK ] LeapYearTest.InvalidInputs (0 ms)
[ RUN      ] LeapYearTest.Leap
[       OK ] LeapYearTest.Leap (0 ms)
[ RUN      ] LeapYearTest.NonLeapYears
[       OK ] LeapYearTest.NonLeapYears (0 ms)
```

## Basic Macros

Click to add Bookmark this page

**Non fatal assertions:-**

```
EXPECT_EQ
EXPECT_NE
EXPECT_LE
EXPECT_LT
EXPECT_GE
EXPECT_GT
EXPECT_TRUE
```

**Fatal Assertions:**

```
ASSERT_EQ
ASSERT_NE
ASSERT_LE
ASSERT_LT
ASSERT_GE
ASSERT_GT
```

**Boolean assertions:-**

```
EXPECT_TRUE
ASSERT_TRUE
```

**String Comparison:-**

```
EXPECT STREQ,ASSERT STREQ
EXPECT STRNE,ASSERT STRNE
EXPECT STRCASEEQ,ASSERT STRCASEEQ
EXPECT_STRCASENE,ASSERT_STRCASENE
```

**Video tutorial:-**
https://www.youtube.com/watch?v=iyDAFpcEy4c

## gcov usage

Bookmark this page

**Install the utilities:-**

```
#gcov will be already installed if gcc package is installed, if not
sudo apt-get install build-essential
```

**Simple method :-**

```
#go to the directory where your code exists
#add the options -fprofile-arcs -ftest-coverage while building
g++ test.cpp -c -fprofile-arcs -ftest-coverage
#similarly compile other files
g++ *.o -o all.out -fprofile-arcs -ftest-coverage
#identify the generated gcno files
#run the code at least once
./all.out
#identify generated gcda files
gcov -b *.cpp
#identify generated *.cpp.gcov file for coverage analysis
```

Note:- ***--coverage*** *is a synonym for* ***-fprofile-arcs -ftest-coverage***

Optionally, you may add following rules to the Makefile

```
all.out:*.o
    g++ $^ -o $@ -ftest-coverage -fprofile-arcs
%.o:%.cpp
    g++ $< -c -ftest-coverage -fprofile-arcs
gcoverage:run
    gcov -b *.cpp
run:./all.out
    ./all.out
```

## lcov usage

Bookmark this page

**Install the utilities:-**

```
sudo apt-get install lcov
```

**Simple method :-**

```
#go to the directory where your code exists
#add the following to compiler options, linker options
                        -fprofile-arcs -ftest-coverage
#run the code atleast once
lcov -d . -c -o coverage.info
genhtml -o coverage coverage.info
```

Identify generated we pages in coverage sub dir, open index.html using any browser

Optionally, you may add following rules to Makefile

```
lcoverage:run
    lcov -d . -c -o coverage.info -t "results"
    genhtml -o res coverage.info
run:./all.out
    ./all.out
```

# Which one is better pass by value or reference?

# 1) r-value argument (pass by value)

```
void print (Sample r)
{
}

Sample s1;
print(s1);
```

Finding:

1. One copy constructor called due to C++ optimization

2)

```
void print(const Sample& r)
{

}
Sample s1;
print(s1);
```

Finding:

1. One default constructor called due to C++ optimization